

Feita para Desenvolvedores de Software e DBAs

SQL

magazine

Ano 7 :: Edição 74 :: R\$ 13,90  DevMedia

MySQL

Trabalhe na prática com Stored Procedures, Views e Triggers

Engenharia de Software

Introdução à Gerência de Configuração

Oracle

Entenda como funciona o otimizador de consultas

SQL Server e PostgreSQL

Trabalhando com replicação de dados

MODELAGEM NA PRÁTICA

Construa os modelos lógicos e físicos
para um sistema de reserva de carros

Modele um banco de dados para
um sistema de condomínio predial

Brinde ::

Compre esta edição e ganhe **3 vídeo-aulas:**

• Programação Web com MVC - Partes 1 a 3

SQL Server

Trabalhando na prática
com Stored Procedures

Banco de Dados

Questões sobre banco de dados
do concurso do TRE do Paraná

ISSN 1677918-5



Hospede seus sites de forma simples, rápida e **ilimitada**.



Plano Básico

- 1 domínio principal
- 50 domínios hospedados
- Transferência **ilimitada**
- Espaço **ilimitado**
- 10 contas de e-mail
- PHP, ASP e ASP.NET
- Painel de controle Plesk
- Suporte 24hs

Tudo isso por apenas

R\$ **15,90** por mês
pagamento trimestral

Assine agora mesmo e descubra por que Hospedix é a hospedagem de sites preferida entre os profissionais.

- Conheça também nossos planos profissionais com registro de domínio grátis e isenção de taxas anuais enquanto for cliente.



HOSPEDIX

HOSPEDAGEM PROFISSIONAL

www.hospedix.com.br

**30 DIAS
GRÁTIS**

Ao assinar, digite o código de desconto exclusivo para leitores desta revista e ganhe 30 dias inteiramente grátis.

RVSTMEDIA

Sumário

Projeto/ Modelagem

Modelo
Pré-Fabricado

06 – Projeto de Banco de Dados para Condomínios

[Mauro Pichiliani]

Projeto de
Software/
Modelagem

13 – Modelando um Sistema de Reserva de Carros

[Arilo Claudio Dias Neto]

Primeiros Passos em
Banco de Dados

20 – Questões sobre banco de dados do concurso do TRE do Paraná

[Mauro Pichiliani]

Tutorial

28 – Disponibilidade no SQL Server 2005/2008

[Priscila Azarias]

Dia a Dia

40 – Stored Procedures no SQL Server 2008

[Drausio Henrique Chiarotti]

Mão na Massa

46 – Utilizando Stored Procedures, Views e Triggers no MySQL

[Arilo Claudio Dias Neto]

Expert

52 – Statistics no Oracle

[José Eduardo Fiamengui Júnior]

Engenharia de
Software

59 – Introdução à Gerência de Configuração

[Rodrigo Spínola]

Desafio SQL

62 – Desafio SQL

[Wagner Crivelini]

Olá, eu sou o **DevMan**! Desta página em diante, eu estarei lhe ajudando a compreender com ainda mais facilidade o conteúdo desta edição. Será um prazer contar com sua companhia! Confira abaixo o que teremos nesta revista:



Brinde na web desta edição

3
Vídeos

1) Programação Web com MVC-Parte 1 - Desenvolvendo as páginas JSP(com códigos html)

2) Programação Web com MVC-Parte 2 - Finalizando procedimento de login e implementando menu principal da aplicação

3) Programação Web com MVC-Parte 3 - Tabela do banco de dados e o objeto DAO

Para visualizar acesse o link:

<http://www.devmedia.com.br/articles/listcomp.asp?keyword=sql74&codigobanca=trex>

Gostou das vídeo aulas? O portal www.devmedia.com.br possui mais de 2 mil vídeo aulas e dezenas de cursos online sobre desenvolvimento de software! Agora você pode comprar as vídeo aulas que preferir e fazer sua própria combinação de vídeos! Saiba mais em www.devmedia.com.br/creditos

Dê seu feedback sobre esta edição!

A .NET Magazine tem que ser feita ao seu gosto. Para isso, precisamos saber o que você, leitor, acha da revista!

Dê seu voto sobre esta edição, artigo por artigo, através do link:

www.devmedia.com.br/sqlmagazine/feedback

Para votar, você vai precisar do código de banca desta edição, que é: **trex**



Expediente

Editorial

Feita para Desenvolvedores de Software e DBAs

SQL magazine

Ano 7 - 74ª Edição 2010 - ISSN 1677918-5 - Impresso no Brasil

Corpo Editorial

Editor Geral

Rodrigo Oliveira Spínola
rodrigo@sqlmagazine.com.br

Sub Editores

Arilo Cláudio Dias Neto, Eduardo Oliveira Spínola
e Ricardo Rezende

Capa e Diagramação

Romulo Araujo
romulo@devmedia.com.br

Revisão e Supervisão

Thiago Vincenzo - thiago.v.ciancio@devmedia.com.br

Coordenação Geral

Daniella Costa - daniella@devmedia.com.br

Na Web

www.devmedia.com.br/sqlmagazine/pagina.asp

Distribuição

Fernando Chinaglia Dist. S/A
Rua Teodoro da Silva, 907
Grajau - RJ - 206563-900

Fale com o Editor!

É muito importante para a equipe saber o que você está achando da revista: que tipo de artigo você gostaria de ler, que artigo você mais gostou e qual artigo você menos gostou. Fique a vontade para entrar em contato com os editores e dar a sua sugestão!
Se você estiver interessado em publicar um artigo na

Atendimento ao Leitor

A DevMedia conta com um departamento exclusivo para o atendimento ao leitor. Se você tiver algum problema no recebimento do seu exemplar ou precisar de algum esclarecimento sobre assinaturas, exemplares anteriores, endereço de bancas de jornal, entre outros, entre em contato com:

Cristiany Queiroz - Atendimento ao Leitor
www.devmedia.com.br/mancad/
(21) 3382-5038

Kaline Dolabella - Gerente de Marketing e Atendimento
kalined@terra.com.br
(21) 3382-5038

Publicidade

Para informações sobre veiculação de anúncio na revista ou no site e para fechar parcerias ou ações específicas de marketing com a DevMedia, entre em contato com:

Kaline Dolabella
publicidade@devmedia.com.br

revista ou no site SQL Magazine, entre em contato com os editores, informando o título e mini-resumo do tema que você gostaria de publicar:

Rodrigo Oliveira Spínola - Editor da Revista
editor@sqlmagazine.com.br

EDITORIAL

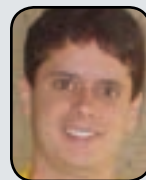
Nesta edição, a SQL Magazine traz como tema de capa Projeto de Banco de Dados. São duas matérias muito interessantes cujos assuntos você confere abaixo.

Na primeira delas, de autoria do Mauro Pichiliani, é apresentada a modelagem da estrutura de armazenamento de dados para um sistema de controle de tarefas administrativas de um condomínio. É um artigo muito interessante uma vez que oferece um modelo inicial para a construção de um sistema que gerencia e controla algumas tarefas administrativas de um condomínio, podendo ser adaptado para diversos outros projetos que necessitem de um controle de serviços e outras manutenções prediais.

Já a segunda matéria, de autoria do Arilo Cláudio, descreve o projeto de um banco de dados para um sistema de locação de carros, apresentando um roteiro que poderia ser seguido para construção de banco de dados a partir de sua especificação até a sua implementação em um SGBD.

Além destas matérias, a SQL Magazine traz nesta edição outras matérias muito interessantes envolvendo assuntos como PostgreSQL, MySQL, SQL Server, Oracle, Desafio SQL e Engenharia de Software.

Rodrigo Oliveira Spínola
editor@sqlmagazine.com.br



Rodrigo Oliveira Spínola

editor@sqlmagazine.com.br
Editor Chefe da SQL Magazine, WebMobile e Engenharia de Software Magazine. Doutorando e Mestre em Engenharia de Software pela COPPE/UFRJ - o maior centro de ensino e pesquisa em engenharia da América Latina. Diretor de Operações da Kali Software (www.kalisoftware.com). Autor de diversos artigos científicos sobre Engenharia de Software publicados em revistas e conferências renomadas, dentro e fora do país.

Assinatura



Mais conteúdo SQL por menos!

A revista SQL Magazine é parte integrante da assinatura SQL PLUS.
Para mais informações sobre o pacote SQL PLUS, acesse:
<http://www.devmedia.com.br/sqlmagazine/pagina.asp>



Modéstia à parte, sua melhor opção para se destacar no mercado!

A Escola Superior da Tecnologia da Informação oferece as melhores opções em cursos, formações, graduações e pós-graduações para profissionais de desenvolvimento e programação.

São programas voltados para a formação de profissionais de elite, com **aulas 100% práticas, corpo docente atuante no mercado**, acesso à mais atualizada biblioteca de TI do Rio, laboratórios equipados com tecnologia de ponta, salas de estudo e exames.

PÓS-GRADUAÇÃO

- ▶ Engenharia de Software: Desenvolvimento Java
- ▶ Engenharia de Software: Desenvolvimento .NET

GRADUAÇÃO

- ▶ Engenharia de Computação
- ▶ Análise e Desenv. de Sistemas

FORMAÇÕES

- ▶ Desenvolvedor Java
- ▶ Desenv. Java: Sist. Distribuídos
- ▶ Gestor de TI
- ▶ Desenvolvedor Web .NET 2008
- ▶ MCITP Server Administrator
- ▶ SQL Server 2008

Acesse nosso site e conheça todos os nossos programas: www.infnet.edu.br/esti



ESCOLA SUPERIOR DA TECNOLOGIA DA INFORMAÇÃO

www.infnet.edu.br - cursos@infnet.edu.br - Central de Atendimento: (21) 2122-8800

EDUCAÇÃO SUPERIOR ORIENTADA AO MERCADO

TURMAS
NO RIO DE
JANEIRO

Nesta seção você encontra artigos sobre projeto, análise ou modelagem de dados

Projeto de Banco de Dados para Condomínios

Veja como modelar um banco de dados para um sistema que controla as informações de um condomínio predial

De que se trata o artigo?

Modelagem da estrutura de armazenamento de dados para um sistema de controle de tarefas administrativas de um condomínio.

Para que serve?

Oferecer um modelo inicial para a construção de um sistema que gerencia e controla algumas tarefas administrativas de um condomínio, podendo ser adaptado para diversos outros projetos que necessitem de um controle de serviços e outras manutenções prediais.

Em que situação o tema é útil?

O modelo de dados para um sistema que gerencia o controle de tarefas administrativas de um condomínio. O modelo apresentado neste artigo é útil para a construção de sistemas que lidem com os gastos, receitas, serviços, fornecedores, ocorrências e reuniões de condomínio. A partir do modelo sugerido pode-se modificar as entidades, atributos e relacionamentos para adequar a estrutura de acordo com outros tipos de sistemas e cenários onde é preciso administrar pessoas morando em uma comunidade.



Mauro Pichiliani

pichiliani@uol.com.br

É bacharel em Ciência da Computação, mestre pelo ITA (Instituto Tecnológico de Aeronáutica) e MCP, MCDBA e MCTS. Trabalha há mais de 9 anos utilizando diversos bancos de dados, como o SQL Server, Oracle e MySQL. É colunista de banco de dados do web site iMasters. (<http://www.imasters.com.br>).

A concentração cada vez maior de pessoas vivendo em cidades metropolitanas acaba gerando a necessidade de regras, estatutos e leis específicas para a convivência harmoniosa de pessoas desconhecidas morando em um mesmo prédio. As tarefas, responsabilidades, direitos e obrigações de cada morador de um prédio devem ser controladas e gerenciadas pelos condomínios visando evitar problemas e atritos com os moradores. Para alcançar este objetivo existem diversos sistemas computacionais que se propõem a facilitar a

administração de cada aspecto relevante de um prédio e seus moradores.

Gerenciar as tarefas administrativas e operacionais de um condomínio pode ser uma tarefa complexa e trabalhosa se não houver nenhum suporte computacional, especialmente se os responsáveis pela administração (síndico, subsíndico e afins) não possuem uma maneira organizada de tratar todos os dados envolvidos no dia-a-dia da administração predial. Com base neste cenário, este artigo apresentará como modelar um banco de dados para um sistema que fornece recursos para

o gerenciamento de aspectos específicos de um condomínio, como o controle de moradores, reuniões, manutenções, receitas, orçamentos e alocação de áreas comuns.

A modelagem que será apresentada neste artigo aborda apenas alguns aspectos administrativos e não se aprofunda nos trâmites e detalhes relacionados à parte financeira, jurídica e contábil do condomínio. Contudo, a modelagem apresentada pode ser facilmente adaptada para suprir as necessidades de condomínios com diferentes características, como torres adicionais, cobranças bancárias, acordos de rateio complexos e contratos específicos com fornecedores. O modelo apresentado também poderá ser adaptado para a integração com outros sistemas que já existam ou que possam ser desenvolvidos visando à integração dos moradores, como intranets, sistemas de controle de telefonia, conexão à internet e outros.

Apesar de contemplar diversos cenários, o modelo apresentado neste artigo é simples e pode ser estendido para diferentes tipos de condomínio ou adaptado visando à integração com o módulo de contas a pagar e receber de um software ERP (ler Nota **DevMan 1**) já customizado e implantado no condomínio ou na empresa imobiliária que administra o prédio.

Entendendo o cenário

O funcionamento de um condomínio varia muito de acordo com dois principais aspectos: a forma que ele foi organizado e a maneira na qual os moradores se comportam e decidem questões pertinentes à convivência e manutenção do prédio. Para entender como funcionará o controle de alguns aspectos administrativos relacionados ao condomínio, é preciso primeiro delimitar o escopo e escolher um cenário típico onde a organização das tarefas administrativas é centralizada em um grupo de pessoas responsáveis pela administração geral. Neste artigo vamos considerar um condomínio organizado para gerenciar o funcionamento de um prédio com sete andares e com quatro apartamentos por andar. Este cenário é comum em prédios metropolitanos e possui as características de funcionamento adequadas para que possamos modelar um banco de dados que armazene informações relevantes a este domínio de problema.

A formação de um condomínio se inicia a partir de uma reunião onde os moradores discutem quem será o grupo de pessoas que terá a responsabilidade sobre as tarefas relacionadas ao dia-a-dia do prédio. A convocação e a pauta da reunião devem ser notificadas para todos os moradores, pois eles têm direito a voto na escolha do grupo de pessoas que vai ocupar os cargos por um período de tempo. Os moradores presentes na reunião também votam outras propostas que precisam de consenso, incluindo regras, normas, estatuto do prédio, autorização para despesas e aumentos do valor do condomínio. A reunião de condomínio é periódica e deve ser utilizada também para comunicados, solicitação de opiniões e comentários gerais de todos os moradores que participarem. Ao término da reunião é preciso fixar as decisões tomadas em um local público do prédio para que os moradores que não compareceram na reunião fiquem cientes do que foi discutido e votado. Cabe aqui destacar que participar de uma reunião de condomínio

pode ser tão desagradável quanto a dor proporcionada por uma pedra no rim ou a visita a um dentista sem anestesia, porém essa é a principal maneira de saber o que acontece na comunidade em que se vive e de participar ativamente das decisões que afetam a todos os moradores.

Uma vez que os cargos do condomínio já estão preenchidos por um grupo de pessoas é preciso começar a gestão administrativa dos moradores escolhidos, cujo principal cargo é o síndico. Existem também cargos como subsíndico, zelador, porteiro, faxineiros e outros, porém geralmente a principal figura que representa o condomínio é o síndico, devido ao seu poder de tomar decisões sancionadas pelos moradores durante as reuniões de condomínio.

Uma das principais funções do síndico é a contratação de serviços para manutenção do prédio. Estes serviços incluem limpeza gerais, revisões de elevadores, pintura, manutenção elétrica, jardinagem, hidráulica, telefonia, alvenaria e diversos outros tipos de serviços. Cada um destes trabalhos requer pesquisa de fornecedores, orçamentos, contratos e acompanhamento. Há também inspeções e datas importantes que devem ser controladas, como inspeções da prefeitura, revisão de bombeiros, datas de vencimentos de extintores e colocação de produtos químicos na piscina do prédio, apenas para citar alguns exemplos.

Alguns condomínios contam também com contratos que geram receitas para todos os moradores. Exemplos destas receitas incluem o aluguel de espaço para a colocação de antenas no topo do prédio, pequenas lojas em espaços da recepção e também a colocação de propagandas na fachada do prédio nas cidades onde tal prática é permitida. Estes contratos e seus respectivos detalhes também devem ser organizados e controlados pelo síndico.

Nos prédios que possuem áreas compartilhadas (churrasqueira, salão de festas e jogos, piscina, academia, quadras poliesportivas) é preciso controlar a alocação de algumas destas áreas de acordo com uma agenda, pois pode haver uma concorrência de acesso quando diferentes moradores solicitarem o uso de tais áreas. Além de controlar a locação, também é preciso definir quais são as regras de uso que incluem horário, limpeza, limite de som, quantidade de pessoas, aluguel de mesas, etc. No caso de eventos que trazem convidados, o síndico e os responsáveis pela portaria e segurança devem estar cientes de tal atividade



Nota do DevMan 1

ERP

ERP (do inglês Enterprise Resource Planning) são os sistemas de informações que integram todos os dados e processos de uma organização em um único sistema. A integração pode ser vista sob a perspectiva funcional (sistemas de finanças, contabilidade, recursos humanos, fabricação, marketing, etc.) e sob a perspectiva sistêmica (sistema de processamento de transações, sistemas de informações gerenciais, sistemas de apoio a decisão, etc.). Estes sistemas envolvem a parte administrativa da empresa e geralmente são de missão crítica, pois envolvem processos fundamentais para o funcionamento da empresa.

no prédio, pois esta comunicação evita atrapalhar o evento social que o morador está realizando.

Infelizmente a convivência de diversas pessoas em um prédio nem sempre acontece de forma harmoniosa. Isso quer dizer que eventualmente algum morador pode desrespeitar alguma regra ou norma e precise ser advertido de acordo com a gravidade e circunstâncias da violação. O síndico deve ser capaz de organizar, armazenar e controlar as reclamações, avisos, notificações e punições de ocorrências devido ao comportamento inadequado e a violação das regras e normas do condomínio para cada morador que desrespeitar o que foi acordado nas reuniões de condomínio e o que está escrito no estatuto.

Para finalizar, vale a pena lembrar que a convivência harmoniosa de pessoas que moram em locais próximos só acontece quando o comportamento dos moradores é regido pelo protocolo social do bom senso e quando há um forte senso de comunidade. Infelizmente muitas pessoas não conseguem se adaptar ao modo de vida que requer respeito, compreensão, tolerância e convivência, fatores cruciais em comunidades cujas moradias são tão próximas. Isso implica que um dos principais fatos que deve sempre estar na mente do grupo de pessoas que mora no condomínio, e principalmente no síndico, é a conscientização que nunca será possível agradar a todos e que diversos problemas vão surgir ao logo da gestão. Por isso, saber lidar com estes problemas de maneira adequada é fundamental para uma gestão de sucesso em um ambiente tão propício a conflitos gerados pela convivência em comunidade.

Modelagem das entidades

A partir da explicação do funcionamento dos aspectos relevantes do modelo de condomínio descrito na seção anterior, podemos começar a modelagem indicando quais são as principais entidades do modelo Entidade Relacionamento. A **Figura 1** apresenta uma versão inicial do modelo com os nomes de treze entidades relevantes para o banco de dados que gerenciará os aspectos administrativos de um condomínio. Por se tratar apenas de um levantamento inicial de entidades, podemos utilizar nomes com acentos e caracteres especiais, porém nos refinamentos do modelo eliminaremos os acentos, espaços em branco e caracteres especiais para evitar problemas

durante a implementação do modelo e a integração do banco de dados com a aplicação.

As entidades apresentadas na **Figura 1** representam apenas o levantamento inicial do que será modelado, ou seja, a partir da explicação dos aspectos relevantes de um condomínio criaram-se diversas entidades. A partir dessas entidades o modelo será construído por sucessivos refinamentos e melhorias. Entretanto, é importante delimitar o escopo e identificar até que ponto as entidades modeladas fazem parte ou não do banco de dados. Esta delimitação quer dizer que é preciso identificar e abstrair certas características dos cenários de modo a melhorar o modelo para que ele represente a realidade apenas nos aspectos relevantes para o banco de dados relacionado ao contexto da administração de um condomínio.

A primeira entidade a ser detalhada é aquela que armazenará informações sobre os moradores. Quem trabalha no ramo imobiliário utiliza o termo unidade para se referir às pessoas que estão habitando determinado apartamento. Desta maneira não se especifica o nome das pessoas que estão ocupando atualmente o apartamento. Geralmente a unidade é identificada pelo seu número do apartamento, que é determinado pelo número do andar mais uma numeração sequencial de acordo com a distância entre os elevadores. Por exemplo: a unidade 41 identifica o apartamento mais próximo do elevador no quarto andar. Caso se tenha diversos prédios administrados por um único condomínio, é comum utilizar o nome da torre para identificar o local exato da moradia.

A entidade **UNIDADES** contém um identificador sequencial, armazenado no atributo **ID_UNIDADE** e que é a chave primária da tabela. Esta entidade também contém o CPF e o nome do responsável pela entidade nos atributos **CPF_RESPONSÁVEL** e **NOME_RESPONSÁVEL**. A identificação do número da entidade pode ser composta por números e letras, e por isso o atributo **NR_UNIDADE** utilizará um tipo de dados que armazena caracteres. A entidade também possui atributos flag (ler Nota **DevMan 2**) para indicar se ela está ativa ou não, ou seja, se há alguém morando atualmente na unidade, e se quem está morando paga aluguel. Os demais atributos da entidade são o nome da torre, telefones de contato, endereço de e-mail, data da compra do apartamento, placas de veículos, número



Figura 1. Treze entidades iniciais do modelo de controle administrativo de condomínio

da vaga na garagem do estacionamento, o valor da área total e da fração ideal. A **Figura 2** apresenta a entidade **UNIDADES** junto com o detalhamento de seus atributos em um modelo entidade relacionamento parcial.

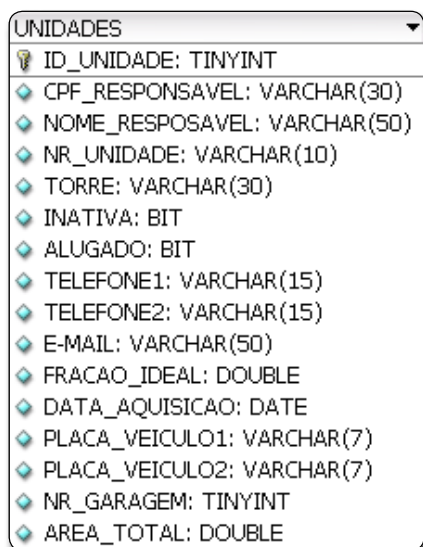


Figura 2. Entidade **UNIDADES** do modelo de controle de condomínio

A fração ideal, armazenada no atributo **FRACAO_IDEAL** da entidade **UNIDADES**, é um número decimal que indica a parte indivisível e indeterminável das áreas comuns e de terreno, proporcional à unidade autônoma de cada condômino e documentada na escritura do imóvel e na convenção do condomínio. Como nem todos os condomínios apresentam unidades autônomas com a mesma área útil, os proprietários dos apartamentos ou casas maiores podem possuir frações ideais diferentes. Além disso, é de acordo com essa fração ideal que são estipulados seus deveres (como o valor do condomínio e taxas extras) e, se a convenção do condomínio não dispuser em contrário, seus direitos como, por exemplo, a maior representatividade nas votações realizadas nas reuniões de condomínio.

A próxima entidade a ser modelada é a entidade que armazenará as informações referentes à reunião de condomínio. Esta reunião deve ser marcada pelos funcionários do condomínio e é aberta para todos os moradores que quiserem participar. Contudo, apenas os proprietários ou seus respectivos representantes com procuração possuem direito a voto nas questões apresentadas durante a reunião.

A entidade **REUNIOES** contém um atributo único que identifica cada reunião (chave primária), a data e hora da reunião, a pauta, informações sobre o local e um atributo flag que indica se a reunião é emergencial ou não. Após a reunião é preciso colocar o que foi discutido, o resultado das votações e demais informações em um atributo especial para armazenar o relatório da reunião.

A presença dos moradores, proprietários e responsáveis pelas votações deve ser armazenada em uma entidade separada, a entidade **PRESENCA_REUNIAO**. Esta entidade indicará



Nota do DevMan 2

Atributo flag

Existem diversas definições do que é um atributo flag. A mais comum diz que um atributo flag é aquele que permite apenas dois tipos de valores: 0 ou 1, também referenciados como SIM/NÃO ou VERDADEIRO/FALSO (TRUE/FALSE). Geralmente atributos flag são utilizados para modelar características ou aspectos binários que podem existir ou não sem nenhum tipo de meio termo. A implementação depende do banco de dados em questão, mas é comum o uso do tipo de dados booleano onde apenas 0 ou 1 são utilizados para armazenar os valores de atributos do tipo flag.

quais unidades compareceram a quais reuniões e conterá em uma chave primária composta os atributos **ID_REUNIAO** e **ID_UNIDADE**. Estes atributos também são chaves estrangeiras que apontam para os atributos de mesmo nome das entidades **UNIDADES** e **REUNIOES**, respectivamente. Além disso, a entidade **PRESENCA_REUNIAO** também contém atributos para armazenar o nome do participante, se ele está munido de uma procuração ou não e um atributo para armazenar observações gerais a respeito da sua participação na reunião, como a sugestão de melhorias, protestos, preocupações ou outros detalhes relevantes. A **Figura 3** apresenta as entidades **UNIDADES**, **REUNIOES** e **PRESENCA_REUNIAO** junto com o detalhamento de seus atributos em um modelo entidade relacionamento parcial.

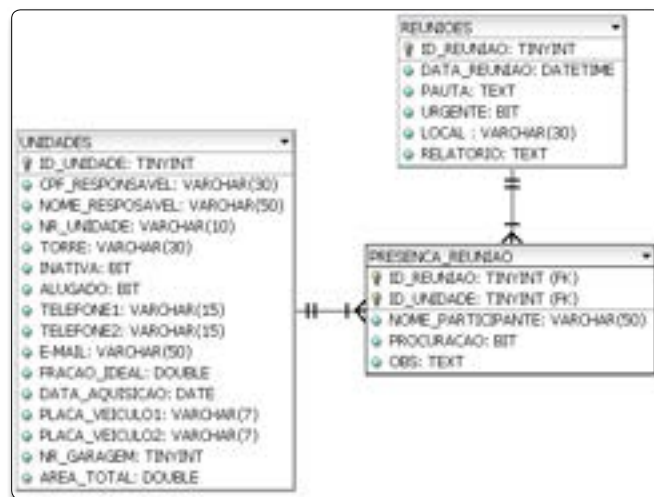


Figura 3. Entidades **UNIDADES**, **REUNIOES** e **PRESENCA_REUNIAO** do modelo de controle de condomínio

O grupo de pessoas que será responsável pela administração do condomínio é definido por votação em uma das reuniões do condomínio, geralmente uma vez por ano. A gestão de cada grupo do condomínio deve ser armazenada no modelo de dados em uma entidade que receberá o nome de **GESTAO**. Esta entidade possuirá uma chave primária, atributos para identificar a data de início e término da gestão, um campo

texto para armazenar as principais realizações durante a gestão e também um atributo que armazena dados binários com o objetivo de guardar o conteúdo do texto que contém o estatuto do condomínio, também conhecido como convenção do condomínio. Por fim, é preciso colocar dois atributos, **ID_SINDICO** e **ID_SUBSINDICO**, que são duas chaves estrangeiras para a entidade **UNIDADES**, pois estes atributos indicam quem é o síndico e o subsíndico daquela gestão.

A entidade **GESTAO** pode ser modificada para incluir mais chaves estrangeiras visando adicionar outros papéis para o grupo da gestão (zelador, advogado do condomínio, etc.) e também pode-se modificar o atributo que armazena dados binários, ou seja, pode-se trocar o tipo de dados deste atributo por um tipo de dados que armazene apenas o caminho do arquivo texto que contém o estatuto do condomínio.

A próxima entidade a ser modelada deve abranger as ocorrências e notificações dos moradores quando há algum tipo de reclamação ou problema no comportamento de algum morador ou de alguém que está nas dependências do prédio. Para simplificar, as entidades **OCORRÊNCIAS** e **NOTIFICAÇÕES**, apresentadas na **Figura 1**, foram agrupadas em apenas uma entidade chamada **OCORRENCIAS**. Esta entidade contém uma chave primária que identifica a ocorrência, um atributo para relacionar a ocorrência com alguma unidade, o nome da ocorrência, uma descrição detalhada do que aconteceu e das circunstâncias, a data e hora da ocorrência, um atributo texto para indicar a ação tomada, incluindo punições, multas, notificações e advertências, e um atributo final com observações pertinentes a esta ocorrência. Uma possível customização para a entidade **OCORRENCIAS** é a alteração desta entidade para que ela permita mais de uma chave estrangeira para a entidade **UNIDADE**, isto é, associar diferentes moradores de unidades distintas a uma mesma ocorrência caso haja necessidade.

O próximo passo na construção do modelo é a especificação de como serão armazenados os dados sobre as áreas compartilhadas do prédio e a reserva para uso das mesmas. A modelagem da **Figura 1** sugeriu o nome da entidade **ÁREAS COMPARTILHADAS** para armazenar no banco de dados este tipo de informação, porém quem trabalha no ramo imobiliário utiliza o termo área comum. Sendo assim, criaremos uma entidade chamada **AREA_COMUM**, que conterá uma chave primária, o nome da área compartilhada (piscina, salão de festas, salão de jogos, etc.) os horários de início e fim que delimitam o uso desta área e um atributo para armazenar as regras de uso, como informações sobre limpeza, acesso, trajés, comportamento adequado, etc.

Para relacionar o uso de uma área comum com uma unidade é preciso antes fazer uma reserva, que será armazenada na entidade **RESERVA_AREA_COMUM**. Esta entidade possui uma chave primária composta a partir dos atributos **ID_UNIDADE** e **ID_LOCAL**, que são chaves estrangeiras para os atributos de mesmo nome das entidades **UNIDADES** e **AREA_COMUM**, respectivamente. A entidade **RESERVA_AREA_COMUM** contém ainda atributos para guardar a data com hora de início e fim da reserva, o nome do evento social organizado

pelo morador e a quantidade esperada de pessoas. A **Figura 4** apresenta o modelo criado até então com a adição das entidades **GESTAO**, **OCORRENCIAS**, **AREA_COMUM** e **RESERVA_COMUM**.

Após a modelagem das entidades relacionadas à área comum vamos para a parte final do modelo, cujo foco é no armazenamento de serviços, fornecedores, contratos, orçamentos e prazos.

A entidade **PRAZOS** armazenará dados a respeito das tarefas de manutenção que devem ser feitas periodicamente pelos funcionários do prédio ou por empresas externas, como por exemplo, manutenção na piscina (limpeza, colocação de cloro, rejunte, etc.), vistoria de elevadores, troca de extintores de incêndio, entre outros. A entidade **PRAZOS** engloba a entidade **MANUTENÇÕES** que foi modelada inicialmente e contém uma chave primária, um atributo para o nome da tarefa, a descrição em forma de texto do procedimento a ser realizado, a periodicidade (diária, semanal, mensal, etc.) a data da última ocorrência desta manutenção e um atributo para armazenar observações gerais sobre o prazo. A entidade **PRAZOS** não possui nenhuma chave estrangeira e está relativamente isolada no modelo, como veremos mais adiante.

A próxima entidade a ser modelada armazenará os dados dos diversos fornecedores que interagem com o condomínio. A entidade **FORNECEDORES** possui uma chave primária, o CNPJ e o CPF do fornecedor (para o caso de empresas ou profissionais autônomos), o nome do fornecedor, informações de contato como telefone, endereço de e-mail e nome do responsável por atender ao condomínio e um atributo flag que indica se o fornecedor provém um produto ou um serviço.

A entidade **FORNECEDORES** armazenará os fornecedores que requerem pagamento e também os fornecedores que geram receitas. Contudo, o controle das receitas e gastos do condomínio é armazenado em uma entidade própria chamada **CONTRATOS**, pois é obrigatório o emprego de um contrato para receitas ou despesas recorrentes do condomínio. A entidade **CONTRATOS** possui uma chave primária (**ID_CONTRATO**), uma chave estrangeira para a entidade **FORNECEDORES** (**ID_FORNECEDOR**), o nome que identifica o contrato (**NOME_CONTRATO**), um atributo para detalhes adicionais (**DETALHES**), as datas de início (**DATA_ASSINATURA**) e vencimento do contrato (**DATA_VENCIMENTO**), o valor mensal (**VALOR_MENSAL**) que será positivo para receitas e negativo para despesas, e um atributo para o armazenamento binário do arquivo que contém o contrato (**ARQ_CONTRATO**).

Cada serviço que for contratado pelo condomínio deve possuir ao menos dois orçamentos. Esta é uma regra que foi criada para evitar o favorecimento de um fornecedor específico e para estimular a concorrência e a busca pelo melhor preço. A entidade que armazenará os orçamentos se chamará **ORCAMENTO_SERVICO** e conterá uma chave primária e duas chaves estrangeiras: uma chave estrangeira para a entidade **FORNECEDOR**, representada no atributo **ID_FORNECEDOR**; e uma chave estrangeira para a entidade **SERVICOS**, que ainda não foi modelada. A entidade **ORCAMENTO_SERVICO**

contém ainda a data do orçamento, o valor e também um atributo para observações gerais.

A entidade final do modelo se chama **SERVICOS** e é responsável pela associação de várias entidades. É preciso relacionar a entidade **SERVICOS** com os orçamentos, a gestão, o local proveniente da área comum e o prazo caso o serviço esteja relacionado a alguma tarefa de manutenção. Isso quer dizer que a entidade **SERVICOS** terá uma chave primária (ID_SERVICO) e quatro chaves estrangeiras: ID_ORCAMENTO, ID_GESTAO, ID_LOCAL e ID_PRAZO. Nota-se que não há uma relação direta entre um serviço e um fornecedor, pois a relação é entre um serviço e um orçamento e entre orçamento e fornecedor. Também se destaca o fato que o relacionamento entre a entidade **SERVICO** e as entidades **AREA_COMUM** e **PRAZO** é opcional, pois o serviço pode ser realizado apenas uma única vez e em um local do prédio que não está cadastrado como uma área comum.

Por fim, a entidade **SERVICOS** conta ainda com atributos para armazenar a data de realização do serviço, o funcionário do fornecedor responsável, o nome do serviço, dados do comprovante (número da nota fiscal, recibo, etc.) e um atributo para digitação de observações gerais, como horário de

indisponibilidade de recursos, valores de medição, garantia e outros. A **Figura 5** apresenta o modelo final do controle de condomínio já com as entidades **PRAZOS**, **FORNECEDORES**, **CONTRATOS**, **ORCAMENTO_SERVICO** e **SERVICOS**. Para facilitar a implementação do modelo, o arquivo *Condominio.xml* será disponibilizado no site da DevMedia. Este arquivo contém o modelo final apresentado na **Figura 5** e pode ser aberto utilizando a ferramenta de código livre para modelagem de banco de dados chamada DBDesigner.

Conclusão

Este artigo mostrou como modelar as principais entidades de um sistema que controla as tarefas administrativas de um condomínio. O modelo apresentado foi gerado a partir de um cenário típico onde síndicos, fornecedores e moradores interagem de diversas formas durante o dia a dia de um prédio. Um modelo de dados foi elaborado a partir da análise das principais tarefas administrativas que não envolvem atividades contábeis, financeiras e jurídicas. Este modelo contém entidades, atributos e relacionamentos que representam moradores, reuniões, ocorrências, áreas compartilhadas,

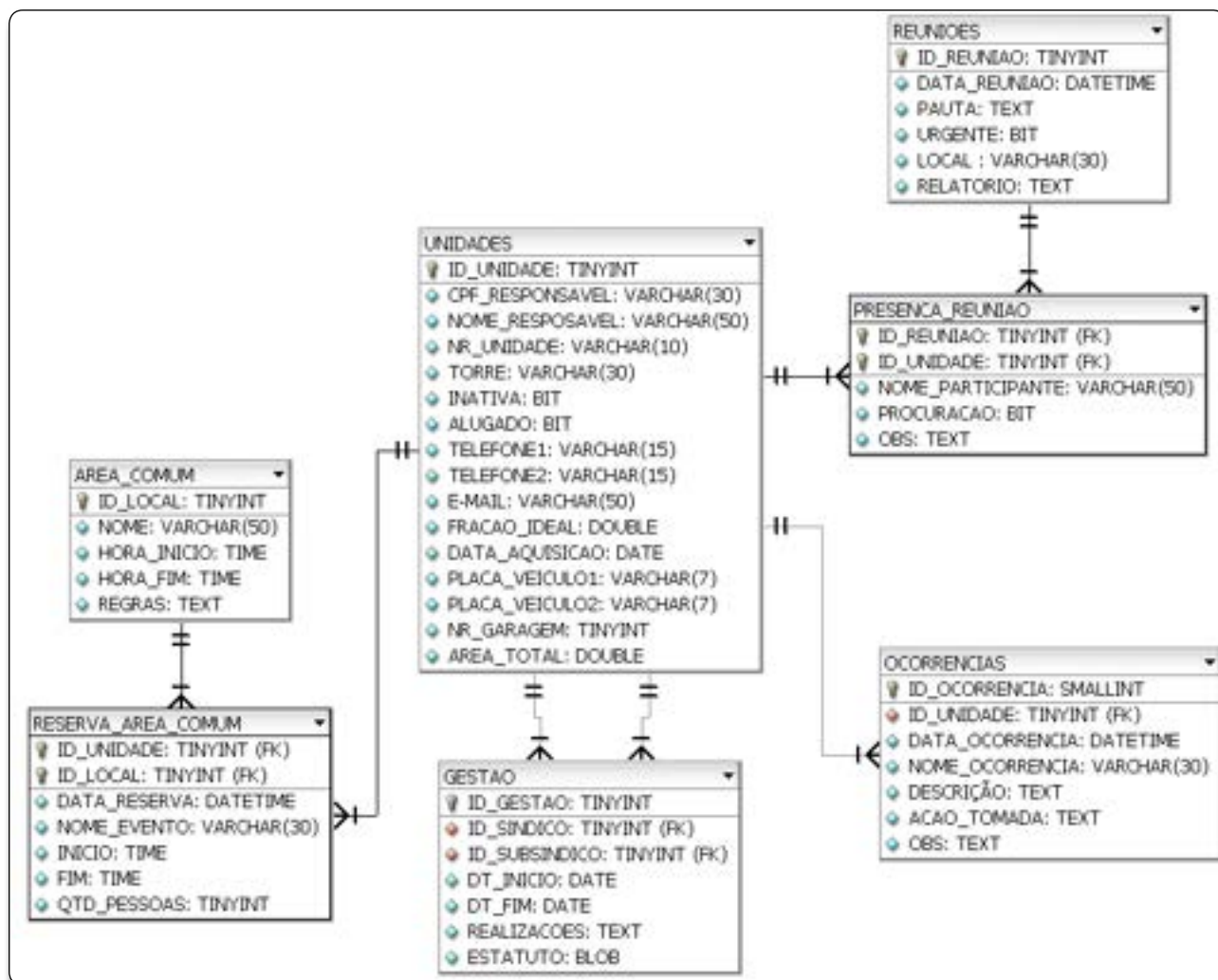


Figura 4. Entidades GESTAO, OCORRENCIAS, AREA_COMUM e RESERVA_AREA_COMUM adicionadas ao modelo parcial de controle de condomínio

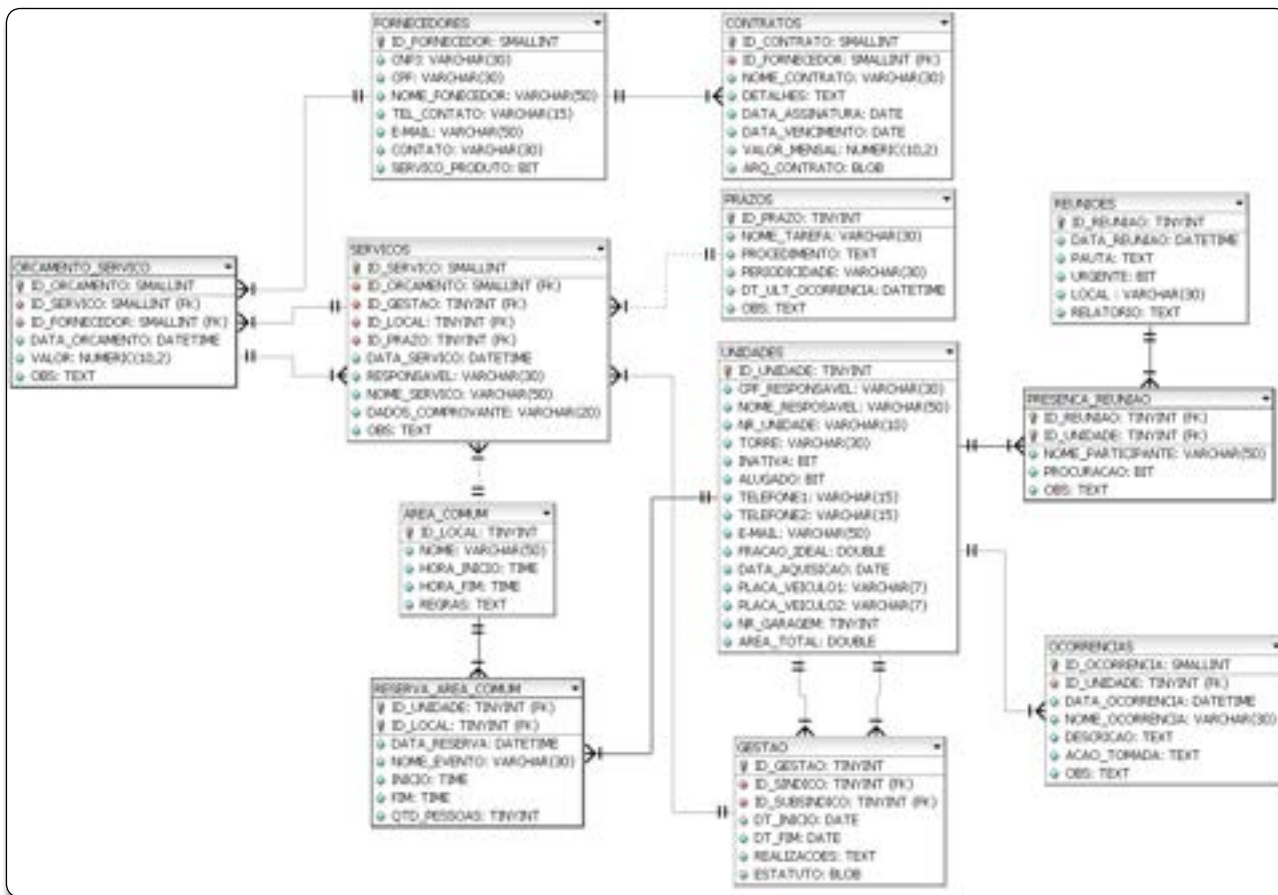


Figura 5. Entidades PRAZOS, FORNECEDORES, CONTRATOS, ORÇAMENTO_SERVICO e SERVICOS adicionadas ao modelo final de controle de condomínio

serviços, fornecedores, orçamentos, prazos e outros aspectos relacionados às atividades administrativas de um prédio de acordo com o cenário escolhido.

O modelo de dados foi apresentado de acordo com a modelagem de cada uma das entidades e seus relacionamentos. Apesar de ser apenas uma sugestão, as principais entidades do cenário descrito foram contempladas pelo modelo de dados, que pode servir como ponto de partida para a modificação e a modelagem de novas entidades relacionadas às tarefas administrativas de um condomínio e também para outros fins, como o controle casas de um lote, condomínios fechados, assentamentos e comunidades de pessoas que vivem juntas e são administradas por um conjunto de pessoas escolhidas para tal tarefa. ●

Referências

Site SindicoNET

<http://www.sindiconet.com.br/>

Download da ferramenta de modelagem DBDesigner

<http://www.fabforce.net/dbdesigner4/>

Dê seu feedback sobre esta edição!

A SQL Magazine tem que ser feita ao seu gosto. Para isso, precisamos saber o que você, leitor, acha da revista!

Dê seu voto sobre este artigo, através do link:

www.devmedia.com.br/sqlmagazine/feedback



Nesta seção você encontra artigos sobre projeto, análise ou modelagem de dados



Modelando um Sistema de Reserva de Carros

Contruindo modelos lógicos e físicos, além de implementar o banco de dados com o MySQL

De que se trata o artigo?

Este artigo descreve o projeto de um banco de dados para um sistema de locação de carros, apresentando um roteiro que poderia ser seguido para construção de banco de dados a partir de sua especificação até a sua implementação em um SGBD.

Para que serve?

Auxiliaria projetistas de banco de dados que possuem a necessidade de modelar e projetar bancos de dados para sistemas de informação, pois apresenta um roteiro que apoiaria em tal tarefa.

Em que situação o tema é útil?

Banco de dados é um elemento essencial em qualquer sistema orientado a informações. Dessa forma, sua modelagem e projeto se torna uma atividade essencial para atingir qualidade e sucesso na aplicação a ser desenvolvida.

Imagine uma situação onde uma empresa deseja divulgar uma super promoção aos seus clientes. Como saber quais seriam seus clientes e como acessá-los? Ter um caderno contendo o nome, telefone e email é algo impraticável nos dias atuais. Em um segundo exemplo, imagine que uma organização deseja ter o controle de todo o seu estoque e deseja saber quando um produto bastante vendido pela loja está em falta, ou melhor, como prevenir a ausência total deste produto em estoque? Imaginar que a organização contrataria um funcionário apenas para manter esse controle contando os produtos ainda em seu depósito é custoso não apenas financeiramente, mas também levaria um custo de tempo muito grande.



Arilo Claudio Dias Neto

ariloclaudio@gmail.com

É Doutor em Engenharia de Sistemas e Computação formado pela Universidade Federal do Rio de Janeiro (COPPE). Possui 6 anos de experiência em análise e desenvolvimento de software. É ainda editor técnico da Revista SQL Magazine, gerenciada pelo Grupo DevMedia.

No desenvolvimento de qualquer sistema computacional orientado a informação, ou simplesmente *Sistemas de Informação*, o banco de dados possui papel essencial para seu funcionamento e sua qualidade. Normalmente, as principais transações que seriam o carro-chefe de

uma organização são processadas e armazenadas em bancos de dados, de forma que estes podem conter boa parte da vida de uma determinada organização.

Neste artigo, trataremos de uma das etapas iniciais do processo de construção de um sistema de informação: a modelagem, projeto e construção do banco de dados de um sistema de informação. Usaremos um estudo de caso de um sistema de uma locadora de carros, que deseja gerenciar as informações sobre seu patrimônio (carros), clientes e reservas. Partiremos da descrição do problema em linguagem natural até a construção do banco de dados utilizando o SGBD MySQL.

Modelagem de Dados

Modelagem de dados é um método usado para definir e analisar requisitos de dados necessários para apoiar o processo de negócio de uma organização. Os requisitos de dados são registrados como um modelo de dados conceitual com definições dos dados e seus tipos. A implementação real de um *modelo conceitual* é chamada de *modelo de dados lógico*. Implementar um modelo de dados conceitual pode requerer vários modelos de dados lógicos. Por fim, os modelos lógicos são transformados em *modelos físicos*, representados em alguma linguagem ou banco de dados. A modelagem de dados define não apenas seus elementos, mas sua estrutura e seus relacionamentos. A modelagem de dados pode ser realizada em vários tipos de projetos e em diversas fases do projeto. Modelos de dados são progressivos; não existe um modelo final para uma aplicação.

Modelo Conceitual

Um modelo mental captura ideias em um domínio de problema, enquanto que um modelo conceitual representa “conceitos” (entidades) e seus relacionamentos em um alto nível de abstração.

O modelo conceitual deve ser definido independentemente dos detalhes de implementação, e seu objetivo é expressar o significado dos termos e conceitos usados por especialistas do domínio para que possamos discutir um problema, e encontrar o relacionamento correto entre os diferentes conceitos.

Entre as principais características de um modelo conceitual, podemos citar:

- Ele inclui as entidades importantes que compõem o problema a ser tratado e o relacionamento entre elas.
- Não há a necessidade de especificar os atributos das entidades e nem dos relacionamentos.
- Nenhuma chave primária é especificada.

Um modelo conceitual pode ser descrito usando várias notações, dentre as principais podemos citar a UML. Na linguagem UML, o modelo conceitual é normalmente descrito através de um Diagrama de Classes, no qual as classes representam os conceitos e associações representam os relacionamentos entre os conceitos. Na notação de Modelo de Entidade-Relacionamento (ER) (ler Nota **DevMan 1**), o modelo conceitual é descrito como um Diagrama de ER no qual as entidades representam

conceitos, cardinalidade e opcionalidade representam relações entre os conceitos.

Existem diversas estratégias e notações para modelagem de dados, tais como o uso de Modelos de Entidade e Relacionamentos (ER) e Diagramas UML como o Diagrama de Classes. No artigo “Convertendo um Diagrama de Entidade e Relacionamento em um Banco de Dados Físico” publicado na edição 63 da SQL Magazine, conhecemos um pouco mais sobre uma forma de representação de modelo lógico e uma estratégia para transformá-lo em um banco de dados físico.

Ao longo deste artigo veremos um exemplo de modelo conceitual a ser seguido a partir de um estudo de caso.



Nota do DevMan 1

Modelo de Entidade e Relacionamentos

Em Engenharia de Software, um Modelo de Entidade e Relacionamento (MER) é uma representação abstrata e conceitual de dados. A modelagem de entidade e relacionamentos é um método de modelagem de banco de dados bastante utilizado, e é usado para produzir um tipo de esquema conceitual ou modelo de dados semântico de um sistema, normalmente um banco de dados relacional, e seus requisitos em uma abordagem top-down.

Um MER é composto por Entidades, Relacionamentos e Atributos.

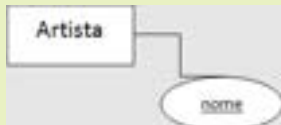
Uma entidade pode ser definida como algo que é reconhecido como sendo capaz de uma existência independente e que pode ser identificada unicamente. Uma entidade é uma abstração de algum elemento de um domínio. Quando falamos de uma entidade, normalmente falamos de algum aspecto do mundo real que pode ser distinguido de outros aspectos. São representados normalmente por nomes (substantivos) e são representados graficamente por retângulos.

Exemplo de entidade: 

Um relacionamento captura como duas ou mais entidades estão relacionadas umas às outras. Relacionamentos são descritos como verbos, ligando dois substantivos. Relacionamentos são representados como diamantes, conectados por linhas para cada entidade que compõe o relacionamento.

Exemplo de relacionamento: 

Entidades e relacionamentos possuem ambos atributos. Exemplos: um empregado possui o número da carteira de trabalho como atributo. Atributos são representados como elipses conectadas às suas entidades através de uma linha.

Exemplo de atributos: 

Exemplo de MER:



Modelo Lógico

Um modelo de dados lógico é uma representação dos dados da organização, organizados em termos de entidades e relacionamentos e ainda é independente de qualquer tecnologia para gerenciamento de dados.

Modelo de dados lógico representa uma estrutura abstrata de algum domínio de informação. Eles são normalmente usados em processos de negócio que buscam capturar características importantes de um domínio e suas relações.

Estes modelos devem se basear na estrutura identificada anteriormente a partir do modelo conceitual, pois o modelo conceitual descreve as semânticas do contexto das informações, e o modelo lógico deve refletir essa semântica. Após aprovado e validado, o modelo de dados lógico pode se tornar a base para um modelo de dados físico, e consequentemente para um projeto de um banco de dados.

Modelo Físico

Um modelo de dados físico é uma representação de um projeto de dados que leva em consideração as facilidades e restrições de um certo sistema gerenciador de banco de dados (SGBD). No ciclo de vida de um projeto, ele é tipicamente derivado a partir do modelo lógico. Um modelo de dados físico completo incluirá todos os artefatos de banco de dados necessários para criar relacionamentos entre tabelas ou obter os objetivos de desempenho, tais como índices, restrições, views, particionamentos, etc.

Por diversas vezes as pessoas tendem a confundir modelo de dados lógico e físico. A **Tabela 1** tenta esclarecer um pouco a diferença entre eles.

A seguir conheceremos o estudo de caso a ser seguido para modelagem e projeto de um banco de dados: o caso da empresa *ACDN Rental Car*.

Estudo de Caso: Locadora de Carros ACDN

O objetivo de nosso estudo de caso é desenvolver um sistema de banco de dados para automatizar as funções de serviços administrativos e de clientes de uma empresa que trabalha com locação de carros, que chamaremos de *ACDN Rental Car*.

A *ACDN Rental Car* permite aos seus clientes reservar carros a partir da Internet ou indo diretamente à sede da empresa. A empresa possui duas sedes, sendo uma a matriz e uma a filial. Dessa forma, a locação de um veículo pode ser feita em uma sede e a devolução na outra sede, sem restrição. Além disso, algumas vezes carros são transferidos de uma localização

para outra, ou seja, o ponto de locação de um carro pode ser alterado.

A *ACDN Rental Car* oferece várias classes de carros e vários preços de locação. Os carros são agrupados nas seguintes classes: Subcompacto, Compacto, Tamanho Médio, Tamanho Grande e Luxo. O preço de locação varia por classe do carro e tempo de locação. A empresa cobra algumas taxas quando o carro não é retornado ao seu local de origem. O valor depende de cada sede.

Algumas regras completam o sistema da *ACDN Rental Car*:

- Um cliente não pode realizar uma segunda reserva caso ainda possua uma locação aberta (cujo carro ainda não foi retornado).
- Apenas clientes com habilitação em dia podem realizar locações, ou seja, se as habilitações cadastradas pelo cliente estiverem com prazo vencido este não poderá realizar uma reserva. Neste caso é necessário atualizar seus dados antes de fazer a reserva.
- Um carro alugado ou que está no ponto que não é seu ponto de origem não pode ser retornado. Este deverá ser retornado pelo cliente ou deve ser transferido pela empresa ao seu ponto de origem.

A seguir iniciaremos o processo de modelagem e projeto do banco de dados que atenda às necessidades da *ACDN Rental Car*, começando pela modelagem conceitual do problema.

Etapa 1: Modelagem Conceitual

Para a modelagem conceitual, iremos utilizar um Diagrama de Classes da UML contendo apenas os nomes das classes identificadas que representam as entidades do problema.

Os conceitos que foram identificados e estão representados por classes são:

- Clientes: entidade que representa os clientes que já estão cadastrados no sistema, de forma que estão aptos a realizar reserva de um carro caso atendam às regras definidas pela empresa.
- Classes de Carro: entidade que representa as diferentes classes de carro que foram adotadas pela empresa para agrupar seus carros.
- Carros: entidade que representa os carros que estão disponíveis para locação pela empresa.
- Sedes: entidade que representa as diferentes sedes (atualmente são apenas duas) de locação que a empresa possui.
- Reservas: entidade que representa as reservas realizadas por clientes em um determinado momento.

Modelo de Dados Lógico	Modelo de Dados Físico
Inclui entidades/tabelas, atributos/colunas/campos e relacionamentos	Inclui tabelas, colunas, chaves, tipos de dados, regras de validação, triggers, stored procedures, restrições de domínio e de acesso
Usa nome de elementos do domínio para seus atributos	Usa nomes de colunas abreviados limitados pelo SGBD
É independente de tecnologia (ex: plataforma ou SGBD)	Inclui chaves primárias e índices para acesso rápido aos dados
É normalizado até a quarta forma normal (ver artigo "Aplicando Normalização a um Modelo de Dados" publicado na edição 58 da SQL Magazine)	Pode ser desnormalizado para atingir requisitos de desempenho baseado na natureza do banco de dados

Tabela 1. Comparação entre Modelo Lógico x Modelo Físico

Os relacionamentos entre os conceitos identificados que compõem o modelo conceitual são:

- Clientes – Reservas: clientes realizam reservas na empresa.
- Sedes – Reservas: cada reserva é realizada em uma sede, e isso influenciará no ponto onde o carro alugado deve ser retornado.
- Carros – Reservas: cada reserva deve estar associada a um carro, selecionado pelo cliente no momento da locação.
- Classes de Carro – Carros: cada carro deve estar associado a uma classe de carro adotada pela empresa.
- Sedes – Carros: cada carro possui como ponto de origem uma das sedes da empresa.

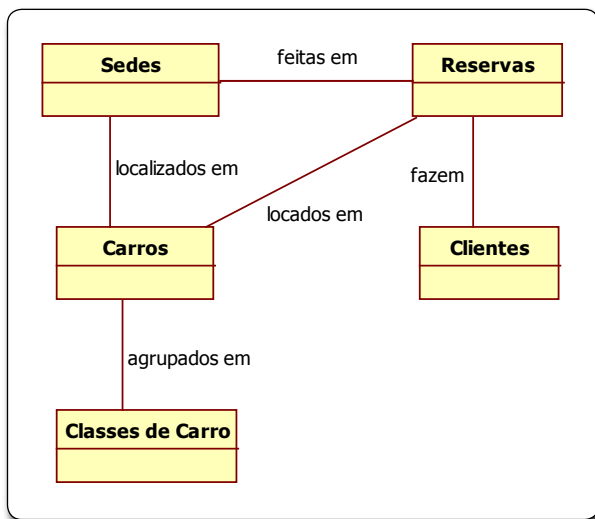


Figura 1. Modelo Conceitual do Sistema da ACDN Rental Car

A Figura 1 apresenta a representação gráfica do modelo conceitual, descrevendo suas classes e relacionamentos conforme já descrito anteriormente.

Etapa 2: Modelagem Lógica

Para a modelagem lógica, iremos ainda utilizar um Diagrama de Classes da UML, porém contendo maiores informações sobre os contendo e relacionamentos identificados, como seus atributos e associações.

A Figura 2 apresenta a representação gráfica do modelo lógico para nosso estudo de caso. Observe as diferenças entre o Modelo Conceitual (Figura 1) e Modelo Lógico (Figura 2), dentre as quais:

- A relação entre as classes *Sedes* e *Reservas* foi dividida em 2, explicitando duas operações distintas: a reserva de um carro em uma sede e a devolução de um carro em uma sede, que pode ser ou não a mesma sede onde este foi locado.
- A relação entre as classes *Sedes* e *Carros* também dividida em 2 para que pudessem ser registradas informações sobre duas situações distintas: um carro possui uma sede como um ponto de origem de onde pode ser reservado (locado), mas um carro pode estar localizado em uma sede diferente de onde é seu ponto de origem (exemplo: quando este for devolvido em uma sede diferente de seu ponto de origem).
- Cada classe ganhou seus atributos, que as caracterizam dentro do problema a ser resolvido.

A lista de atributos por classe e sua justificativa está apresentada na Tabela 2.

A multiplicidade dos relacionamentos também precisam ser explicadas, e faremos isso a seguir:

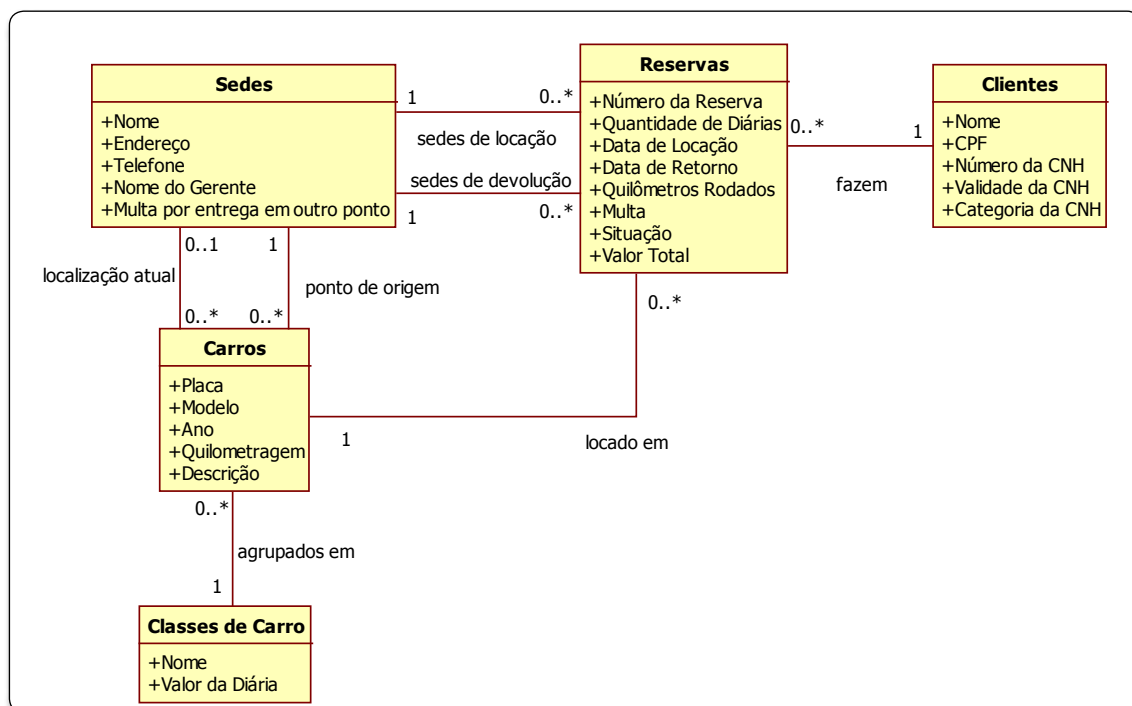


Figura 2. Modelo Lógico do Sistema da ACDN Rental Car

- (Sedes-Reservas [locação]): uma reserva só pode ser realizada em uma única sede (não existe reserva compartilhada entre sedes), porém uma sede pode ter nenhuma, uma ou várias reservas realizadas.
- (Sedes-Reservas [devolução]): de maneira análoga à anterior, uma reserva só pode ser finalizada (entrega do carro) em uma única sede, porém uma sede receber várias devoluções de carros, sem limites.
- (Sedes-Carros [ponto de origem]): um carro pode não estar localizado em uma sede (quando está alugado) ou estar localizado em apenas uma sede por vez, podendo esta sede ser diferente de seu ponto de origem.
- (Sedes-Carros [ponto de origem]): um carro pode não estar localizado em uma sede (quando está alugado) ou estar localizado em apenas uma sede por vez, podendo esta sede ser diferente de seu ponto de origem.
- (Classes de Carro-Carros): um carro só pode estar agrupado em uma única classe de carros, que por sua vez pode possuir nenhum, um ou vários carros, sem limites.
- (Carros-Reservas): uma reserva só pode estar associada a um carro, não existindo reservas com vários carros simultaneamente. No entanto, um carro pode possuir várias reservas ao longo do tempo, porém elas não podem ser realizadas em um

mesmo intervalo de tempo (não é possível alugar um carro já alugado), conforme indicado nas regras do sistema.

- (Clientes-Reservas): uma reserva só pode estar associada a um cliente, não existindo reservas com vários clientes associados a ela. No entanto, um cliente pode realizar várias reservas ao longo do tempo, porém elas não podem ser realizadas em um mesmo intervalo de tempo (um cliente só pode realizar uma nova reserva se a primeira estiver finalizada), conforme indicado nas regras do sistema.

Na próxima seção realizaremos a construção do banco de dados projetado até o momento usando o MySQL, consistindo em nosso modelo físico.

Etapa 3: Construção do Banco de Dados usando MySQL

O primeiro passo para construção do nosso banco de dados é exatamente criar o banco de dados no MySQL. Para isso, devemos abrir o MySQL e digitar o código SQL apresentado na **Listagem 1**.

Podemos confirmar a criação do banco de dados usando o comando SHOW DATABASE, conforme a **Listagem 2**.

Feito isso, o passo seguinte consiste em construir as tabelas que irão compor nosso banco de dados. As tabelas devem ser

Atributo	Justificativa
Sedes	
Nome	Nome dado para a sede da empresa
Endereço	Endereço onde a sede está localizada
Telefone	Telefone para contatar a sede
Nome do Gerente	Responsável por esta sede
Multa por entrega em outro ponto	Valor a ser pago pelo cliente caso entregue um carro alugado nesta sede em outra localização (sede)
Carros	
Placa	Placa de identificação do veículo
Modelo	Modelo do veículo
Ano	Ano do modelo e de fabricação
Cor	A cor do veículo
Quilometragem	A quilometragem corrente do veículo
Descrição	Descreve mais detalhes sobre o carro
Situação	Indica se o veículo está "disponível", "alugado" ou "fora do ponto de origem".
Classes de Carro	
Nome	O nome dado à classe de carros, conforme o padrão adotado pela empresa (descrito no enunciado)
Valor da Diária	O valor pago por dia para locação de carros nesta classe
Clientes	
Nome	Nome do cliente
CPF	CPF do cliente a fim de identificação
Número da CNH	Número da Carteira Nacional de Habilitação, confirmando que o cliente está apto a dirigir
Validade da CNH	Data de validade da CNH do cliente
Categoria da CNH	A categoria de veículos que pode dirigir
Reservas	
Número da Reserva	Número gerado para reserva
Quantidade de Diárias	Quantidade de dias em que o carro ficará/ficou locado
Data da Locação	Data em que o carro foi locado
Data de Retorno	A data em que o veículo foi retornado
Quilômetros rodados	Quantidade de quilômetros rodados pelo cliente neste reserva
Multa	A multa (em reais) paga ou não pelo cliente no momento da devolução
Situação	Indicação do estado da reserva: ativa, atrasada, finalizada.
Valor total	Valor total pago na reserva, multiplicando o valor da diária pela quantidade de dias que o carro ficou locado e somando-se a multa que pode ter sido paga por entrega fora do ponto

Tabela 2. Lista de Atributos compondo o Modelo Lógico do Sistema da ACDN Rental Car

construídas com os atributos descritos no modelo lógico, acrescidas de suas chaves primárias e eventuais chaves estrangeiras representando os relacionamentos entre as classes. A **Listagem 3** apresenta o código SQL para criação das tabelas representando o modelo físico do sistema da *ACDN Rental Car*. Neste momento, apenas criaremos as tabelas, sem incluir as chaves estrangeiras. Isso será feito num segundo momento.

Listagem 1. Código para criação do Banco de Dados

```
mysql> create database if not exists AcdnRentalCar;
Query OK, 1 row affected (0.03 sec)
```

Listagem 2. Visualizando o banco de dados criado

```
mysql> show databases;
+-----+
| Database |
+-----+
| AcdnRentalCar |
| test |
+-----+
2 rows in set (0.00 sec)
```

Listagem 3. Código para criação das tabelas

```
mysql> use AcdnRentalCar;
Database changed

mysql> create table sedes (
-> id int(10) unsigned not null auto_increment,
-> nome varchar(50) not null,
-> endereco varchar(80) unsigned not null,
-> telefone varchar(20) not null,
-> nomeGerente varchar(50) not null,
-> multa float(8,2) not null,
-> primary key (id));
Query OK, 0 rows affected (0.14 sec)

mysql> create table carros (
-> id int(10) unsigned not null auto_increment,
-> placa varchar(10) not null,
-> modelo varchar(40) not null,
-> ano varchar(9) not null,
-> cor varchar(20) not null,
-> quilometragem float(8,2) not null,
-> descricao varchar(100) not null,
-> situacao varchar(30) not null,
-> origemCarro int(10) not null,
-> localizacaoCarro int(10) not null,
-> classeCarro int(10) not null,
-> primary key (id));
Query OK, 0 rows affected (0.14 sec)

mysql> create table classesCarro (
-> id int(10) unsigned not null auto_increment,
-> nome varchar(20) not null,
-> valorDiaria float(8,2) not null,
-> primary key (id));
Query OK, 0 rows affected (0.14 sec)

mysql> create table clientes (
-> id int(10) unsigned not null auto_increment,
-> nome varchar(50) not null,
-> cnh varchar(20) not null,
-> validadeCnh DATE not null,
-> categoriaCnh varchar(3) not null,
-> primary key (id));
Query OK, 0 rows affected (0.14 sec)

mysql> create table reservas (
-> numero int(10) not null auto_increment,
-> diarias int(10) not null,
-> dataLocacao DATE not null,
-> dataRetorno DATE,
-> quilometrosRodados float(8,2),
-> multa float(8,2),
-> situacao varchar(15) not null,
-> total float(8,2),
-> carro_reserva int(10) not null,
-> cliente_reserva int(10) not null,
-> sedeLocacao int(10) not null,
-> sedeDevolucao int(10) not null,
-> primary key (numero));
Query OK, 0 rows affected (0.14 sec)

mysql>
```

Observe que até aqui não há nada diferente. As tabelas refletem praticamente o modelo lógico. A partir de agora iremos adicionar os relacionamentos do modelo lógico em nosso modelo físico. Podemos observar na **Figura 2** que todos os relacionamentos entre as classes possuem multiplicidade “1..*” ou “0..*”. Esse tipo de multiplicidade possibilita 2 tipos de tratamento para construção do modelo físico: (1) criar uma tabela auxiliar contendo as chaves primárias de ambas as tabelas que fazem parte do relacionamento ou (2) usar uma chave estrangeira na tabela que possui multiplicidade “*” para a chave primária da tabela que possui multiplicidade “0” ou “1” no relacionamento, que é a opção mais “elegante”. Optaremos pela segunda opção: usaremos chaves estrangeiras.

Essa decisão irá requerer mudanças na estrutura de algumas tabelas do nosso modelo. A **Listagem 4** apresenta o código SQL que realiza tais mudanças estruturais, adicionando chaves estrangeiras nas tabelas para refletir ao relacionamento entre as tabelas.

Listagem 4. Código para inclusão dos relacionamentos entre as tabelas

```
mysql> use AcdnRentalCar;
Database changed

//relacionamento entre as tabelas Sedes e Carros (ponto de origem)
mysql> ALTER TABLE `carros` ADD CONSTRAINT `fk_sedesOrigem`
FOREIGN KEY (`origemCarro`) REFERENCES `sedes` (`id`);
Query OK, 0 rows affected (0.14 sec)

//relacionamento entre as tabelas Sedes e Carros (localizacao atual)
mysql> ALTER TABLE `carros` ADD CONSTRAINT `fk_sedesLocAtual`
FOREIGN KEY (`localizacaoCarro`) REFERENCES `sedes` (`id`);
Query OK, 0 rows affected (0.14 sec)

//relacionamento entre as tabelas Sedes e Reservas (localizacao)
mysql> ALTER TABLE `reservas` ADD CONSTRAINT `fk_sedesLocacao`
FOREIGN KEY (`sedeLocacao`) REFERENCES `sedes` (`id`);
Query OK, 0 rows affected (0.14 sec)

//relacionamento entre as tabelas Sedes e Reservas (devolucao)
mysql> ALTER TABLE `reservas` ADD CONSTRAINT `fk_sedesDevolucao`
FOREIGN KEY (`sedeDevolucao`) REFERENCES `sedes` (`id`);
Query OK, 0 rows affected (0.14 sec)

//relacionamento entre as tabelas Carros e Classes Carros
mysql> ALTER TABLE `carros` ADD CONSTRAINT `fk_classes` FOREIGN
KEY (`classeCarro`) REFERENCES `classesCarro` (`id`);
Query OK, 0 rows affected (0.14 sec)

//relacionamento entre as tabelas Carros e Reservas
mysql> ALTER TABLE `reservas` ADD CONSTRAINT `fk_carros` FOREIGN
KEY (`carro_reserva`) REFERENCES `carros` (`id`);
Query OK, 0 rows affected (0.14 sec)

//relacionamento entre as tabelas Clientes e Reservas
mysql> ALTER TABLE `reservas` ADD CONSTRAINT `fk_clientes`
FOREIGN KEY (`cliente_reserva`) REFERENCES `clientes` (`id`);
Query OK, 0 rows affected (0.14 sec)
```

Com isso, temos o nosso banco de dados totalmente criado, respeitando todos os relacionamentos e multiplicidade dos relacionamentos descritos no modelo lógico apresentado na seção anterior. Com isso, registros já podem ser inseridos nas tabelas, respeitando as regras de negócio. Observe que algumas das regras não foram implementadas no banco de dados (exemplo: não permitir que um cliente com carro alugado realize uma segunda reserva). Neste caso, deverá ser papel da aplicação fazer tais controles.

Etapa 4: Próximos Passos

Esta seção não é o foco principal do artigo. Ela possui apenas o objetivo de alertar aos leitores e projetistas de banco de dados que o nosso trabalho não termina no momento em que criamos o banco de dados de uma aplicação. O gerenciamento de um banco de dados é tão ou mais importante que a sua criação.

Sendo assim, o DBA deve ficar atento a diversas características do banco de dados, tais como: desempenho das consultas (queries) e operações de manipulação de dados, taxa de utilização de CPU, disponibilidade do banco de dados, dentre outras. Tais características requerem diferentes estratégias para manter o banco de dados sempre pronto para exercer seu objetivo principal no contato de uma aplicação. Dentre tais estratégias podemos citar:

- Estratégias de Tuning: consiste em realizar ajustes na estrutura do banco de dados a fim de se obter um melhor desempenho nas operações do banco de dados. Isso inclui a criação de índices, desnормализação das tabelas, dentre outras possíveis estratégias.
- Estratégias de Backup: consiste em realizar cópias periódicas do banco de dados (estrutura e conteúdo) a fim de se prevenir de eventuais perdas de dados ocasionadas por falhas ou outros problemas que podem ocorrer em um banco de dados durante sua utilização.
- Estratégias de Replicação de Dados: consiste em manter mais de um servidor de banco de dados ativo ou em alerta a fim de substituir o servidor principal em caso de falhas técnicas ou naturais que podem ocorrer em servidor de banco de dados.

Conclusões

Este artigo teve como objetivo apresentar uma possível estratégia para modelagem e projeto de banco de dados, iniciando de seu levantamento inicial até a sua construção física em um SGBD. Para isso, utilizamos um estudo de caso de uma locadora de carros, que nos permitiu discutir uma série de questões importantes que podemos nos confrontar durante o desenvolvimento de um banco de dados. Ao final, o banco de dados foi construindo usando o MySQL, e este estaria pronto para ser utilizado em uma aplicação real.

Mais uma vez é importante ressaltar que diversas regras de negócio descritas no problema não foram tratadas através do banco de dados. Nesse caso, tais tratamentos devem ser realizados pela aplicação durante a sua codificação. Esta é apenas uma possível solução de modelagem para o problema apresentado. Diversas outras poderiam resolver o mesmo problema com a mesma ou até maior eficiência. Abraços e até a próxima! ●

Dê seu feedback sobre esta edição!

A SQL Magazine tem que ser feita ao seu gosto. Para isso, precisamos saber o que você, leitor, acha da revista!

Dê seu voto sobre este artigo, através do link:

www.devmedia.com.br/sqlmagazine/feedback



Conhecimento faz diferença!



Faça um up grade em sua carreira.

Em um mercado cada vez mais focado em qualidade ter conhecimentos aprofundados sobre requisitos, metodologia, análises, testes, entre outros, pode ser a diferença entre conquistar ou não uma boa posição profissional. Sabendo disso a DevMedia lança para os desenvolvedores brasileiros sua primeira revista digital totalmente especializada em Engenharia de Software. Todos os meses você irá encontrar artigos sobre Metodologias Ágeis; Metodologias tradicionais (document driven); ALM (application lifecycle management); SOA (aplicações orientadas a serviços); Análise de sistemas; modelagem; Métricas; orientação à objetos; UML; testes e muito mais. Assine já!



Nesta seção você encontra artigos sobre banco de dados, SQL ou persistência



Questões sobre banco de dados do concurso do TRE do Paraná

De que se trata o artigo?

Apresentar a correção das questões com conteúdo de banco de dados do concurso público do TRE do Paraná para as carreiras de Analista e Programador de sistemas.

Para que serve?

O conteúdo apresentado neste artigo serve para treinar candidatos e estudantes de banco de dados para futuras provas e concursos.

Em que situação o tema é útil?

A resolução das questões apresentadas neste artigo é útil para o candidato que almeja seguir a carreira de servidor público em um dos diversos cargos relacionados à área de tecnologia da informação, mais especificamente para aqueles que precisam estudar o conteúdo de banco de dados voltados para a programação de sistemas.

O Tribunal Regional Eleitoral (TRE) do estado do Paraná abriu concurso público em setembro de 2009 para preencher 27 vagas do cadastro de reserva. Os cargos disponíveis neste concurso foram divididos em dois grupos: analistas judiciários e técnicos judiciários.

No grupo de analistas judiciários foi criada a vaga de especialista em Análise de sistemas, vaga esta que contava com remuneração de R\$ 6.611,39 mensais por 40 horas semanais de trabalho. Os candidatos a esta vaga deveriam possuir um diploma, devidamente registrado, de conclusão de curso de nível superior na área de Informática, fornecido por uma instituição de ensino superior reconhecida pelo Ministério da Educação.



Mauro Pichiliani

pichiliani@uol.com.br

É bacharel em Ciência da Computação, mestre pelo ITA (Instituto Tecnológico de Aeronáutica) e MCP, MCDBA e MCTS. Trabalha há mais de 9 anos utilizando diversos bancos de dados, como o SQL Server, Oracle e MySQL. É colunista de banco de dados do web site iMasters. (<http://www.imasters.com.br>).

No grupo de técnicos judiciários foi criada a vaga de Programador de sistemas, que contava com remuneração de R\$ 4.052,96 mensais por 40 horas semanais de trabalho. Os candidatos a esta vaga deveriam possuir um certificado,

devidamente registrado e expedido por uma instituição de ensino reconhecida pelo Ministério da Educação, de conclusão de curso técnico com ênfase em manutenção e suporte em informática ou redes de computadores (antigo segundo grau profissionalizante), ou de ensino médio (antigo segundo grau), acrescido 180 horas, devidamente comprovadas, de capacitação na área de operação de computadores, redes de computadores, manutenção de equipamentos de computação, bancos de dados ou sistemas operacionais.

A instituição organizadora do concurso foi o CESPE (Centro de Seleção e Promoção de Eventos), entidade vinculada à Universidade Federal de Brasília (UnB). As provas para ambos os cargos de Analista e Programador de sistemas continham 40 questões que cobravam conhecimentos básicos (português, raciocínio lógico e noções de informática, direito administrativo e eleitoral) e 80 questões que cobravam conhecimentos específicos. Todas as 120 questões deveriam ser respondidas apenas como C (certa) ou E (errada) durante a prova cuja duração máxima foi de 3 horas e 30 minutos.

O edital indicou que os candidatos ao cargo de Analista de sistemas deveriam possuir conhecimentos em redes, sistemas operacionais, programação, lógica, infraestrutura e bancos de dados (Oracle e MySQL) dentre outros. Já para a prova de Programação de sistemas a ênfase dos conhecimentos necessários foi dada à programação, bancos de dados e noções de ITIL e COBIT. A prova para o cargo de Programação de sistemas contou com 14 questões específicas de bancos de dados e a prova para o cargo de Análise de sistemas contou com apenas duas questões de bancos de dados.

Neste artigo será apresentada a correção apenas das questões com conteúdo de banco de dados da prova para as carreiras de Analista e Programação de sistemas. As questões serão apresentadas, corrigidas e comentadas junto com o gabarito oficial.

Questões do cargo de Programação de sistemas

Julgue os itens a seguir, a respeito de projeto e modelagem de banco de dados relacional.

81) A derivação de uma especialização do modelo lógico para o projeto físico de um banco de dados relacional não admite múltiplas formas. Cada entidade especializada dá origem a uma relação.

Gabarito: Errado

Comentário: Esta questão diz respeito à derivação de um modelo lógico de banco de dados para um modelo físico, ou seja, de como transformar o que está no modelo lógico para o modelo físico para a posterior implementação em um banco de dados relacional. O enunciado afirma que uma especialização modelada em um modelo lógico será implementada em um banco de dados relacional apenas de uma única forma e que cada entidade especializada será representada como uma relação.

As afirmações contidas no enunciado desta questão estão incorretas. Em primeiro lugar, o modelo lógico é abstrato e conceitual e por isso deve ser criado antes do modelo físico e da implementação deste modelo em um banco de dados relacional.

Isso quer dizer que um modelo lógico ainda sofrerá modificações, melhorias e refinamentos e o que os objetos modelados no modelo lógico podem não existir no modelo físico. É comum indicar relacionamentos de generalização e especialização no modelo lógico, porém eles podem ser implementados de diversas formas, como tabelas com auto-relacionamentos, relacionamentos 1 para N ou mesmo em apenas uma tabela com várias colunas que implementem a especialização. Isso quer dizer que a especialização nem sempre pode resultar em uma relação. Devido às informações erradas no enunciado, o candidato deve marcar esta questão como errada.

82) A preservação da dependência é uma meta do projeto de banco de dados relacional, de forma que a validação das atualizações no banco de dados possa ser executada sem o uso de junções, não resultando em relações ilegais.

Gabarito: Certo

Comentário: Esta questão cobra do candidato conhecimentos sobre dependências de um projeto de banco de dados. Apenas para relembrar, recomenda-se que um banco de dados esteja na terceira forma normal (3FN) para evitar redundâncias e outros problemas. As três formas normais são:

- 1) Uma estrutura de dados encontra-se na **primeira forma normal (1FN)** se todas as suas colunas não apresentarem atributos multivalorados, ou seja, atributos que podem conter mais de um valor para uma mesma instância da entidade como, por exemplo, o telefone (comercial, residencial e celular);
- 2) Uma estrutura de dados encontra-se na **segunda forma normal (2FN)** se já estiver na primeira forma normal (1FN) e se todas as suas colunas que não são chave primária não apresentarem a anomalia da dependência funcional parcial em relação à chave primária. A anomalia da dependência funcional parcial ocorre quando a chave primária é composta por dois ou mais atributos (colunas), e outros atributos não chave dependem de parte dessa chave primária;
- 3) Uma estrutura de dados encontra-se na **terceira forma normal (3FN)** se já estiver na segunda forma normal (2FN) e se não existir a anomalia da dependência transitiva ou indireta entre um atributo e a chave primária. Portanto, uma estrutura de dados estará na terceira forma normal (3FN) se todos os seus atributos dependerem funcionalmente apenas da chave primária e de nenhum outro atributo.

A aplicação das três formas normais elimina dependências funcionais parciais e também dependências transitivas ou indiretas entre um atributo e a chave primária. O tipo de dependência à qual o enunciado se refere está relacionada à integridade referencial, ou seja, a dependência entre uma chave primária e estrangeira. As três formas normais geram este tipo de dependência permitindo que as validações das atualizações possam ser realizadas sem o uso de junções. Devido a isto, o enunciado da questão está correto e o candidato deve marcar esta questão como certa.

Com relação à modelagem dimensional, julgue os itens que se seguem.

83) Modelagem dimensional é uma técnica de projeto físico de banco de dados usada no desenvolvimento de data warehouses, e que também é aplicada ao projeto de sistemas de informações operacionais.

Gabarito: Errado

Comentário: A modelagem dimensional é uma técnica de modelagem conceitual de negócios que facilita a investigação, o resumo e a organização de dados para a análise de indicadores relevantes ao negócio. Esta modelagem envolve a especificação da estrutura e relações entre dimensões e fatos. O modelo dimensional relaciona tabelas de fatos com tabelas de dimensões em um banco de dados do tipo OLAP, possibilitando inúmeras combinações. Além disso, esta modelagem é realizada de forma a ganhar performance nas consultas, possuindo regras diferentes da modelagem relacional padrão, ou seja, a aplicação de formas normais no modelo. O modelo multidimensional visa somente consultas analíticas em um data warehouse e não é adequada e nem recomendada para sistemas de informações operacionais, como ERPs e CRMs. Devido a esta característica a questão deve ser marcada como errada.

84) Snowflaking, uma técnica de modelagem dimensional utilizada para salvar espaço em disco, não é recomendada para ambientes de data warehouse, pois torna a sua apresentação mais complexa ao aumentar o número de tabelas no banco de dados.

Gabarito oficial: Certo. Opinião do autor: Errado

Comentário: Esta questão cobra do candidato um importante conceito relacionado à modelagem dimensional: a técnica de *design* conhecida como snowflake.

Esta técnica, também conhecida como esquema floco-de-neve (*snowflake schema*) propõe uma visão para a modelagem de base de dados para sistemas de apoio a decisão. No *design* floco-de-neve as tabelas dimensionais estão ligadas a uma tabela de fatos central, porém existem tabelas ligadas diretamente às tabelas dimensão e que não possuem um relacionamento direto com a tabela fato. O nome foi adotado devido à semelhança do modelo com um floco de neve que contém diversas “pontas” ligadas a cada uma das ramificações centrais do modelo. No “centro” deste modelo existe uma tabela de fatos, rodeada por tabelas auxiliares, chamadas de dimensões que, por sua vez, possuem seus próprios relacionamentos. Quando for preciso obter informações sobre as tabelas de dimensão é preciso realizar diversas operações de junção entre as tabelas que se relacionam com a tabela de dimensão. O resultado destas junções é então aplicado em mais uma junção com os dados da tabela fato. Uma implicação do uso da técnica *snowflake* é o aumento do número de tabelas no banco de dados que podem tornar o processamento mais complexo devido à necessidade de junções entre as tabelas de dimensão e a tabela fato.

Contudo, o enunciado da questão está errado. A princípio a técnica *snowflake* gera mais tabelas que podem tornar o processamento mais complexo. Mais tabelas podem até tornar a apresentação do modelo mais complexa, porém isso não quer dizer que a apresentação dos dados para o usuário será mais

complexa, uma vez que esta apresentação dos dados geralmente envolve conceitos de usabilidade da interface gráfica que não são relacionados diretamente ao processamento dos dados. Além disso, em muitas situações a técnica *snowflake* é recomendada, principalmente em modelos híbridos que utilizam a técnica *snowflake* em conjunto com outras técnicas. Quanto à afirmação do espaço em disco, é preciso avaliar o custo de armazenamento de acordo com diversos fatores de implementação e não durante a fase de modelagem. Ou seja, é possível que modelos que utilizem a técnica *snowflake* gastem mais espaço de armazenamento que outros.

Apesar disso, o gabarito oficial indica que a questão está certa. Com base no que foi apresentado anteriormente, esta questão cabe recurso solicitando a sua anulação ou a mudança do gabarito oficial. Provavelmente a intenção do avaliador foi simplificar e desmerecer a técnica *snowflake* devido à maior quantidade de tabelas envolvidas, entretanto isso não é o suficiente para generalizar e dizer que esta técnica não é recomendada para ambientes de data warehouse. Se o avaliador indicasse que um autor específico fez esta afirmação ou um contexto específico seria possível considerar a questão correta, porém não é este o caso.

85) A alteração de informações em bancos de dados dimensionais pode ser realizada sobrepondo-se o registro de dimensão com novos valores, sem a perda do histórico.

Gabarito: Errado

Comentário: Nesta terceira questão do concurso que cobra assuntos relacionados à implementação de um B.I. (*Business Intelligence*) o enunciado diz que informações de dados dimensionais podem ser sobrepostas com novos valores sem perder dados, ou seja, é possível guardar novos dados sobrepondo os antigos e não perder nenhuma informação. Este enunciado está errado, pois um conceito básico de banco de dados diz que quando uma sobreposição de informações substitui um valor antigo por um valor novo o valor antigo é perdido. Este princípio também se mantém em bancos de dados dimensionais, caracterizando a questão como errada.

Julgue os itens subsequentes, acerca da linguagem SQL.

86) O comando `create table` e a declaração `select` são utilizados, respectivamente, para a definição de tabelas e consultas aos dados. O comando `create table` cria uma tabela no banco de dados e a declaração `select` é usada para obter informações contidas em uma tabela.

Gabarito: Certo

Comentário: Esta questão cobra do candidato conhecimentos a respeito das instruções `CREATE TABLE` e `SELECT` que fazem parte do padrão SQL. A questão afirma que o comando `CREATE TABLE` é responsável pela criação da tabela em um banco de dados e que a instrução `SELECT` deve ser utilizada para retornar dados que estejam na tabela. As duas afirmações estão corretas, pois o comando `CREATE TABLE` faz parte do conjunto de instruções DDL (*Data Definition Language*) do SQL e a instrução `SELECT` faz parte do conjunto de instruções DML (*Data Manipulation Language*).

Apenas uma observação nesta questão: alguns autores se referem ao **SELECT** como um comando enquanto outros autores preferem utilizar o termo **declaração**. Os dois termos são adequados.

87) O padrão SQL-92 define os conceitos de sessão SQL e ambiente SQL. Sessões SQL provêem os comandos SQL normais, mas não oferecem os comandos **commit** e **rollback**.

Gabarito: Errado

Comentário: Esta é uma questão que cobra do candidato profundos conhecimentos a respeito do padrão SQL. Em geral este padrão é utilizado através das implementações dos bancos de dados como, por exemplo, o PL/SQL do Oracle.

Para obtermos o documento que contém o padrão SQL-92 é preciso adquirir uma cópia mediante pagamento. Contudo, podemos consultar algumas versões preliminares deste documento chamadas de *draft* ou rascunho. Apesar de não conter a versão final, os *drafts* podem nos auxiliar a compreender o que o padrão determina.

De acordo com o segundo rascunho de revisão informal do padrão SQL-92 publicado em 30 de julho de 1992, os conceitos de sessões SQL (*SQL-sessions*) e ambiente SQL (*SQL-Environment*) são definidos no capítulo 4, responsável por abordar os conceitos do padrão. Ou seja, a primeira parte do enunciado está correta.

A segunda parte do enunciado diz que Sessões SQL provêem os comandos SQL normais, mas não oferecem os comandos **commit** e **rollback**. Bem, de acordo com o padrão, as instruções possíveis de uma sessão SQL são dadas pela sintaxe a seguir que é apresentado no item 12.5 do segundo rascunho de revisão informal do padrão SQL-92:

```
<SQL session statement> ::=
  <set catalog statement>
  | <set schema statement>
  | <set names statement>
  | <set session authorization identifier statement>
  | <set local time zone statement>
```

Ou seja, na definição formal de uma sessão SQL, de acordo com o padrão, apenas alguns comandos de configuração de ambiente podem ser utilizados e não os comandos normais. Isso quer dizer que o enunciado contém uma afirmação inválida, pois os comandos SQL normais não são providos por uma Sessão SQL de acordo com a sintaxe apresentada acima. Nota-se também que os comandos **commit** e **rollback** não são definidos como comandos válidos de uma sessão SQL, de acordo com a definição formal do padrão. Devido a esta característica a questão deve ser marcada como incorreta.

Destaca-se o fato que raramente o profissional precisará conhecer tão a fundo detalhes específicos do padrão SQL-92 durante o seu dia-a-dia.

88) No uso dos operadores padrão SQL-92 **union**, **intersection** e **except**, as relações participantes podem ter conjuntos de atributos diferentes.

Gabarito: Errado

Comentário: Esta questão cobra do candidato o conhecimento dos operadores **union**, **intersection** e **except**, que são operadores baseados nas operações em conjuntos. O enunciado diz que as relações que utilizem estes operadores podem ter um conjunto de atributos diferentes. De acordo com o padrão SQL-92, é preciso obrigatoriamente que todas as relações que fizerem parte de operações **union**, **intersection** e **except** tenham a mesma quantidade e tipos de dados dos atributos. Isso se deve ao fato que cada um destes operadores precisa analisar os dados de cada atributo para realizar sua função e se houverem conjuntos de atributos diferentes esta comparação será inválida. Portanto, o enunciado da questão está errado e o candidato deve marcar esta questão como errado.

89) No padrão SQL-92, o uso de uma condição de junção não é obrigatório para junções externas e é opcional para junções internas. Nesse último caso, se a condição de junção for omitida, o resultado será um produto cartesiano. Sintaticamente, a condição de junção natural aparece antes do tipo de junção.

PENSE...

QUANTO TEMPO
VOCÊ GASTARIA
PARA DESENVOLVER
COBRANÇA COM BOLETOS
BANCÁRIOS PARA
APENAS UM BANCO
NO SEU SOFTWARE

COBREBEMX

- 56 BANCOS E MAIS DE 430 CARTEIRAS DE COBRANÇA PARA IMPRESSÃO E/OU ENVIO DE BOLETO BANCÁRIO POR EMAIL;
- GERAÇÃO DE BOLETOS ON LINE;
- GERAÇÃO E LEITURA DE ARQUIVOS (REMESSA/RETORNO) NOS PADRÕES FEBRABAN E CNAB;
- MAIS DE 40 EXEMPLOS EM DIVERSAS LINGUAGENS DE PROGRAMAÇÃO

DOWNLOADS E INFORMAÇÕES EM WWW.COUREBEM.COM

obrebem
Tecnologia

Gabarito: Certo

Comentário: Nesta questão o avaliador pretende verificar se o candidato conhece bem os tipos e usos de junções em instruções SQL, que geralmente são aplicadas por meio da cláusula JOIN na instrução SELECT.

No enunciado da questão deve-se identificar que uma junção interna significa um inner join, ou seja, a especificação de uma condição de junção que retornará apenas as linhas das tabelas envolvidas que satisfizerem a condição de junção. Já uma junção externa significa um outer join, ou seja, a especificação de uma condição de junção que retornará as linhas das tabelas envolvidas que satisfizerem a condição de junção e também as linhas de tabelas envolvidas que não satisfizerem a condição de junção. Já uma junção natural é definida como uma junção interna onde o nome das colunas envolvidas na condição de junção é o mesmo e o operador de comparação é o igual (=). A colocação da condição de junção é opcional tanto para as junções internas como externas e naturais.

A questão afirma que quando não houver uma condição de junção em uma junção interna o resultado será um produto cartesiano. Esta afirmação está correta, pois a não especificação de uma condição de junção fará com que a instrução relacione cada linha de cada tabela com todas as outras linhas de todas as tabelas, o que caracteriza um produto cartesiano. Além disso, o enunciado também afirma corretamente que em uma junção natural a condição deve aparecer antes do tipo de junção, pois geralmente a palavra-chave NATURAL aparece antes da especificação do tipo de junção (inner ou outer join). Sendo assim, o candidato deve marcar esta questão como correta.

A **Listagem 1** apresenta as relações de um esquema do banco de dados relacional EMPRESA, em que as chaves primárias estão sublinhadas.

Listagem 1. Script contendo o esquema de banco de dados

```
EMPREGADO (NOME, SOBRENOME, NSS, SEXO, SALARIO, NSS_SUPERVISOR, NUD)  
DEPARTAMENTO (NOME, NUD, NSS_GERENTE, TOTAL_SAL)  
PROJETO (NOME, NUMEROP, LOCALIZAP, NUMD)  
TRABALHA_EM (NSSE, NUMP, HORAS)  
DEPENDENTE (NSSE, NOME_DEPENDENTE, SEXO, PARENTESCO)
```

Considerando essas relações, julgue os itens de 90 a 93, no que se refere à visão (view) e triggers.

90) Uma visão é uma exibição personalizada dos dados contidos em uma ou mais tabelas ou em outras visões. Nesse contexto, uma visão é uma consulta armazenada ou uma tabela virtual.

Gabarito: Certo

Comentário: Esta questão cobra do candidato o conhecimento sobre views (visões), que nada mais são do que objetos no banco de dados utilizados para encapsular instruções SELECT com vários propósitos. O enunciado diz que uma visão é uma consulta armazenada em uma ou mais tabelas e que também pode ser chamada de tabela virtual. Esta afirmação está correta, pois uma view é criada a partir de um comando SELECT que pode acessar dados de tabelas com joins ou não. Notem que

esta questão é puramente conceitual e não apresenta nenhum relacionamento direto com as tabelas apresentadas no script da **Listagem 1**.

91) Triggers são procedimentos disparados implicitamente sempre que uma tabela ou visão é modificada ou quando ocorrem algumas ações do usuário ou ações do sistema de banco de dados. O padrão SQL-92 dispõe da implementação de triggers conforme a sintaxe apresentada no trecho de código da **Listagem 2**.

Listagem 2. Trecho de código da questão 91

```
CREATE TRIGGER TOTALSAL1  
AFTER INSERT ON EMPREGADO  
FOR EACH ROW  
WHEN (NEW.NUD IS NOT NULL)  
UPDATE DEPARTAMENTO  
SET TOTAL_SAL=TOTAL_SAL + NEW.SALARIO  
WHERE NUD=NEW.NUD;
```

Gabarito: Errado

Comentário: Esta questão cobra do candidato o conhecimento de um dos objetos mais importantes de um banco de dados relacional: os triggers, que também são conhecidos como gatilhos. Estes objetos são utilizados para a implementação de regras de negócio e são automaticamente acionados sempre que ações pré-determinadas ocorrerem.

Para responder a esta questão o candidato deve separar o enunciado da questão em duas partes. A primeira parte contém a definição do que é um trigger e está parcialmente correta, pois este objeto pode estar associado a uma tabela e o seu disparo é implícito e automático de acordo com ação na qual ele está associado. Contudo, não é possível associar triggers a views. Já a segunda parte do enunciado diz que os triggers são implementados no padrão SQL-92 de acordo com o trecho de código da **Listagem 2**. Apesar do padrão SQL-92 definir como triggers devem ser implementados, o código apresentado na **Listagem 2** não segue o padrão SQL-92 porque ele é todo baseado no PL/SQL, ou seja, a codificação segue uma implementação específica do padrão SQL-92 do produto Oracle. Como apenas uma parte do enunciado é verdadeiro podemos marcar a questão como errada.

92) Considere o seguinte trecho de código SQL da **Listagem 3**.

Listagem 3. Trecho de código da questão 92

```
CREATE VIEW TRABALHA_EM1  
AS SELECT NOME, SOBRENOME, NOME_P, HORAS  
FROM EMPREGADO, PROJETO, TRABALHA_EM  
WHERE NSS=NSSE AND NUMP=NUMEROP;
```

Utilizando-se as tabelas da base por meio do uso de duas junções, pode-se obter o mesmo resultado do código da **Listagem 3**, conforme a seguir (**Listagem 4**).

Gabarito oficial: Certo. Opinião do autor: Errado

Comentário: Esta questão apresenta ao candidato uma view que relaciona as tabelas EMPREGADO, PROJETO e

TRABALHA_EM definidas na **Listagem 1**. Em seguida, a questão apresenta as **Listagens 3 e 4** indicando que pode-se obter o mesmo resultado utilizando-se tabelas da base por meio do uso de duas junções.

Listagem 4. Segundo trecho de código da questão 92

```
SELECT NOME, SOBRENOME
FROM TRABALHA_EM1
WHERE NOMEP='ProjetoX';
```

O enunciado desta questão é confuso. A **Listagem 3** cria uma view chamada TRABALHA_EM1 que relaciona as tabelas EMPREGADO e TRABALHA_EM pelas colunas NSS e NSSE, respectivamente. Depois a instrução relaciona a tabela PROJETO com a tabela TRABALHA_EM pelas colunas NUMP e NUMEROP, respectivamente. Sendo assim, o resultado da execução do código da **Listagem 3** é a criação de uma view chamada TRABALHAR_EM1. Até este ponto o enunciado não contém nenhum problema.

Contudo, em seguida o enunciado se perde, dizendo que utilizando-se as tabelas da base por meio do uso de duas junções pode-se obter o mesmo resultado do código da **Listagem 3**, conforme a **Listagem 4**. Ora, o código da **Listagem 4** não cria uma view e por isso ele nunca poderá obter o mesmo resultado da **Listagem 3**. Além do mais, a **Listagem 4** faz uso da view, inclui um novo filtro na coluna NomeP e retorna apenas duas colunas (NOME e SOBRENOME), ou seja, os resultados da execução das duas listagens serão bem diferentes.

Entretanto, de acordo com o gabarito oficial esta questão está marcada como Certa. Provavelmente o que o examinador quis perguntar foi o seguinte: é possível obter dois resultados iguais utilizando uma view e utilizando as tabelas na qual esta view se baseia? Porém esta pergunta é confusa no enunciado e nos códigos apresentados, o que abre a possibilidade de recurso solicitando a anulação da questão ou a mudança do gabarito de certo para errado.

93) No SQL, o mesmo nome pode ser aplicado a mais de um atributo, desde que os atributos estejam em relações diferentes. Por isso, no caso de uma consulta das relações EMPREGADO e DEPARTAMENTO utilizando-se o atributo NUD, seria necessário o uso de qualificadores e de alias para evitar ambiguidade.

Gabarito: Certo

Comentário: Esta questão cobra do candidato o conhecimento da instrução SELECT e, mais especificamente, de como a linguagem se comporta se houverem atributos (ou colunas) com o mesmo nome em tabelas. De acordo com o enunciado, é possível colocar mais de um atributo com o mesmo nome em tabelas e, neste caso, para utilizar os atributos com o mesmo nome é preciso empregar qualificadores e *aliases* (o plural de *alias*) com o objetivo de evitar a ambiguidade.

O enunciado da questão 93 está correto, pois é preciso utilizar *aliases* quando se trabalha com nomes de colunas repetidos. Geralmente utiliza-se o formato <tabela>.<coluna> para

evitar ambiguidades em instruções SQL, mas esta sintaxe pode variar. Além disso, o enunciado citou um exemplo prático ao indicar que o atributo NUD, presente nas tabelas EMPREGADO e DEPARTAMENTO definidas antes na **Listagem 1**, necessitam de qualificadores para evitar ambiguidades quando estes atributos forem utilizados em uma mesma instrução.

A respeito de transação de dados em SQL, julgue os próximos itens.

94) Os níveis de consistência especificados pelo padrão SQL-92 incluem o read repetitivo, o read com efetivação e o read sem efetivação. No read repetitivo, é permitido que as transações leiam registros, mesmo sem sua efetivação.

Gabarito: Errado

Comentário: Esta é outra questão da prova que cobra do candidato conhecimento a respeito do padrão SQL-92. Neste caso o candidato deve conhecer quais são os níveis de consistência (ou níveis de isolamento) de transação definidos no padrão SQL-92, a saber: Read repetitivo (*Repeatable Read*), Read com efetivação (*Read Committed*) e Read sem efetivação (*Read Uncommitted*). O padrão contém ainda o nível de isolamento Serializado (*Serializable*) que não foi citado no enunciado e que é o nível de isolamento padrão para as transações.

De acordo com a questão, no nível de isolamento read repetitivo é permitido que as transações leiam registros mesmo sem a sua efetivação. Esta informação é inválida, pois apenas o nível read sem efetivação permite tal ação. Nota-se que no nível de isolamento read sem efetivação é possível a ocorrência dos três fenômenos clássicos decorrentes da execução concorrente de instruções SQL: Leituras sujas (*dirty reads*), leitura não repetível (*non-repeatable read*) e registros fantasmas (*phantom records*). No nível de isolamento read repetitivo é possível ocorrer apenas o fenômeno registros fantasmas. Como o enunciado contém uma afirmação inválida sobre o nível de isolamento read repetitivo, o candidato deve marcar esta questão como errada.

95) Um sistema de banco de dados operando com base no padrão SQL-92 assegura a serialização e o retorno em cascata para operações de manipulação de dados.

Gabarito: Certo

Comentário: Esta questão cobra de forma implícita o conceito de transações. O enunciado afirma que o padrão SQL-92 assegura a serialização, isto é, o resultado gerado pela execução de um conjunto de instruções em uma determinada ordem será igual à execução deste mesmo conjunto de instruções em outra ordem. Este é um dos princípios básicos do conceito de transações e está presente no padrão SQL-92. Além disso, a questão diz que o retorno em cascata, ou seja, o cancelamento de instruções encadeadas em uma transação, será assegurado. Mais uma vez, o conceito de transações garante que o cancelamento retornará o banco de dados para um estado válido antes do início da transação, independente do nível de encadeamento das operações contidas na transação. Sendo assim, o candidato deve marcar a questão como certa.

Questões do cargo de Analista de sistemas

Com relação a SQL, julgue os itens que se seguem.

54) Na versão Oracle® 10g Release 2 (10.2), o uso da diretiva de compilação Autonomous_Transaction permite que as operações SQL, em uma sub-rotina, delimitadas pela declaração da diretiva possam ser confirmadas ou revertidas, sem confirmar ou reverter os dados da transação principal.

Gabarito: Certo

Comentário: Esta questão cobra do candidato o conhecimento de uma diretiva de compilação específica do Oracle chamada Autonomous_Transaction. Esta diretiva é utilizada em scripts codificados na linguagem PL/SQL, que é o dialeto do SQL padrão utilizado pelo Oracle.

A diretiva Autonomous_Transaction modifica a maneira que um subprograma se comporta dentro de uma transação. Um subprograma que for marcado com esta diretiva evita que seja preciso utilizar um commit ou um rollback na transação principal. Esta definição está de acordo com o enunciado da questão, ou seja, o candidato deve marcar esta questão como correta. Destaca-se que esta diretiva é raramente utilizada e possui um potencial grande de gerar problemas devido à possibilidade de alteração ou exclusão nos dados sem a necessidade de comandos específicos para a confirmação ou cancelamento das alterações ou exclusões.

55) Na versão Oracle® 10g Release 2 (10.2), triggers são procedimentos escritos em PL/SQL, Java ou C que são disparados implicitamente sempre que uma tabela ou visão é modificada ou quando ocorrem algumas ações do usuário ou ações do sistema de banco de dados. Assim como triggers, visões (views) podem ser consultadas, alteradas, ter dados inseridos ou deletados, com certas restrições. Todas as operações executadas em uma view afetam as tabelas nas quais a view se baseia.

Gabarito: Errado

Comentário: Esta questão cobra do candidato o conhecimento dos objetos de banco de dados trigger e view. A primeira parte do enunciado está parcialmente correta, pois triggers são procedimentos disparados implicitamente sempre que os dados armazenados em uma tabela são modificados diretamente pela tabela ou view ou quando ocorrem algumas ações do usuário ou ações do sistema de banco de dados. Dizer que um trigger é disparado pela modificação em uma tabela ou visão é incorreto, pois a modificação da estrutura da tabela (alteração de colunas, mudança de tipos de dados, etc.) ou da estrutura da view não dispara os triggers, porém a modificação dos dados por meio de instruções SQL que afetem tabelas ou view disparam os triggers já programados.

A segunda parte do enunciado diz que as views podem ser consultadas, alteradas, ter dados inseridos e deletados como triggers. Ora, triggers não permitem a inserção e

deleção de dados como views, pois eles são objetos diferentes e não podem ser utilizados como as views. Além disso, nem todas as operações executadas em uma view afetam as tabelas nas quais a view se baseia. Por exemplo, pode-se criar um índice em uma view que não afeta a tabela na qual a view se baseia, ou seja, o índice é da view e não da tabela. Por causa destas inconsistências no enunciado o candidato deve marcar a questão como errada.

Conclusão

A prova apresentou muitos conceitos práticos e alguns aspectos teóricos, com destaque para as questões que cobram os aspectos relacionados ao padrão SQL-92. Outro ponto que merece ser destacado é a inclusão de questões específicas sobre data warehouse, o que vem se confirmando como tendência para as provas que cobram conteúdos de banco de dados.

A prova está bem elaborada com diversas questões cobrando conceitos fundamentais para quem vai trabalhar com banco de dados. Apenas duas questões da prova do cargo de Analista de sistemas cobraram conteúdo específico do Oracle. Já a prova para o cargo de Programação de sistemas focou em princípios gerais sem recorrer a detalhes muito específicos. Apesar da prova contar com uma questão que apresenta tabelas modeladas, não houve nenhuma necessidade de análise do modelo e avaliação de instruções SQL como o SELECT, INSERT, UPDATE e DELETE, algo que é fundamental para qualquer profissional que trabalha com banco de dados.

Contudo, a prova possui algumas questões que abrem margem para a entrada de recurso. Em particular, as questões 84 e 92 da prova para o cargo de Programação de sistemas não possuem enunciado claro e deveriam ser reavaliadas pelos examinadores. Em geral, a dificuldade desta prova pode ser classificada como fácil, condizente com o cargo de nível júnior ou pleno. ●

Referências

Download do edital, gabarito e resultados:

<http://www.cespe.unb.br/concursos/TREPR2009/>

Manual Oracle® Database PL/SQL User's Guide and Reference 10g Release 2 (10.2):

http://download-west.oracle.com/docs/cd/B19306_01/appdev.102/b14261/toc.htm

Segundo rascunho de revisão informal do padrão SQL-92 publicado em 30 de julho de 1992:

<http://www.contrib.andrew.cmu.edu/~shadow/sql/sql1992.txt>

Dê seu feedback sobre esta edição!

A SQL Magazine tem que ser feita ao seu gosto. Para isso, precisamos saber o que você, leitor, acha da revista!

Dê seu voto sobre este artigo, através do link:

www.devmedia.com.br/sqlmagazine/feedback



Cursos Online



A revista **Java Magazine** oferece para seus assinantes uma série de **Cursos Online** de alto padrão de qualidade.

Conheça abaixo o curso já disponível.

Curso em destaque

Introdução ao desenvolvimento para celulares com J2ME

Confira neste curso os principais recursos do J2ME. Aprenda também com o passo a passo para criar sua primeira aplicação J2ME. Neste curso você irá aprender diversas funcionalidades desta tecnologia para desenvolvimento de dispositivos móveis.

Confira o plano de aula completo:
www.devmedia.com.br/celularesj2me

Assine a **Java Magazine** e comece já seu treinamento!
www.devmedia.com.br/assine

A sua melhor opção de aprendizagem!

Outros cursos disponíveis: www.devmedia.com.br/curso



DevMedia
GROUP

Mais Informações: www.devmedia.com.br/central - Tel.: 21 3382-5038/ 3382-5025

Nesta seção você encontra artigos sobre banco de dados, SQL ou persistência



Disponibilidade no SQL Server 2005/2008

Trabalhando com Replicação de dados no SQL Server 2005/2008 e PostgreSQL

De que se trata o artigo?

Este artigo estratégias de replicação de dados entre servidores SQL Server e entre o SQL Server e o PostgreSQL a fim de manter bancos de dados disponíveis com maior segurança para uma aplicação.

Para que serve?

Empresas que necessitam movimentar os dados entre ambientes remotos para troca de dados, geração de relatórios.

Em que situação o tema é útil?

A replicação de dados é útil para empresas que necessitam centralizar suas informações ou distribuí-las entre seus ambientes remotos.

Normalmente nas médias e grandes empresas há movimentação de dados entre diversos ambientes remotos para geração de relatórios, troca de dados com usuários móveis, integração de dados de diversas fontes heterogêneas, entre outros. Sendo necessário buscar formas de realizar essas diversas integrações.

Uma solução interessante que se aplica neste cenário é a *replicação*, onde a principal proposta é distribuir duas ou mais cópias dos dados em diferentes servidores. A estratégia de replicação pode ser utilizada também como uma

solução para tolerância a falhas, permitindo que o serviço permaneça operacional mesmo após algum problema de hardware ou software.

Este artigo tem como objetivo descrever sobre os principais conceitos sobre replicação de dados no SQL Server 2005/2008, assim como criar um estudo de caso onde será demonstrada replicação de dados entre os SGBDs SQL Server e PostgreSQL.



Priscila Azarias

pryazarias@gmail.com

Especializada em Gestão Industrial: Conhecimento e inovação. Formada pela Universidade Tecnológica Federal do Paraná (UTFPR) em Sistemas de Informações. Certificada em MCTS SQL Server 2005 e ITIL V3 Foundation. Com experiência em administração de banco de dados SQL Server 2005. Atualmente trabalha na TECMICRO SISTEMAS na gerência de implantações de sistemas ERP.

Replicação de Dados

Antes de implementar a replicação em qualquer ambiente, devemos avaliar algumas questões que são importantes nesse cenário, como a infraestrutura da rede e hardware para saber se o ambiente comporta a replicação, conhecer a estrutura e o comportamento do banco de dados, o intervalo de tempo em que dados deverão ser replicados (latência), o tipo de replicação que será utilizado e qual a topologia de replicação.

É muito importante fazer uma análise cuidadosa sobre cada item citado anteriormente, documentar e planejar todo o processo para que a replicação ocorra com sucesso e atenda todas as necessidades de negócio da empresa.

Vamos conhecer os principais papéis encontrados na replicação para iniciarmos nossos estudos sobre replicação:

- **Publisher (Publicador):** Mantém a cópia *master* dentro da arquitetura da replicação, ou seja, o banco de dados que será replicado.
- **Subscriber (Assinante):** é o banco de dados que receberá os dados replicados do *Publisher*. Pode receber as mudanças de um ou mais *Publishers*. E existem dois modos que o *Subscriber* poderá receber os dados replicados:
 - **Push Subscription:** onde o servidor que reside o *Distributor* envia os dados;
 - **Pull Subscription:** onde o servidor que está como *Subscriber* solicita os dados que devem ser replicados para o *Distribuidor*.
- **Distribuidor (Distribuidor):** é responsável em enviar os dados do *Publisher* para os *Subscriber*. É o papel mais importante dentro da arquitetura da replicação.

Topologia de Replicação

Vamos começar conhecendo as topologias de replicação, que descreve como ocorre o fluxo de dados dentro da replicação. Basicamente, podem ser resumida em dois tipos:

- **Topologia Publisher Central:** consiste de um único *Publisher* que tem um ou mais *subscribers*. Esta topologia, geralmente replica os dados de uma única fonte de dados, o *Publisher*, e envia as mudanças para um ou mais *subscribers*. A **Figura 1** demonstra esse fluxo.
- **Topologia Subscriber Central:** Consiste de um único *Subscriber* que tem um ou mais *Publisher*. As mudanças são escritas nos múltiplos *Publishers* e são consolidadas dentro de um único *Subscriber*. Normalmente é usado para consolidar múltiplos bancos de dados ou um banco de dados central de relatórios. A **Figura 2** mostra esta topologia.

Existem outras topologias, mas o que basicamente varia é o fluxo entre o *Publisher* e o *Subscriber*. Entendendo como ocorre o fluxo do envio dos dados na replicação, veremos a seguir quais são tipos de replicação que podem ser configuradas no *SQL Server 2005/2008*.

Tipos de Replicação

No *SQL Server 2005/2008* é possível encontrar três tipos de replicação:

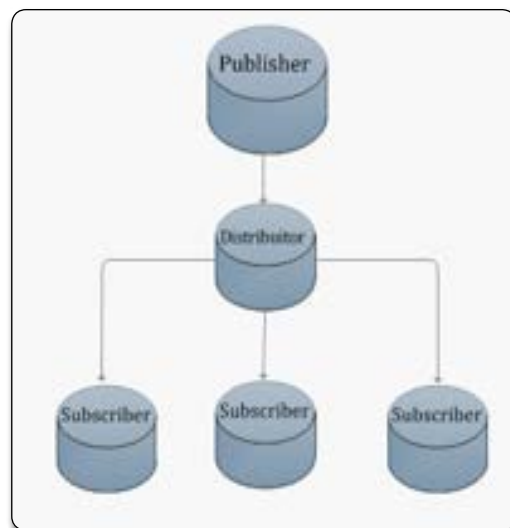


Figura 1. Topologia Publisher Central

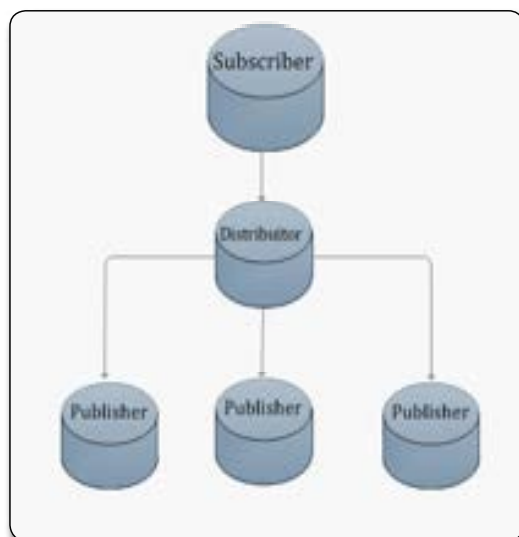


Figura 2. Topologia Subscriber Central

Replicação Snapshot: na replicação *Snapshot*, é enviada uma cópia completa dos dados em intervalos de tempos determinados. Qualquer transação que ocorra dentro do *Publisher* é capturada e enviada para o *Subscriber* no intervalo determinado. O uso da replicação *snapshot* é apropriado quando as mudanças não são frequentes nos dados e a replicação é feita em uma pequena quantidade de dados. Este tipo de replicação exige um grande tráfego de rede, pois envia uma cópia completa do *Publisher* para o *Subscriber*, não somente dos dados, mas também da estrutura.

Replicação Transactional: a replicação *transactional* é iniciada com uma fotografia (*snapshot*) do *Publisher* que é enviada para os *Subscribers* para assegurar que os ambos estão sincronizados. E as transações realizadas após esta imagem são enviadas do *Publisher* para o *Subscriber* em um fluxo incremental. A replicação *Transactional* normalmente é utilizada quando se deseja propagar mudanças incrementais para os *Subscribers*, os aplicativos requerem uma baixa latência dos dados entre as

mudanças feitas no *Publisher* e as mudanças que são enviadas ao *Subscriber* e quando o *Publisher* possui um alto número de operações de inserção, atualização e deleção de registros. Este tipo de replicação também é uma boa escolha quando se deseja manter uma cópia de segurança para disponibilidade ou geração de relatórios. Está configuração é comum em ambiente servidor-para-servidor.

Replicação Merge: a replicação *Merge* é utilizada normalmente para ambientes móveis, onde ocorre processamento desconectado. É semelhante à replicação *Transactional*, com a diferença de que as mudanças podem ocorrer tanto no *Publisher* quanto no *Subscribers*. O SQL Server tratará as informações para evitar que ocorram conflitos durante a sincronização dos dados. É apropriada para ambientes onde necessitam que os dados sejam alterados em múltiplos locais e precisam de replicação bi-direcional.

Quando se trabalha com replicação no SQL Server 2005/2008 é preciso entender que as operações ocorrem externamente do *Database Engine* e que ela ocorre através de um conjunto de executáveis conhecidos como Agentes de replicação. Os principais agentes são:

- Agente *Snapshot* (snapshot.exe): responsável por extrair o esquema e os dados que precisam ser enviados do *Publisher* para o *Subscriber*. Utilizado nos três tipos de replicação (*Snapshot*, *Transactional* e *Merge*).
- Agente *Log Reader* (logread.exe): responsável assegurar que cada transação seja escrita dentro do banco de dados *Distribution* na mesma sequência das transações realizadas no *Publisher* e depois sejam enviadas ao *Subscriber*. Utilizado somente na replicação *Transactional*.
- Agente *Distribution* (distrib.exe): possui duas funções, que são (1) aplicar *snapshots* e (2) enviar transações. É responsável em aplicar cada *snapshot* gerado com a replicação *Snapshot* e *Transactional* para todos os *Subscribers*. E é ainda responsável por aplicar todas as transações escritas pelo agente *Log Reader* no banco de dados *Distribution* para todos os *Subscribers*. Utilizado nas replicações *Snapshot* e *Transactional*.
- Agente *Merge* (replmerg.exe): responsável em aplicar os *snapshots* gerados quando o *Subscriber* é inicializado e por trocar transações entre o *Publisher* e *Subscriber*. Utilizado na replicação *Merge*.
- Agente *Queue Reader* (qrdrsvc.exe): é usado somente quando a opção *queued updating* está habilitada nas replicações *Transactional* e *Snapshot*. Ele é responsável por transferir a fila de mudanças ocorridas no *Subscriber* seja propagada para o *Publisher*.

Até este momento, tivemos uma visão geral dos principais componentes que compõe a arquitetura de replicação, o que é suficiente para iniciarmos nosso estudo de caso. Para quem desejar se aprofundar no assunto, existem excelentes artigos já publicados na Revista SQL Magazine que podem ser recomendados como leitura (ver seção Saiba Mais).

Estudo de Caso

Vamos trabalhar então! Imagine o seguinte cenário: sua empresa deseja que sejam consolidados os dados de suas filiais em um único banco de dados para a geração de relatórios sobre as vendas. Porém, uma das filiais é operada com um aplicativo antigo que utiliza *PostgreSQL* (Versão 8.3 R2) para armazenar os dados e você deve encontrar uma forma de enviar estes dados para o banco de dados que centralizará as informações para a geração dos relatórios.

Analisando os requisitos de negócio, decidiu-se que será utilizada a replicação do tipo *Transactional* para os bancos que estão em instâncias SQL Server, e para o *PostgreSQL* será utilizado *Linked Servers* para enviar os dados.

Primeiramente, vamos criar o banco de dados que será replicado (no SQL Server 2005/2008 e PostgreSQL). Além disso, criaremos apenas uma tabela (*vendas*) para simplificar o estudo de caso.

Para criação do Banco de dados e de suas tabelas no SQL Server, proceda da seguinte forma. Abra o SQL Server Management Studio (Menu Iniciar → Programas → Microsoft SQL Server 2005/2008 → Microsoft SQL Server Management Studio), insira as informações para login, solicite nova query (conforme a Figura 3) e digite o código apresentado na Figura 4 e execute a consulta.

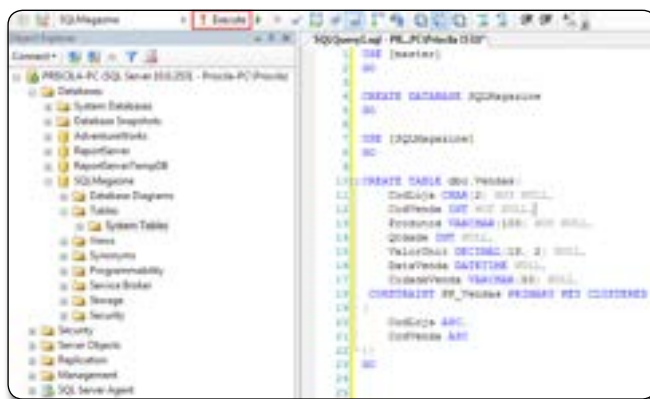


Figura 3. Solicitando uma nova query no SQL Server

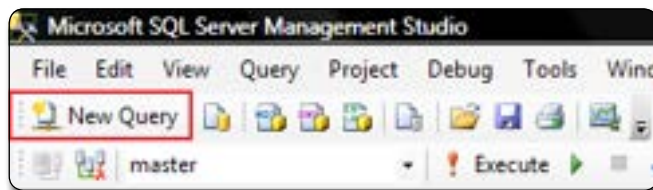


Figura 4. Executando o código de criação do BD e tabela Vendas

Para realizar a criação do BD e suas tabelas agora no PostgreSQL, que também compõe o ambiente de nosso estudo de caso, proceda da seguinte forma. Abra o *Postgres Studio* (Menu Iniciar → Programas → Postgres Plus Advanced Server 8.3 R2 → Postgres Studio). Conecte-se ao serviço, solicite uma nova SQL (Figura 5), digite o código SQL apresentado na Figura 6 e execute a consulta.

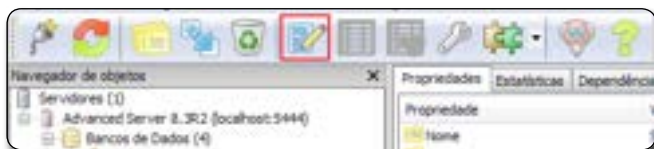


Figura 5. Criando Nova Consulta no PostgreSQL

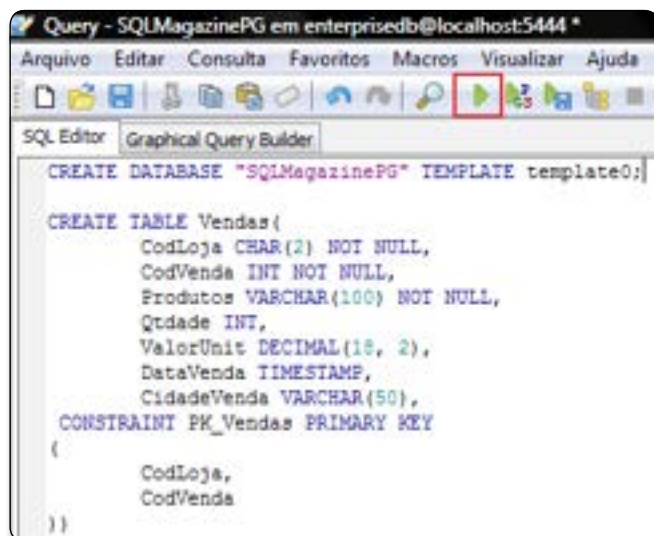


Figura 6. Executando o Script de criação do BD e tabela Vendas

Agora que já possuímos o banco de dados e as tabelas criados em ambos *SQL Server* e *PostgreSQL*, execute o código apresentado nas **Listagens 1 e 2** nos respectivos banco de dados para a inserção de alguns registros.

Listagem 1. Inserindo registros na tabela de vendas no SQL Server

```

USE [SQLMagazine]
GO

INSERT INTO dbo.Vendas(CodLoja, CodVenda, Produto, Qtdade,
ValorUnit, DataVenda, CidadeVenda)
VALUES ('MT', 1, 'Televisão', 1, 2800.00, '26/01/2010', 'São Paulo')
GO

INSERT INTO dbo.Vendas(CodLoja, CodVenda, Produto, Qtdade,
ValorUnit, DataVenda, CidadeVenda)
VALUES ('MT', 2, 'Aparelho DVD', 2, 200.00, '26/01/2010', 'São Paulo')
GO

INSERT INTO dbo.Vendas(CodLoja, CodVenda, Produto, Qtdade,
ValorUnit, DataVenda, CidadeVenda)
VALUES ('MT', 3, 'Aparelho de Som', 1, 1500.00, '26/01/2010',
'São Paulo')
GO
  
```

Listagem 2. Inserindo registros na tabela de vendas no PostgreSQL

```

INSERT INTO Vendas(CodLoja, CodVenda, Produto, Qtdade,
ValorUnit, DataVenda, CidadeVenda)
VALUES ('FL', 1, 'Guarda Roupa', 1, 3000.00, '26/01/2010', 'Curitiba');

INSERT INTO Vendas(CodLoja, CodVenda, Produto, Qtdade,
ValorUnit, DataVenda, CidadeVenda)
VALUES ('FL', 2, 'Cama BOX', 1, 1000.00, '26/01/2010', 'Curitiba');

INSERT INTO Vendas(CodLoja, CodVenda, Produto, Qtdade,
ValorUnit, DataVenda, CidadeVenda)
VALUES ('FL', 3, 'Cabeceira', 1, 400.00, '26/01/2010', 'Curitiba');
  
```

Pronto! Nosso ambiente está criado e já possuímos nossos *Publishers*. Com isso, poderemos começar a iniciar a replicação. Iniciaremos com a replicação entre as instâncias do *SQL Server*.

Replicação no SQL Server – Configurando o Distributor

Na primeira etapa, configuraremos o *Distribuidor* da replicação que desempenha um papel importante na arquitetura da replicação. Para isso, abra o *SQL Server Management Studio* e conecte-se na instância onde foi criado o banco de dados *SQLMagazine*. Vá até a pasta *Replication* e clique com o botão direito do mouse, selecionando a opção *Configure Distribution...* (Figura 7).

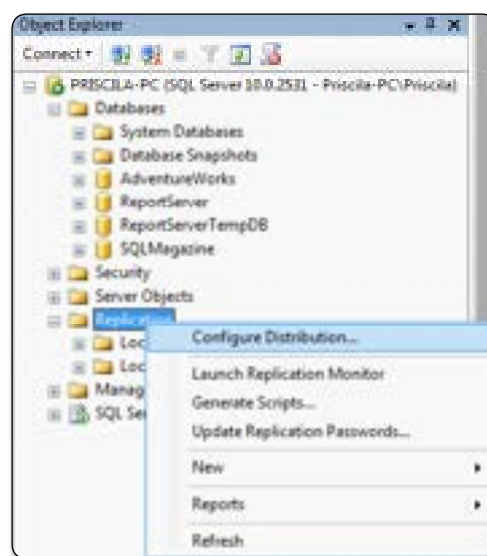


Figura 7. Configure Distribution...

Em seguida abra o assistente de configuração. Clique em *Next* na janela de apresentação. Em seguida, será necessário informar em qual instância irá residir o *Distribuidor* (Figura 8). Deixe a opção padrão, pois ela está na mesma instância que o banco de dados que será replicado no nosso estudo de caso. Caso deseje, poderá escolher outra instância em uma máquina diferente.



Figura 8. Selecionando a instância do Distribuidor

Clique em *Next*, na próxima janela deverá ser informado o local da pasta *Snapshot*. Nesta pasta serão armazenadas as informações do esquema e dos dados extraídas pelo agente *Snapshot* (Figura 9).

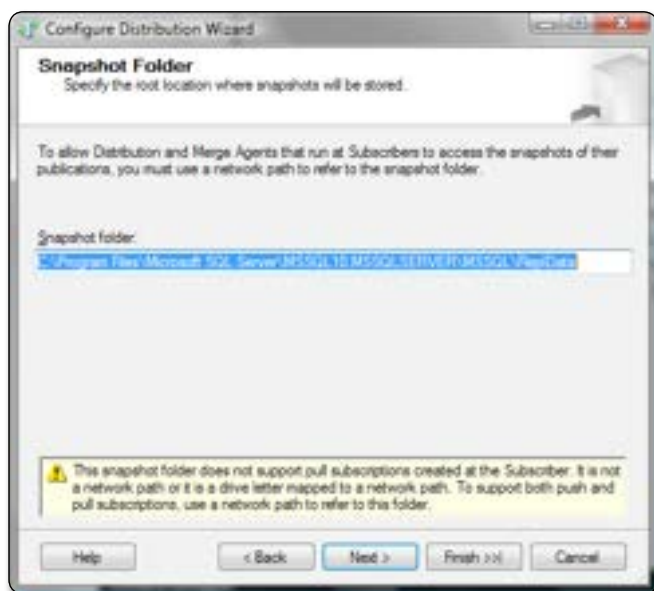


Figura 9. Local da pasta *Snapshot*

Deixe a pasta padrão e clique em *Next*. Na tela que aparece, serão solicitadas as informações sobre o nome do banco de dados do *Distribuidor* e o local dos arquivos de dados (Figura 10), deixe as opções padrão. No banco de dados *Distribution* serão armazenados todos os metadados e o histórico de todos os tipos de transações e as transações da replicação *Transactional*. Cada *Publisher* pode ser atribuído a apenas um *Distribuidor*, mas vários *Publishers* podem compartilhar o mesmo *Distribuidor* (Nota DevMan 1).

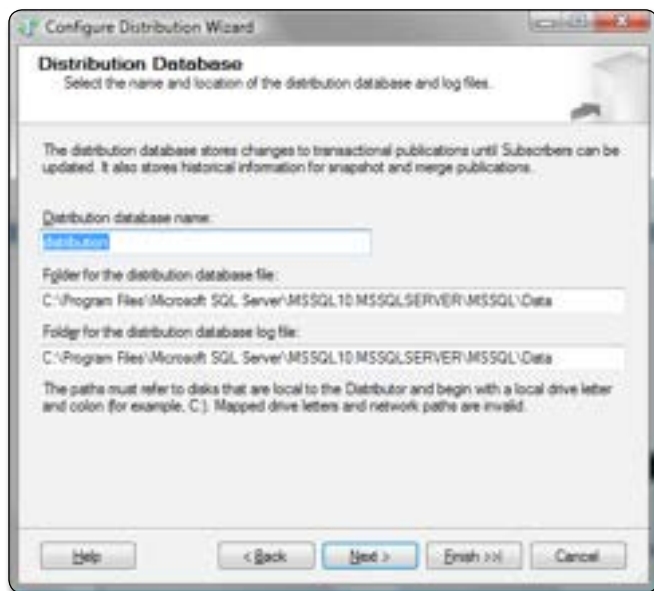


Figura 10. Banco de dados *Distribution*



Nota do DevMan 1

Banco de Dados *Distribution*

O Distribuidor usará os seguintes recursos adicionais onde estiver localizado:

- Espaço adicional em disco se os arquivos *Snapshot* estão armazenados no Distribuidor;
- Espaço adicional em disco para armazenar o banco de dados *Distribution*;
- Uso adicional de processador por agentes de replicação;

Após configurar o banco de dados *Distribution*, clique em *next*. Na próxima etapa poderemos configurar o *Publisher*. Caso já exista um *Publisher* configurado, podemos selecionar a opção *Add* → *Add SQL server Publisher* (Figura 11). Para o nosso exemplo clique em *Next*.

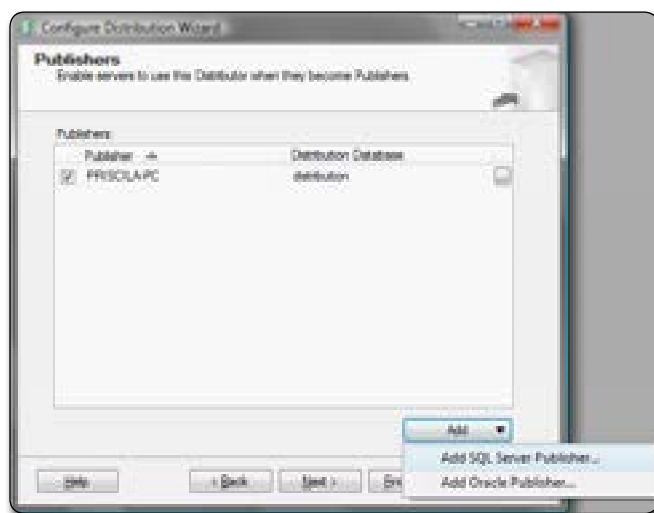


Figura 11. Adicionando o *Publisher*

A janela que aparece irá finalizar a configuração do *Distribuidor* e você terá a opção de gerar os scripts da configuração caso deseje realizar a mesma operação em outro servidor. Deixe as opções padrão e clique em *Finish*. O assistente finalizará as configurações.

A segunda parte da configuração é publicar o banco de dados que queremos replicar (Nota DevMan 2). No *SQL Server Management Studio*, expanda a pasta *Replication* e clique com o botão direito do mouse na pasta *Local Publication*. Em seguida, selecione a opção *New Publication...*, onde outro assistente será iniciado. Nesse assistente, clique em *Next* na primeira tela. Na próxima tela serão listados todos os bancos de dados que a instância possui. Nesse momento selecione o bando de dados *SQLMagazine* e clique em *Next* (Figura 12).

Replicação no SQL Server – Replicação *Transactional*

Agora vem uma importante etapa que é escolhermos o tipo de replicação que será utilizada, o que decidirá qual será o comportamento da replicação. No nosso estudo de caso usaremos a replicação *Transactional* (Nota DevMan 3). Selecione *Transaction publication* e clique em *Next*.(Figura 13).

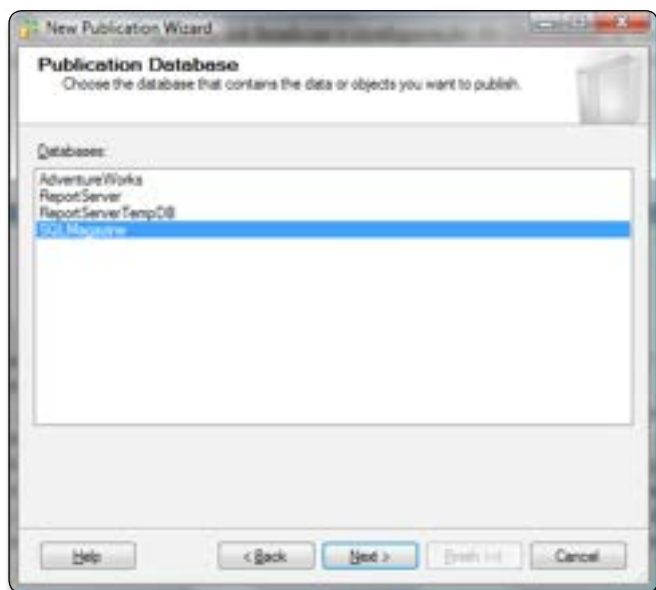


Figura 12. Selecionando o banco de dados que será replicado

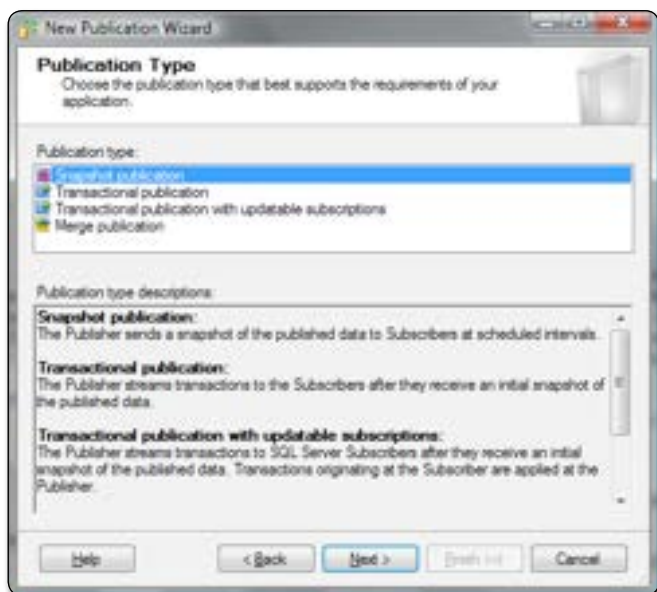


Figura 13. Selecionando o tipo de replicação

Na tela que se segue poderemos selecionar tabelas, *views*, *stored procedures* ou *functions* que irão compor a replicação. No nosso exemplo, aparece a nossa única tabela que criamos (*Vendas*) e devemos selecioná-la (Figura 14). Nesta janela também é possível aplicar os filtros de acordo com os critérios desejados.

É importante observar durante a seleção dos artigos que serão publicados a integridade dos objetos, por exemplo, se estiver replicando uma tabela que possui chaves estrangeiras, você terá que replicá-las para manter a integridade referencial.

Feito isso, clique em *Next* e aparecerá a janela para configuração dos filtros nas tabelas. Neste exemplo não aplicaremos filtros, então clique em *Next* novamente.

Na janela *Snapshot Agent* poderemos configurar como será iniciado nosso *subscribers*. Existem duas opções:

- *Create a snapshot immediately and keep the snapshot available to initialize subscriptions*, que iniciará o *Subscriber* assim que este se torne disponível;
- *Schedule the Snapshot Agent to run at following times*, é agendado uma data e hora para ser iniciado o *Subscriber*.

Deixe a primeira opção marcada e clique em *Next*. Na janela que aparece, serão solicitadas as informações de segurança do Agente *Snapshot*. Informe um usuário e senha válidos e clique em *Next* até que seja solicitado um nome para publicação (Figura 15). Nesse momento, informe **PubSQLMagazine** e clique em *finish* para finalizar a configuração.

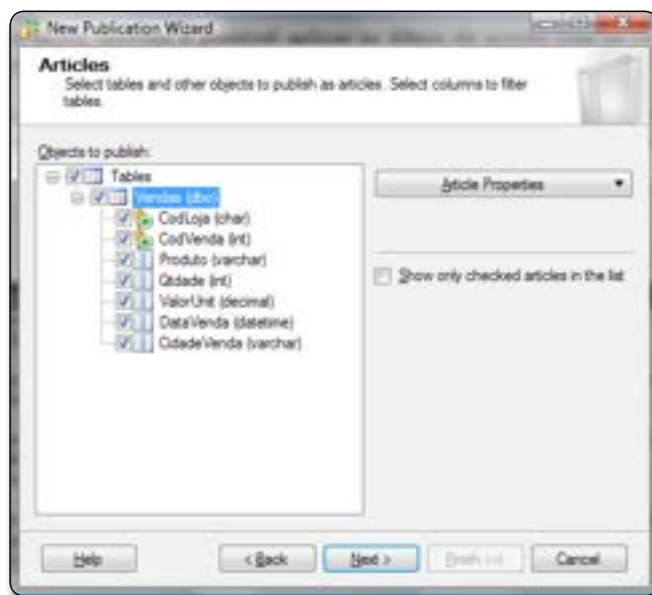


Figura 14. Selecionando os artigos que compõe a replicação



Nota do DevMan 2

Componentes da replicação

Os dados replicados são definidos usando três componentes principais:

- **Articles (Artigos):** é a construção básica da replicação que define o nível que ocorrerá a distribuição dos dados. Um artigo pode ser definido contra uma tabela, view, stored procedures e functions.
- **Publication (Publicação):** publicações são grupos de artigos definidos na replicação.
- **Filters (Filtros):** são filtros definidos que restringem um conjunto de dados replicados. Estes filtros podem ser aplicados em nível de coluna ou linha.



Nota do DevMan 3

Transactional Publication with updatable subscription

A replicação do tipo Transaction Publication with updatable subscription tem as mesmas configurações que a replicação Transaction com a diferença de que nos *Subscribers* poderão ocorrer mudanças que serão replicadas para o *Publisher*.



Figura 15. Nomeando a Publicação

Ao clicar em *finish* aparecerá uma tela mostrando as configurações que foram configuradas com sucesso, conforme a Figura 16.

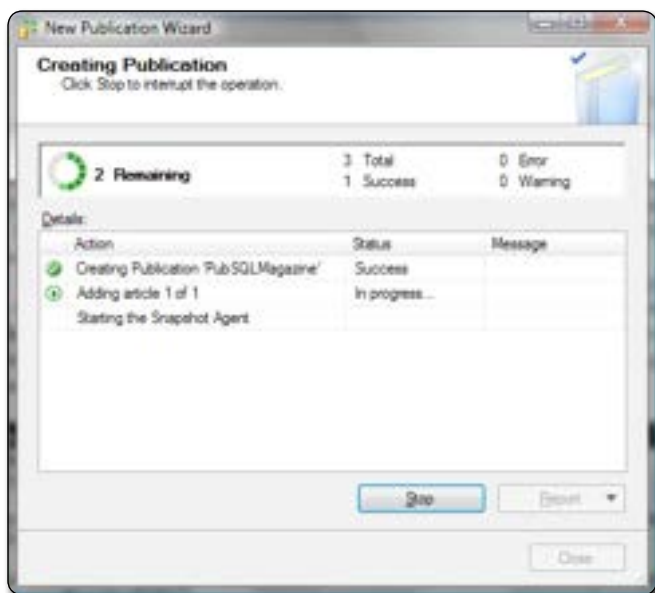


Figura 16. Criando a publicação

Ao ser finalizada a configuração, poderemos observar no SQL Management Studio a publicação que acabamos de criar, conforme a tela apresentada na Figura 17.

Replicação no SQL Server – Configurar o Subscriber

Agora a última etapa é criar o *Subscriber* que receberá os dados replicados. Para tal, conecte-se em uma segunda instância, vá até a pasta *Replication*, clique com o botão direito do mouse na pasta *Local Subscriptions* e selecione a opção *New Subscriptions* (Figura 18).

Clique em *Next* na primeira tela que aparece. Em seguida, conecte-se na instância onde configuramos a publicação (Figura 19).

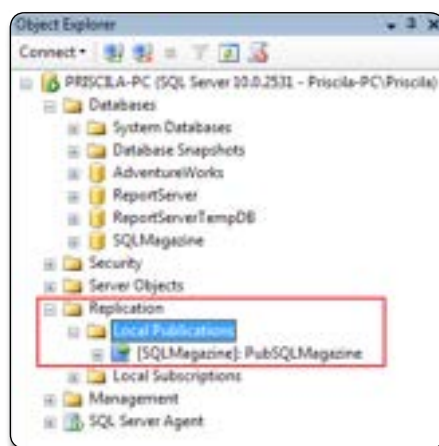


Figura 17. Publicação criada do banco de dados SQLMagazine

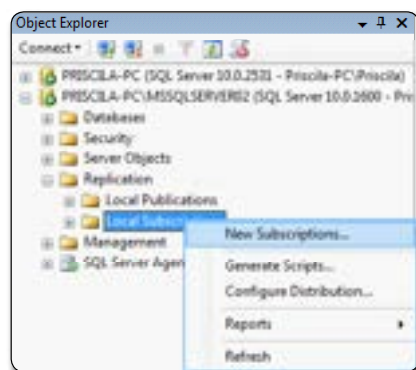


Figura 18. Novo Subscriber

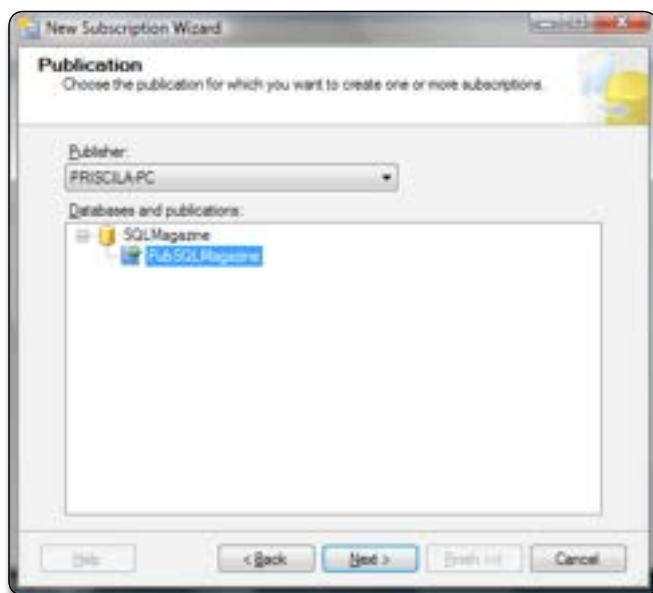


Figura 19. Localizando a publicação

Nesta tela, selecione a publicação *PubSQLMagazine* criada anteriormente e clique em *Next*. Nesta próxima tela poderemos escolher o modo como os dados serão enviados para o *Subscriber*, marque a opção *Run all agents at the Distributor...* (*push subscriptions*). Isto fará o *Distributor* enviar os dados (Figura 20). Clique em *Next*.



Figura 20. Selecionando o modo como os dados serão enviados

Na tela mostrada na **Figura 21** é possível selecionar o banco de dados para onde serão replicados os dados. Na lista que aparece em *Subscription Database*, selecione *New Database*, onde irá surgir uma nova tela para inserir as informações do novo banco de dados. Nomeie-o como *SubSQLMagazine* e clique em OK.

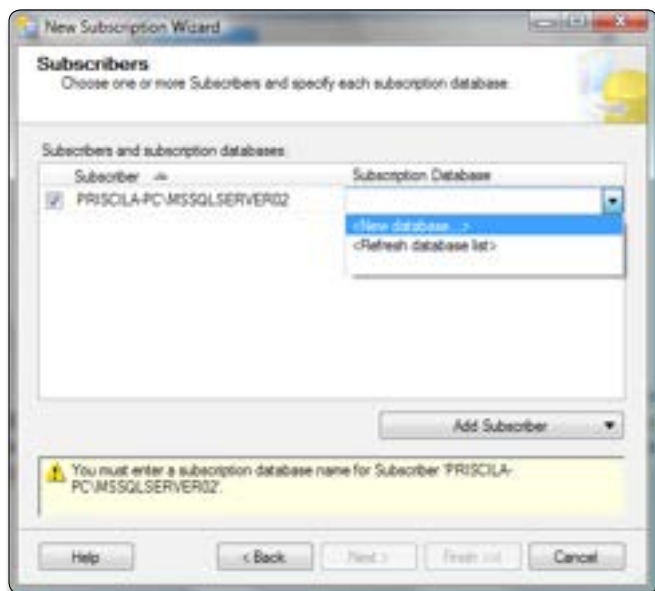


Figura 21. Selecionando o banco de dados *Subscriber*

Voltando à janela da **Figura 21**, clique em *Next*, e na tela que se segue serão solicitadas as informações de segurança do *Agent Distribution*. Clique no botão que aparece [...], informe as informações necessárias, clique em OK e em seguida em *Next*.

Estamos quase finalizando a configuração. Nas **Figuras 22** e **23** são configurados a agenda de sincronização dos dados e o momento de inicialização do *Subscriber*. Deixe as opções padrões em ambas e clique em *Next*.

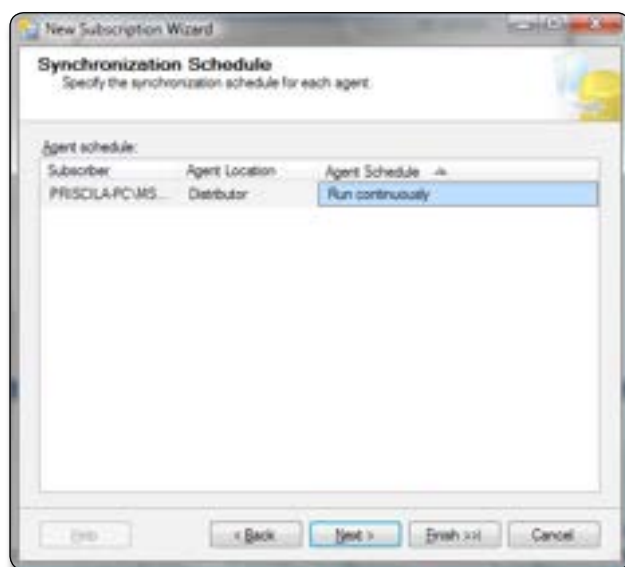


Figura 22. Agenda de Sincronização dos dados

De acordo com as necessidades de negócio poderá ser criada uma agenda personalizada para a sincronização dos dados, onde é possível informar a periodicidade das atualizações (diariamente, semanalmente ou mensalmente). Basta clicar na caixa *Agent Schedule* e selecionar a opção *<Define Schedule...>* para definir uma agenda.

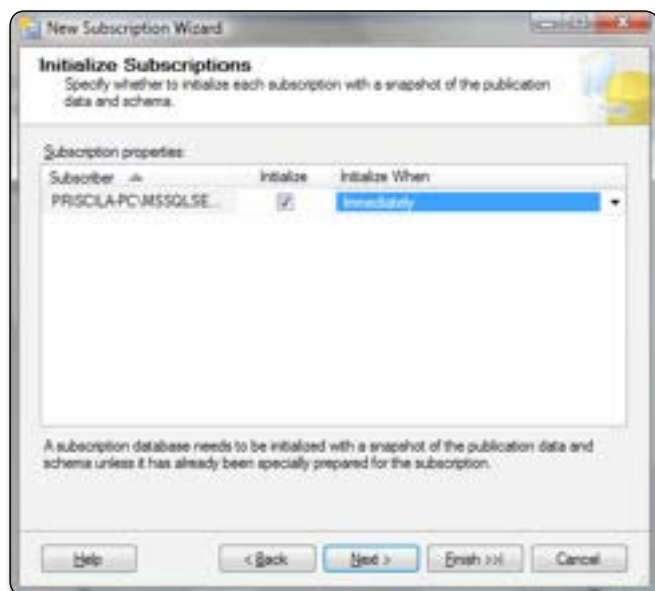


Figura 23. Inicialização do *Subscriber*

A tela que se segue solicitará as ações que o assistente de configuração deverá realizar criar o *Subscriber* e/ou gerar os scripts de criação. Deixe a opção padrão e clique em *Finish*. Será iniciada a criação do *Subscriber* e a sincronização dos dados, conforme a **Figura 24**.

Finalizada a criação dos *Subscriber*, já podemos ver os dados replicados entre as instâncias do SQL Server. Abra uma nova *query* na instância onde está o *Subscriber* e execute a consulta apresentada na **Figura 25**.

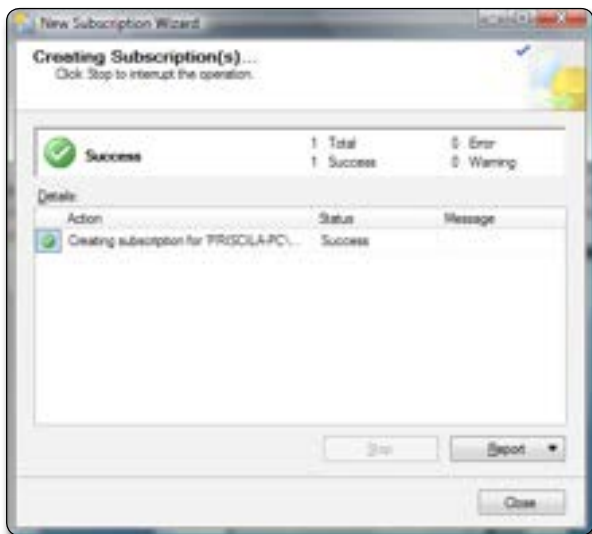


Figura 24. Criando o Subscriber



Figura 25. Consultando os dados replicados

Agora vamos inserir um novo registro no banco *SQLMagazine* e verificar se o registro é replicado para o *SubSQLMagazine*. Para tal, execute o código apresentado na **Listagem 3** em uma nova *query*.

Após a inserção, vamos realizar uma nova consulta na *Subscriber* para ver o registro replicado (Ver **Figura 26**).

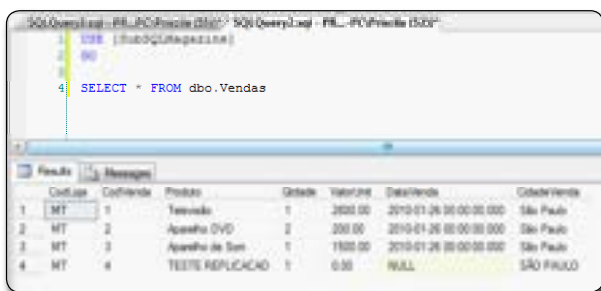


Figura 26. Verificando registro replicado

Listagem 3. Inserindo um novo registro no Publisher

```
USE [SQLMagazine]
GO
```

```
INSERT INTO dbo.Vendas (CodLoja, CodVenda, Produto, Qtde,
ValorUnit, CidadeVenda) VALUES ('MT', 4, 'TESTE REPLICAO', 1,
0, 'SÃO PAULO')
```

Uma valiosa ferramenta para acompanhar o processo de replicação é o *Replication Monitor*. Ele vai informar as atualizações pendentes e se ocorreu algum erro durante o processo. Para acessá-lo, clique com o botão direito do mouse na pasta *replication* do *SQL Server Management Studio* e selecione a opção *Launch Replication Monitor*. Você verá uma tela semelhante à **Figura 27**. Para saber mais detalhes sobre o *Replication Monitor* consulte o Books Online.

Pronto! Os dados já estão sendo replicados entres os bancos de dados do SQL Server. Mas e a replicação dos dados do PostgreSQL?

O SQL Server 2005/2008 não disponibiliza o mesmo assistente de configuração, então teremos que utilizar uma solução diferente para alcançar este objetivo. A solução é bem simples. Criaremos um *Linked Server* que será responsável em gerar a conexão com o PostgreSQL para pegar os dados necessários.

Linked Servers

Um *Linked Server* permite acessar fontes de dados externas, como instâncias remotas do SQL Server, Oracle, Access, PostgreSQL, dentre outros. Para criar um *Linked Server*, é preciso um *OLE DB Provider* que permitirá a comunicação entre as fontes. A chave para um bom desempenho é selecionar um bom *OLE DB Provider*.

Ao ser executada uma consulta, o SQL Server interpreta o comando e então o divide para *OLE DB provider* de destino utilizando solicitações *rowset*. Um *rowset* representa um conjunto de linhas e colunas de dados. Depois de criar os objetos *rowset*, o *OLE DB provider* chama a origem de dados, abre os arquivos necessários e retorna as informações solicitadas como *rowsets*. O SQL Server então formata os *rowsets* como conjuntos de resultados e adiciona quaisquer parâmetros aplicáveis de saída.

É preciso criar um *Linked Server* para cada fonte de dados externa que se deseja acessar e configurar o contexto de segurança. Depois de criada a conexão, é possível utilizar a função *Transact-SQL OPENQUERY* para executar as consultas distribuídas.



Figura 27. Replication Monitor

Para o nosso estudo caso, criaremos um Linked Server utilizando uma fonte de dado ODBC do Windows. Mas antes é necessário que esteja instalado na máquina que fará a conexão o driver do PostgreSQL, PostgreSQL OLE DB provider for Windows que pode ser baixado no site <http://www.postgresql.org/download/products/2>.

Após baixar e instalar, acesse menu Iniciar → Painel de Controle → Ferramentas Administrativas → Fontes de dados ODBC (Figura 28).

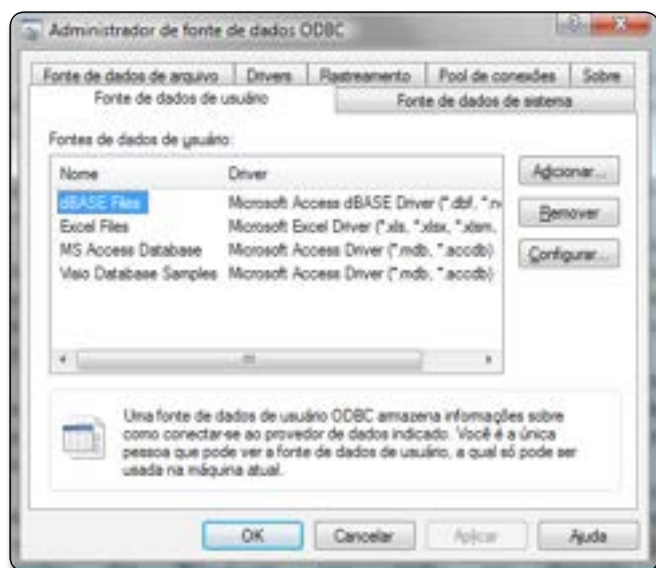


Figura 28. Fontes de Dados ODBC

Vá até a aba *Fontes de Dados de Sistemas* e clique no botão *Adicionar*. Na lista de *drivers* que aparece, selecione *PostgreSQL ANSI* (Figura 29) e clique em concluir.

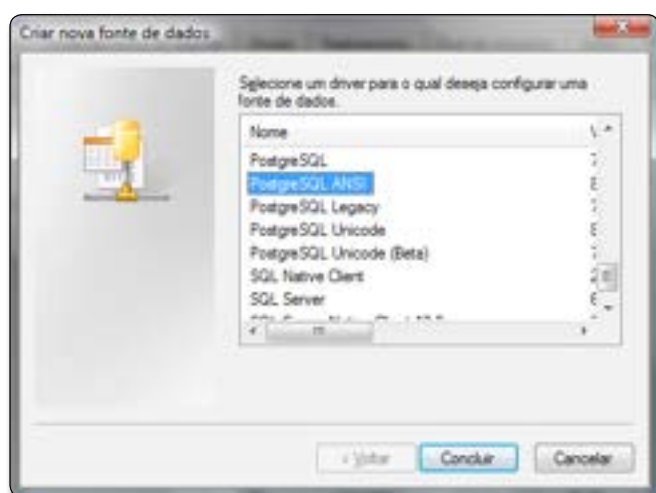


Figura 29. Selecionando o driver PostgreSQL

Em seguida, informe um nome para a conexão, o nome do banco de dados, nome ou IP onde o banco se localiza, a porta, usuário e senha de conexão, conforme pode ser visualizado na Figura 30. Feito isso, teste a conexão e salve.



Figura 30. Configurando a conexão

Agora que criamos a conexão ODBC com o PostgreSQL, poderemos criar o *Linked Server* no SQL Server. Existem duas formas de criar: (1) utilizando a interface gráfica ou (2) através de um script SQL. Vamos criar utilizando um script para que possamos ver os principais parâmetros que devem ser configurados.

Solicite uma nova *query* no *SQL Server Management Studio* na instância que é o *Subscriber* de nossa replicação e digite o código apresentado na Listagem 4.

Listagem 4. Criando um novo Linked Server

```
-- CRIANDO LINKED SERVER
1. EXEC master.dbo.sp_addlinkedserver
2.   @server = N'ConnPostgre',
3.   @srvproduct=N'Microsoft OLE DB Provider for ODBC Driver',
4.   @provider=N'MSDASQL',
5.   @datasrc=N'PGReplicacao',
6.   @location=N'localhost',
7.   @provstr=N'public'
8. GO

-- CONFIGURANDO A SEGURANÇA
9. EXEC master.dbo.sp_addlinkedsrvlogin
10.  @rmtsrvname=N'ConnPostgre',
11.  @useself=N'True',
12.  @locallogin=NULL,
13.  @rmtuser=NULL,
14.  @rmtpassword=NULL
15. GO
```

Os parâmetros informados nas linhas 1 a 7 criam o *Linked Server* e possuem o seguinte significado:

- @Server: é o nome do Linked Server criado;
- @srvproduct: Descrição do OLE DB provider;
- @provider: é o OLE DB provider que corresponde a fonte de dados que se deseja conectar;
- @datasrc: é o nome da fonte de dados interpretada pelo OLE DB Provider;
- @location: local onde está a fonte de dados;
- @provstr: é a sequência de conexão específica do OLE DB provider que identifica uma fonte de dados exclusiva.

Das linhas 9 a 14 é configurado o contexto de segurança do *Linked Server* e os parâmetros significam:

- @rmtsrvname: nome do *Linked Server* que se está configurando a segurança;
- @useself: determina se ao se conectar no *rmtsrvname* será representado por logins locais ou deve-se informar explicitamente um login ou senha;

- @locallogin: é um login no servidor local, o padrão é NULL;
- @rmtuser: nome do login remoto que deverá ser utilizado caso o @useself seja FALSE;
- @rmtpassword: senha do login informado no @rmtuser;

A configuração de um *Linked Server* é bem simples, e ao executar o código da **Listagem 4** o SQL Server criará a conexão com a fonte de dados remota. Você pode visualizar o *Linked Server* criado no SQL Management Studio através da opção *Server Objects* → *Linked Server* (Ver **Figura 31**).

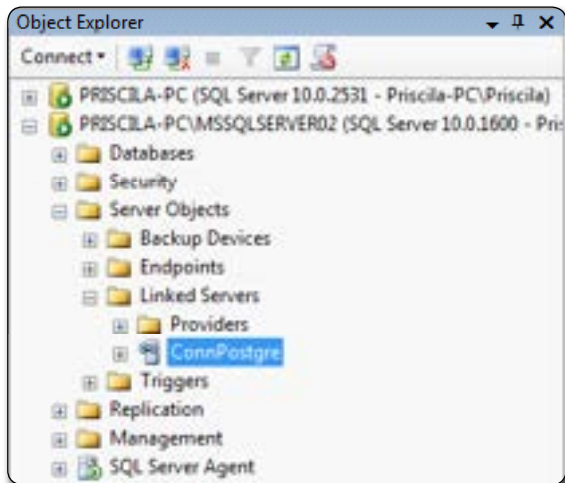


Figura 31. *Linked Server* criado

Agora que já temos a nossa conexão criada, poderemos pegar os dados que desejamos replicar para o SQL Server. Primeiro vamos executar uma consulta para verificar se a conexão ocorreu com sucesso. Para isto, digite a consulta apresentada na **Figura 32** em nova *query*.

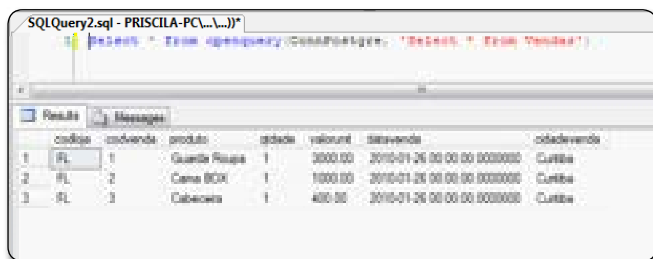


Figura 32. Consultando o PostgreSQL através do *Linked Server*

Pronto! Já podemos pegar os dados do PostgreSQL e enviar para o nosso *Subscriber*. Agora basta criar um script ou uma *stored procedure* para inserir os dados. Veja na **Listagem 5** como é possível realizar esta inserção.

Agora se realizarmos uma nova consulta no banco de dados *SubSQLMagazine*, veremos os dados inseridos que estavam no PostgreSQL (Ver **Figura 33**).

Agora basta automatizar a rotina, e para isto poderia ser usado o SQL Server Agent (Consulte o Books Online para maiores detalhes) ou o Agendador de tarefas do Windows caso esteja utilizando um *Subscriber* na Edição SQL Server Express.

Listagem 5. Inserindo os dados no banco de dados SubSQLMagazine

```
USE [SubSQLMagazine]
GO

INSERT INTO dbo.Vendas (CodLoja, CodVenda, Produto, Qtde,
ValorUnit, DataVenda, CidadeVenda)
SELECT CodLoja, codvenda, produto, qtde, valorunit, datavenda,
cidadevenda
FROM OPENQUERY(ConnPostgre, 'Select * from Vendas')
```

É claro que o nosso exemplo é uma inserção simples sem qualquer tratamento. Em uma situação real você deve se preocupar com a integridade e a confiabilidade das informações replicadas, realizando os tratamentos necessários.

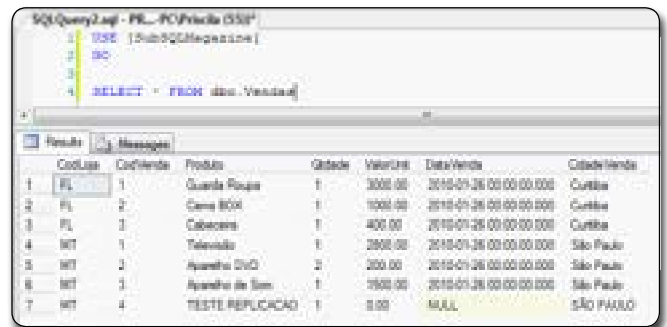


Figura 33. Consultando os dados inseridos

Vimos aqui como criar um *Linked Server* para acessar fontes de dados remotas, e se observarmos na instância onde está nosso *Publisher* veremos que o SQL Server cria dois *Linked Servers* para o processo de replicação dos dados (**Figura 34**).

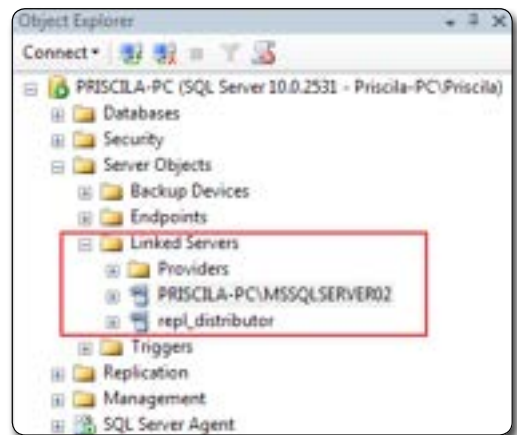


Figura 34. *Linked Servers* criados pelo SQL Server

Conclusão

Hoje as empresas necessitam cada vez mais integrar suas informações que estão distribuídas em diversas fontes de dados ou então distribuí-las para fontes remotas.

A replicação pode ajudá-las a conquistar este objetivo, mas deve ser planejada com muito cuidado e todos os conflitos de dados devem ser resolvidos para que a replicação ocorra com sucesso atendendo todas as necessidades de negócio.

A replicação também pode ser utilizada para fornecer alta disponibilidade para os aplicativos caso ocorra alguma falha que possa deixar o serviço indisponível para o usuário final.

Já nos casos que precisam integrar ou distribuir dados para fontes heterogêneas, poderemos utilizar o *Linked Server*, que como vimos é fácil de utilizar, fornecendo uma maior flexibilidade para se trabalhar. ●

Links e Artigos

Books Online SQL Server 2005

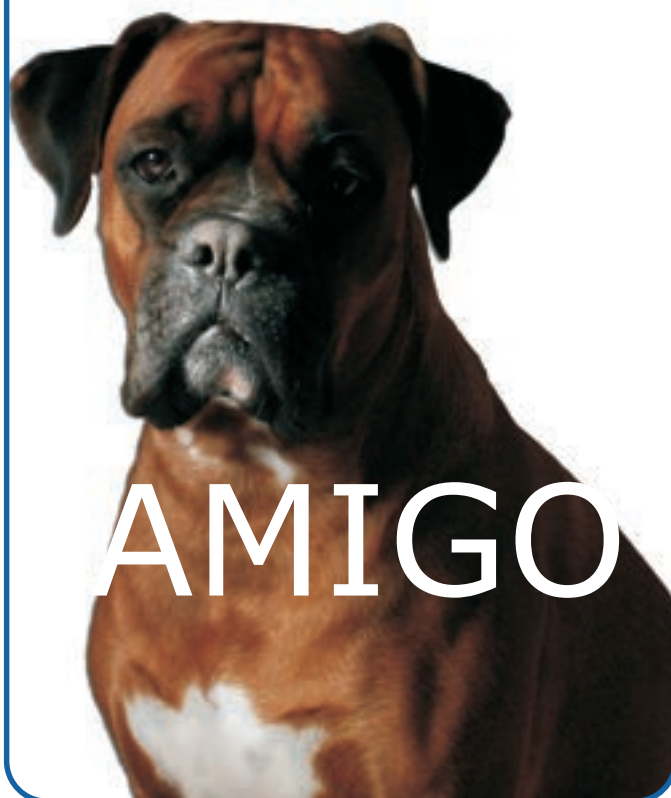
<http://msdn.microsoft.com/en-us/library/ms130214.aspx>

Dê seu feedback sobre esta edição!

A SQL Magazine tem que ser feita ao seu gosto. Para isso, precisamos saber o que você, leitor, acha da revista!

Dê seu voto sobre este artigo, através do link:

www.devmedia.com.br/sqlmagazine/feedback



AMIGO

Existem coisas que não conseguimos ficar sem!

...só pra lembrar,
sua assinatura pode
estar acabando!

Renove Já!

www.devmedia.com.br/renovacao

Para mais informações:
www.devmedia.com.br/central



Nesta seção você encontra artigos sobre banco de dados, SQL ou persistência

Stored Procedures no SQL Server 2008

Criando uma SP completa com tratamento de exceções e de transações

De que se trata o artigo?

Criação de Stored Procedures (SP) com tratamento de exceções e de transações para manter registros em uma base de dados. Além disso, é apresentada uma alternativa para criar uma única SP para inserir, alterar e excluir registros em uma tabela.

Para que serve?

Criação de procedimentos que ficam armazenados no banco de dados, ou seja, guardar os comandos de inserção, alteração e remoção dentro do próprio banco.

Em que situação o tema é útil?

A criação de Stored Procedures pode ser uma alternativa no desenvolvimento de aplicações Desktop e Web, organizando e armazenando o código SQL no banco de dados, facilitando assim a administração e manutenção dos comandos SQL.



Drausio Henrique Chiarotti

drausio.henrique@gmail.com

MCTS em Microsoft .Net Framework. Especialista em Tecnologia da Informação e da Comunicação e Bacharel em Ciência da Computação. Professor do curso de Sistemas de Informação, ministrando aulas de programação de aplicativos na plataforma .Net e Java. Atua também como analista de sistemas pleno utilizando a plataforma .Net em uma grande empresa do ramo farmacêutico no interior de São Paulo.

Um cenário muito comum nos dias de hoje são aplicações que contêm código SQL dentro de suas classes. As aplicações mais modernas e atualizadas centralizam o código SQL na camada de negócios, porém muitas delas ainda possuem código espalhado por toda parte. É errado? Não! Porém, existem inúmeras vantagens em tirar o código SQL das aplicações e guardá-los em Stored Procedures (SP).

A primeira vantagem é a facilidade de manutenção. Se o código SQL está dentro da aplicação e precisa ser alterado (ou contém erros), os programadores

precisam copiar este código para a ferramenta de administração do banco de dados (como o Microsoft SQL Server Management Studio), em seguida tem todo o trabalho de tirar as aspas e os caracteres de concatenação, e finalmente quando o código está limpo eles

manipulam, efetuam as alterações e levam novamente para dentro da aplicação tendo todo o trabalho inverso. Por exemplo, um cenário onde é solicitado que o campo nome da tabela de clientes passe a ser exibido em maiúsculo. Se o código SQL está dentro da aplicação, ela será alterada e terá que ser compilada e publicada novamente.

Agora imagine este mesmo cenário utilizando SP. O DBA entra na SP de consulta e altera o campo nome adicionando a função UPPER(), ou seja, "UPPER(Nome) AS Nome". Pronto, não precisa compilar, não precisa publicar, entre outras tarefas. Basta substituir a SP no cliente.

Uma segunda vantagem é quando um banco de dados precisa ser migrado. Por exemplo, uma empresa vai migrar do SQL Server 2000 para o SQL Server 2008. É mais fácil encontrar softwares que façam a migração das SPs do que softwares que tenham que sair varrendo a aplicação à procura de comandos SQL e verificar se esses comandos são compatíveis com a nova versão. Isso se estes softwares conseguirem encontrar os comandos SQL no meio de tanta concatenação.

Neste artigo será apresentado como manter uma tabela de pessoas no banco de dados SQL Server utilizando SP. Será apresentado como criar uma SP que recebe parâmetros, e com base nestes parâmetros insere, altera e exclui registros na tabela pessoa. Para concluir, também será apresentado como aplicar tratamento de exceções e de transações na SP.

Criando a Base de Dados

Inicie criando a base de dados, também conhecida como catálogo, que é utilizada para agrupar as tabelas e outros recursos pertencentes à mesma aplicação. Crie uma base de dados (de acordo com a **Listagem 1**) com o nome **ArtigoSqlMagazine**, onde iremos definir a tabela pessoa e a SP responsável por manter esta tabela.

Listagem 1. SQL criar Database.

```
CREATE DATABASE ArtigoSqlMagazine
```

Criando a Tabela

Crie uma tabela com o nome **tblPessoa** e adicione os campos IdPessoa, Nome, DataNascimento e Sexo, conforme a **Listagem 2**.

Listagem 2. SQL criar tabela Pessoa.

```
CREATE TABLE tblPessoa (
  IdPessoa INT IDENTITY(1,1) NOT NULL,
  Nome VARCHAR(100) NOT NULL,
  DataNascimento DATETIME NOT NULL,
  Sexo BIT NOT NULL,
  CONSTRAINT PK_tblPessoa PRIMARY KEY (IdPessoa))
```

Note que a tabela é composta por quatro campos que não aceitam valor nulo. A chave primária (*primary key*) é o campo IdPessoa, isto é, o campo IdPessoa é um campo único, obrigatório e que será utilizado para identificar os registros da tabela. O campo IdPessoa não precisa ser informado na

inserção de um novo registro, basta inserir os outros campos que ele será gerado automaticamente, iniciando com valor 1 e incrementando de 1 em 1.

A **Figura 1** exibe a janela **Object Explorer** da ferramenta Microsoft SQL Server Management Studio. Note que foi criada uma base de dados chamada **ArtigoSqlMagazine** e na pasta **Tables** (tabelas) foi gerada a tabela **tblPessoa**. Observe também que existe uma pasta chamada **Programmability** (programação), que possui uma subpasta chamada **Stored Procedures**, onde são criadas as SPs.



Figura 1. Object Explorer

Criando a Stored Procedure

No Microsoft SQL Server Management Studio uma SP pode ser criada clicando com o botão direito na pasta **Stored Procedures** e então selecionando a opção **New Stored Procedure....** No Management Studio é possível utilizar o modo gráfico para montar a estrutura e alterar os parâmetros. Para isso, basta acessar o menu **Query** e a opção **Specify Values For Template Parameters**. No modo gráfico é possível alterar o autor, data, descrição, nome da SP, tipo e valor padrão dos parâmetros.

Neste artigo deixaremos de lado o modo gráfico e vamos criar passo a passo a SP. Dessa forma, clique com o botão direito sobre a base de dados **ArtigoSqlMagazine** e escolha a opção **New Query** para abrir uma consulta em branco onde vamos definir nossa SP. Sempre que uma nova consulta for aberta, é importante verificar na barra SQL Editor (*view>Toolbars>SQL Editor*) qual base de dados será afetada, ou seja, em qual base de dados nossa SP será criada. A **Figura 2** mostra que a base de dados selecionada é **ArtigoSqlMagazine**.

Sempre que uma nova SP é criada, é importante adicionar um cabeçalho informando o autor, a data de criação e uma breve descrição, principalmente quando mais de uma pessoa tem permissão para criar SPs no banco. Se futuramente a SP precisar ser alterada, é interessante que o DBA/Desenvolvedor que melhor a conhece a altere.



Figura 2. SQL Editor

A **Listagem 3** apresenta a sintaxe para criar uma SP. Note que logo no início do script foi adicionado um cabeçalho no formato de comentário. Este cabeçalho informa que a SP foi criada por Drausio em 04/08/2010 e a função dela é manter a tabela pessoa.

Listagem 3. Sintaxe para criar Stored Procedure.

```
-- =====
-- Author:      Drausio Henrique Chiarotti
-- Create date: 04/08/2010
-- Description: Manter tabela tblPessoa
-- =====
CREATE PROCEDURE nome_stored_procedure
--Parâmetros
AS
BEGIN
--Instruções SQL
END
GO
```

O script para criar uma nova SP é composto pela instrução `CREATE PROCEDURE` seguido do nome que será dado para a SP. Uma SP pode conter um ou mais parâmetros que podem ser declarados na instrução `CREATE PROCEDURE`. A **Listagem 3** exibe o local onde os parâmetros devem ser declarados. As palavras `BEGIN` e `END` demarcam o corpo da SP. É após o `BEGIN` e antes do `END` que as instruções devem ser adicionadas.

Vamos criar uma SP chamada **uspCadastrarPessoa**. Para padronizar e identificar a SP serão utilizadas três letras no início do nome. As letras significam *User Stored Procedure* e ajudam o DBA e os programadores a identificar a SP dentro do banco de dados ou de uma aplicação. Por exemplo, se um programador vê uma palavra iniciada com *usp* na camada de negócios ele sabe que trata-se de uma SP. Da mesma forma, se o DBA utilizar *uvw* para Visões (*Views*) e *tbl* para tabelas (*Tables*), em qualquer parte do banco de dados ou de uma aplicação (.Net ou Java, por exemplo) tanto o programador quanto o DBA saberão identificar a palavra como sendo SP, visões ou tabelas.

Criando Parâmetros

A tabela de pessoa é composta pelos campos `IdPessoa`, `Nome`, `DataNascimento` e `Sexo`. A SP deve conter quatro parâmetros para receber estes dados e um quinto parâmetro que será utilizado quando for requisitada a exclusão de dados. Todo parâmetro deve ser iniciado utilizando arroba (@) como caractere inicial.

O próximo passo é atribuir valores padrão (default) para estes parâmetros. No nosso exemplo atribuiremos o valor falso (0) para o parâmetro que controla a exclusão, e para os demais parâmetros o valor nulo (NULL), como apresentado na **Listagem 4**.

Os valores são atribuídos para que a SP possa ser executada sem ter que passar todos os parâmetros, por exemplo, se for efetuar uma exclusão basta informar o parâmetro “@Excluir = 1” e o parâmetro “@IdPessoa = 1”, os outros parâmetros

recebem valor nulo e a SP é executada normalmente. Se não forem atribuídos valores padrão para os parâmetros na criação da SP, eles tornam-se obrigatórios, e ao executar a SP será exibida uma mensagem de erro que os parâmetros são esperados e não foram informados.

Listagem 4. Declarando parâmetros.

```
CREATE PROCEDURE uspCadastrarPessoa
@Excluir BIT = 0,
@IdPessoa INT = NULL,
@Nome VARCHAR(100) = NULL,
@DataNascimento DATETIME = NULL,
@Sexo BIT = NULL
AS
BEGIN
--Instruções SQL
END
GO
```

Criando o Tratamento de Exceções

Implementar tratamento de exceções no SQL Server é muito parecido com a linguagem C#. São utilizados os blocos `TRY/CATCH`. A instrução `BEGIN TRY` marca o início do tratamento, e a instrução `END TRY` marca o fim do tratamento da exceção. Por sua vez, a palavra `BEGIN CATCH` inicia o tratamento da captura do erro e a palavra `END CATCH` finaliza o tratamento.

As instruções que estão dentro do bloco `TRY` serão executadas e caso ocorram erros, eles serão capturados e tratados pelo bloco `CATCH` (**Listagem 5**). Note que no bloco `CATCH` foi utilizada a instrução `ERROR_MESSAGE()`, que retorna um texto contendo o erro gerado no bloco `TRY`. O `SELECT` (apelidado de retorno) é utilizado para que a mensagem de erro seja retornada pela SP.

Criando o Tratamento de Transações

O SQL Server permite aplicar tratamento de transações. Imagine que você tem um procedimento que retira dinheiro de uma conta e adiciona em outra. Ao ser executado, o procedimento retira o dinheiro da conta de origem e, antes de adicionar o valor na conta destino, um erro é gerado. Um segundo cenário é adicionar um item em uma nota fiscal. Para isso foi criado um procedimento que salva o item, gera os cálculos dos impostos, salva os cálculos e, por fim, atualiza o total da nota. Você executa a SP que salva o item e no meio dos cálculos ocorre um erro. O item foi inserido, mas o resto do procedimento não foi executado. Para solucionar cenários como estes, é utilizado o controle de transação.

Para controlar transações são utilizadas três instruções: `BEGIN TRAN`, `COMMIT TRAN` e `ROLLBACK TRAN`. A instrução `BEGIN TRAN` inicia a transação, `COMMIT TRAN` finaliza a transação e `ROLLBACK TRAN` desfaz todas as modificações efetuadas caso ocorra algum erro. Note, na **Listagem 5**, que as instruções `BEGIN TRAN` e `COMMIT TRAN` foram inseridas dentro do bloco `BEGIN TRY`, e a instrução `ROLLBACK TRAN`

foi inserida dentro do bloco BEGIN CATCH. Caso ocorra um erro, ele será capturado pelo bloco BEGIN CATCH, e antes que o erro seja retornado e a SP finalizada, serão desfeitas todas as modificações efetuadas até o ponto onde ocorreu o erro. Em outras palavras, se você executa a SP que cadastra o item da nota fiscal, efetua os cálculos, e quando vai atualizar o total um erro é gerado, o comando ROLLBACK TRAN desfaz todas as ações executadas anteriormente (**Listagem 5**).

Listagem 5. Tratamento de exceções e de transações.

```
CREATE PROCEDURE uspCadastrarPessoa
@Excluir BIT = 0,
@IdPessoa INT = NULL,
@Nome VARCHAR(100) = NULL,
@DataNascimento DATETIME = NULL,
@Sexo BIT = NULL
AS
BEGIN
BEGIN TRY
BEGIN TRAN
--Instruções
COMMIT TRAN
END TRY
BEGIN CATCH
ROLLBACK TRAN
SELECT ERROR_MESSAGE() AS Retorno
END CATCH
END
GO
```

Criando as Instruções de Inserção, Alteração e Exclusão

Dentro do controle de transações, o primeiro passo é utilizar o parâmetro @Excluir para verificar se será executada a exclusão de um registro na tabela. Para efetuar esta verificação foi utilizado o comando IF, como exige a **Listagem 6**. Observe que se o parâmetro @Excluir for igual a 1 (verdadeiro), será executada a instrução para excluir o registro da tabela tblPessoa onde o campo IdPessoa for igual ao valor do parâmetro @IdPessoa. Após a instrução de exclusão existe um SELECT que retorna o id da pessoa excluída. Note ainda, que foram utilizadas as instruções BEGIN e END para delimitar o corpo do comando IF.

Se o parâmetro @Excluir não for igual a 1 (verdadeiro), será executado a inserção ou alteração. A chave primária da tabela pessoa é o campo IdPessoa, isto quer dizer que, se o parâmetro @IdPessoa for nulo significa que uma inserção está sendo solicitada. Caso contrário, significa que os dados de uma pessoa devem ser alterados.

Para verificar se o parâmetro @IdPessoa é nulo foi utilizado o comando IF. Se a expressão for verdadeira será inserida uma nova pessoa utilizando os valores dos parâmetros @Nome, @DataNascimento e @Sexo. Note que após a instrução de inserção existe um SELECT que retorna o IdPessoa (auto-incremento) gerado para a pessoa inserida em tblPessoa. A função @@IDENTITY retorna o último id inserido na tabela.

Se não for exclusão nem inserção, resta a instrução de alteração. Neste contexto todos os parâmetros serão utilizados. Note que após a instrução de alteração existe um SELECT que retorna o id da pessoa que teve os dados alterados.

É importante salientar que todas as partes da SP retornam um valor. Na inserção, na alteração e na exclusão será retornado o id da pessoa, e caso ocorra algum erro será retornada uma mensagem de erro.

Listagem 6. Instruções de Inserção, Alteração e Exclusão.

```
-- =====
-- Author: Drausio Henrique Chiarotti
-- Create date: 04/08/2010
-- Description: Manter pessoa
-- =====

CREATE PROCEDURE uspCadastrarPessoa
@Excluir BIT = 0,
@IdPessoa INT = NULL,
@Nome VARCHAR(100) = NULL,
@DataNascimento DATETIME = NULL,
@Sexo BIT = NULL
AS
BEGIN
BEGIN TRY
BEGIN TRAN

IF(@Excluir = 1)
BEGIN
DELETE FROM tblPessoa
WHERE IdPessoa = @IdPessoa

SELECT @IdPessoa AS Retorno
END
ELSE
IF(@IdPessoa IS NULL)
BEGIN
INSERT INTO tblPessoa (Nome, DataNascimento, Sexo)
VALUES (@Nome, @DataNascimento, @Sexo)

SELECT @@IDENTITY AS Retorno
END
ELSE
BEGIN
UPDATE
tblPessoa
SET
Nome = @Nome,
DataNascimento = @DataNascimento,
Sexo = @Sexo
WHERE
IdPessoa = @IdPessoa

SELECT @IdPessoa AS Retorno
END

COMMIT TRAN
END TRY
BEGIN CATCH
ROLLBACK TRAN
SELECT ERROR_MESSAGE() AS Retorno
END CATCH
END
```

Este é um modelo onde o programador que utiliza a SP pode criar regras de negócios que todos os métodos retornam o tipo string (texto). Na camada de apresentação, onde forem utilizadas estas regras de negócios, o programador recebe uma string e tenta converter a string para um inteiro. Caso converta com sucesso, quer dizer que deu tudo certo e que a SP retornou um número inteiro, que neste contexto será IdPessoa. Caso não seja possível converter o retorno para um inteiro, é uma mensagem de erro. Neste caso o programador pode exibir a mensagem na tela informando que ocorreu um erro. Uma segunda sugestão é na própria camada de negócios, o programador faz a conversão, e caso não seja possível converter para inteiro, o programador salva a mensagem de erro em um log e retorna uma mensagem amigável para o usuário.

Executando a Stored Procedure

Para testar os procedimentos criados, insira, altere e exclua registros na tabela de pessoa. Para isso, clique com o botão direito sobre a base de dados **ArtigoSqlMagazine** e escolha a opção *New Query* para abrir uma consulta em branco.

Vamos executar a SP e passar os parâmetros para inserir uma nova pessoa. Note na **Listagem 7** que o primeiro parâmetro

é o valor 0 (falso), informando que não é uma exclusão. O segundo parâmetro informa o IdPessoa, caso seja alteração. Neste momento estamos inserindo uma nova pessoa. Por este motivo deve ser passado nulo (null). O terceiro parâmetro é o nome da pessoa que será inserido, por exemplo Drausio H. Chiarotti. O quarto parâmetro é a data de nascimento, e o quinto e ultimo parâmetro é o sexo, onde foi informado 1, que significa masculino.

Listagem 7. Executar inserção.

```
exec uspCadastrarPessoa 0, null, 'Drausio H. Chiarotti',
'2010-01-01', 1
```

A **Figura 3** exibe a janela de resultados (CTRL+R) com o valor retornado pela SP. Neste caso foi inserida uma nova pessoa que recebeu o código 1. Se ocorrer algum erro, a janela de resultados exibe a mensagem de erro.

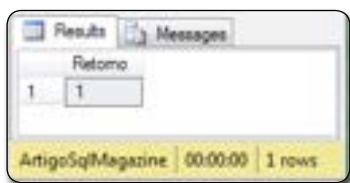


Figura 3. SQL Editor

Agora execute outro procedimento, passando parâmetros para atualizar os dados da pessoa inserida na **Listagem 7**. A **Listagem 8** mostra este comando. Note que foi alterado o segundo parâmetro, informando o código da pessoa que será alterada (id igual a 1), e também a data de nascimento, que deve ser alterada de 01/01/2010 para 02/02/2010.

Listagem 8. Executar alteração.

```
exec uspCadastrarPessoa 0, 1, 'Drausio H. Chiarotti', '2010-02-02', 1
```

Novamente o retorno será exibido na janela de resultados. Neste caso, o código 1, informando que os dados foram alterados com sucesso.

Para finalizar, vamos executar a SP e passar os parâmetros para excluir uma pessoa (**Listagem 9**).

Listagem 9. Executar exclusão.

```
exec uspCadastrarPessoa 1, 1
```

Para excluir uma pessoa é passado o primeiro parâmetro com valor 1 (utilizado para exclusão), e o segundo parâmetro, informando o código da pessoa que será excluída, que neste exemplo será a pessoa com código igual a 1.

Alterando a Stored Procedure

É comum modificar a estrutura de tabelas no banco de dados adicionando e removendo campos. Estas ações refletem

diretamente nas aplicações, fazendo com que os desenvolvedores tenham que alterar seus projetos. Com a utilização de SP, a parte de persistência de dados torna-se mais simples, pois o DBA tem o trabalho de abrir uma ou duas SPs e fazer alguns ajustes. Como exemplo, vamos alterar a tabela tblPessoa adicionando os campos altura e peso (**Figura 4**).

Column Name	Data Type	Allow Nulls
IdPessoa	int	☐
Nome	varchar(100)	☐
DataNascimento	datetime	☐
Sexo	bit	☐
Altura	decimal(3, 2)	☐
Peso	int	☐

Figura 4. Tabela tblPessoa modo designer

Uma forma rápida e prática de adicionar novos campos em uma tabela no SQL Server 2008 é utilizar o modo design. Clique com o botão direito sobre a tabela tblPessoa e selecione a opção Design (ver **Figura 5**). Será exibida a janela com os campos que compõem a tabela selecionada. Para adicionar um novo item, selecione a última linha (que está em branco) e informe o nome da coluna (Column Name), o tipo de dado (Data Type) e se o item permite valor nulo (Allow Nulls). Por padrão, o SQL Server 2008 não permite que as tabelas sejam alteradas, porém, basta acessar o menu *Tools>Options...*, na janela *Options* selecionar *Designers/Table and Database Designers* e desmarcar a opção *Prevent saving changes that require table re-creation*. Feito isso o SQL Server 2008 permite que as tabelas sejam alteradas via designer.

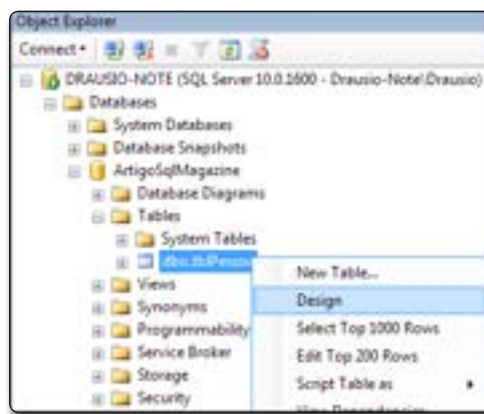


Figura 5. Modificar Tabela

Para modificar a SP e adicionar os novos campos, clique com o botão direito sobre a SP **uspCadastrarPessoa** e selecione a opção *Modify*, como apresenta a **Figura 6**. É importante notar que foi aberto o script implementado para criar a SP com a diferença que a palavra CREATE foi alterada para ALTER. Quando criamos a SP utilizamos *CREATE PROCEDURE uspCadastrarPessoa*, e ao selecionar a opção *Modify*, o script gerado altera o comando para *ALTER PROCEDURE uspCadastrarPessoa*.

O trabalho do programador é adicionar os parâmetros @Peso e @Altura na SP e alterar os comandos INSERT e UPDATE incluindo os novos campos, conforme **Listagem 10**.

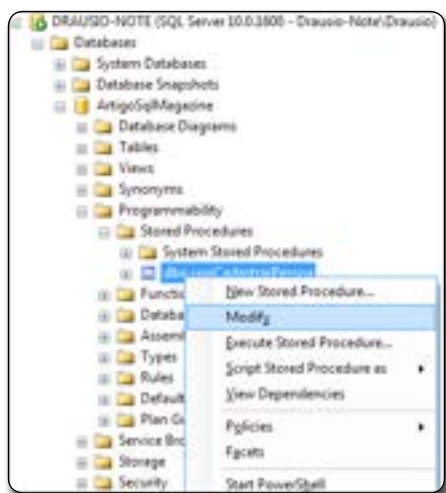


Figura 6. Modificar Stored Procedure

Para verificar se a SP foi alterada com sucesso, faça um teste executando o código da **Listagem 11**.

O DBA responsável irá alterar a tabela tblPessoa e com os dados em mente, fará as alterações nas SPs. Em um contexto onde os comandos SQL estejam na aplicação, o DBA teria todo o trabalho de informar a tabela, os campos e os tipos de campos para que os desenvolvedores façam as devidas alterações.

Conclusão

Neste artigo foi apresentado como criar e alterar Stored Procedures (SP) com tratamento de exceções e de transações. O artigo apresenta uma alternativa que armazena o código SQL em uma SP que tem a função de manter a tabela de pessoas. A vantagem em utilizar este modelo é separar o código SQL do código da aplicação (C#, VB.Net, Java, etc.). Com um pouco de lógica e complexidade é possível criar uma única SP para inserir, alterar e excluir registros na tabela. Para procedimentos que executam um conjunto de ações, é possível utilizar o tratamento de transações para garantir atomicidade e consistência dos dados.

Para os iniciantes na área de desenvolvimento, a dica é: utilizem SP. Para os desenvolvedores que já possuem projetos com todo o código SQL dentro da aplicação, façam um teste e criem algumas SPs para confirmar os benefícios. ●

Listagem 10. Alterando a Stored Procedure.

```
--
-- Author:      Drausio Henrique Chiarotti
-- Create date: 04/08/2010
-- Description: Manter pessoa
--
ALTER PROCEDURE [dbo].[uspCadastrarPessoa]
@Excluir BIT = 0,
@IdPessoa INT = NULL,
@Nome VARCHAR(100) = NULL,
@DataNascimento DATETIME = NULL,
@Sexo BIT = NULL,
@Altura DECIMAL(3,2) = NULL,
@Peso INT = NULL
AS
BEGIN
    BEGIN TRY
        BEGIN TRAN

            IF (@Excluir = 1)
            BEGIN
                DELETE FROM tblPessoa
                WHERE IdPessoa = @IdPessoa

                SELECT @IdPessoa AS Retorno
            END
            ELSE
            BEGIN
                IF (@IdPessoa IS NULL)
                BEGIN
                    INSERT INTO tblPessoa (Nome, DataNascimento, Sexo,
                    Altura, Peso)
                    VALUES (@Nome, @DataNascimento, @Sexo,
                    @Altura, @Peso)

                    SELECT @@IDENTITY AS Retorno
                END
                ELSE
                BEGIN
                    UPDATE
                        tblPessoa
                    SET
                        Nome = @Nome,
                        DataNascimento = @DataNascimento,
                        Sexo = @Sexo,
                        Altura = @Altura,
                        Peso = @Peso
                    WHERE
                        IdPessoa = @IdPessoa

                    SELECT @IdPessoa AS Retorno
                END
            END
        COMMIT TRAN
    END TRY
    BEGIN CATCH
        ROLLBACK TRAN
        SELECT ERROR_MESSAGE() AS Retorno
    END CATCH
END
```

Listagem 11. Executar inserção utilizando SP alterada.

```
exec uspCadastrarPessoa 0, null, 'Drausio H. Chiarotti',
'2010-01-01', 1, 1.91, 86
```

Dê seu feedback sobre esta edição!

A SQL Magazine tem que ser feita ao seu gosto. Para isso, precisamos saber o que você, leitor, acha da revista!

Dê seu voto sobre este artigo, através do link:

www.devmedia.com.br/sqlmagazine/feedback



Nesta seção você encontra artigos sobre banco de dados, SQL ou persistência



Utilizando Stored Procedures, Views e Triggers no MySQL

Aplicando estes conceitos em situações práticas

De que se trata o artigo?

Este artigo descreve a continuidade do projeto de um banco de dados para um sistema de locação de carros, onde são usados recursos de stored procedures, views e triggers no MySQL para simplificar as operações a serem realizadas por este sistema.

Para que serve?

Auxiliaria projetistas de banco de dados que possuem a necessidade de implementar bancos de dados, partindo de seus modelos físicos até a construção do banco de dados e seus objetos de apoio, como stored procedures, views e triggers.

Em que situação o tema é útil?

Stored procedures, views e triggers são recursos importantes a serem aplicados na implementação de um banco de dados, pois simplificam diversas operações a serem realizadas pelo banco de dados, assim como torna mais simples a codificação do sistema, definindo uma camada intermediária de controle entre o banco de dados físico e o código fonte da aplicação.

Os SGBDs atuais oferecem diversos recursos que facilitam as tarefas de manipulação de dados e, conseqüentemente, o gerenciamento de um banco de dados. Dentre os principais recursos, que não são tão inovadores, mas que possuem uma importância gigantesca no trabalho de um DBA podemos citar as *stored procedures*, *views* e *triggers*.

Neste artigo, iremos dar continuidade ao desenvolvimento de uma aplicação responsável pelo gerenciamento de uma locadora de carros apresentada no artigo “Modelando um Sistema de Reserva de Carros”, publicado também nesta edição da SQL Magazine. Até o momento já possuímos suas tabelas e relacionamentos, nada mais que isso. Na continuidade deste projeto, iremos construir *Stored Procedures* (SPs), *Views*

e *Triggers* que nos apoiem na realização das diversas operações realizadas por esta empresa.



Arilo Claudio Dias Neto

ariloclaudio@gmail.com

É Doutor em Engenharia de Sistemas e Computação formado pela Universidade Federal do Rio de Janeiro (COPPE). Possui 6 anos de experiência em análise e desenvolvimento de software. É ainda editor técnico da Revista SQL Magazine, gerenciada pelo Grupo DevMedia.

Stored Procedures

Uma *stored procedure*, ou rotina, é simplesmente um repositório (container) para um conjunto de declarações SQL. *Stored procedures* podem conter declarações condicionais e processamento lógico que normalmente estaria indisponível em uma declaração SQL tradicional gerada dinamicamente.

Por exemplo, uma *stored procedure* pode contar comandos de loop ou condicionais, como *do / while*, *if / then / else*, dentre outros, permitindo a realização de operações mais complexas. Ou seja, uma SP funciona como um procedimento sobre o banco de dados, de forma similar aos procedimentos que usamos durante a codificação usando linguagens procedurais, tais como C ou Pascal.

Possuir códigos SQL em forma de *stored procedures* possibilita à aplicação minimizar a quantidade de código SQL que aparece no código fonte da aplicação e coloca estes códigos sob o controle da camada do banco de dados. Isso faz com que o projeto da aplicação fique mais claro e deixa as páginas de código fonte da aplicação livres de códigos SQL dinâmicos, que seriam desnecessários. Com isso, caso haja a necessidade de mudar alguma instrução em SQL, basta alterarmos o código da *stored procedure*, sem haver a necessidade de alterar o código fonte da aplicação e ter que recompilá-la.

No MySQL, *stored procedures* são rotinas criadas com as instruções `CREATE PROCEDURE`. Sua chamada ocorre através da instrução `CALL` e só pode passar valores de retorno usando variáveis de saída.

Atualmente o MySQL só preserva o contexto para o banco de dados padrão. Uma rotina herda o banco de dados padrão de quem a chama, assim geralmente as rotinas devem utilizar uma instrução `USE nome_bancodebdados`, ou então devemos especificar todas as tabelas com uma referência de banco de dados explícita, ex: `nome_bancodebdados.nome_tabela`.

O MySQL suporta uma extensão muito útil que permite o uso da instrução regular `SELECT` dentro de uma *stored procedure*. O resultado de tal consulta é simplesmente enviado diretamente para o cliente.

A sintaxe para criação de uma SP está apresentada na **Listagem 1**.

A lista de parâmetros entre parênteses deve estar sempre presente. Se não houver parâmetros, uma lista de parâmetros vazia de “()” deve ser usada. Cada parâmetro é um parâmetro IN por padrão. Para especificar outro tipo de parâmetro, use a palavra chave OUT ou INOUT antes do nome do parâmetro.

Além do `CREATE PROCEDURE`, existem comandos para alterar (`ALTER PROCEDURE`), excluir (`DROP PROCEDURE`) e visualizar o código de criação (`SHOW CREATE TABLE`) de uma SP. Ao longo do artigo usaremos este comando em exemplos práticos associados ao nosso estudo de caso.

Views

Uma *view* consiste em uma consulta armazenada acessível como uma tabela virtual composta pelo resultado de um conjunto de consultas (*queries*). Ao contrário de uma tabela ordinária (tabelas bases) em um banco de dados, uma *view*

não faz parte do esquema físico: ela é uma tabela dinâmica e virtual computada ou colada a partir de dados existentes no banco de dados. Ao alterar os dados em uma tabela, os dados exibidos por uma *view* serão alterados automaticamente.

Listagem 1. Sintaxe para criação de uma SP no MySQL

```
CREATE PROCEDURE nome_sp ([parametro[...]])
[característica ...] corpo_rotina

parameter:
[ IN | OUT | INOUT ] nome_parametro tipo

tipo:
Qualquer tipo de dados válidos no MySQL

característica:
LANGUAGE SQL
| [NOT] DETERMINISTIC
| SQL SECURITY (DEFINER | INVOKER)
| COMMENT string

corpo_rotina:
Declaração(ões) de procedimento em SQL válida
```

Views podem prover diversas vantagens ao ser comparadas com tabelas:

- *Views* podem representar um subconjunto dos dados contidos em uma tabela.
- *Views* podem realizar junções e simplificar múltiplas tabelas em uma simples tabela virtual.
- *Views* podem atuar como tabelas agregadas, onde o banco de dados agrega dados (somas, média, etc) e apresenta os resultados calculados como parte dos dados.
- *Views* podem ocultar a complexidade dos dados; por exemplo, uma *view* pode ser exibida como `Locações2000` ou `Locações2001`, particionando de forma transparente a tabela atual.
- *Views* usam muito pouco espaço para serem armazenadas; o banco de dados contém apenas a definição de uma *view*, não uma cópia de todos os dados que ela apresenta.
- Dependendo do SGBD adotado, *views* podem prover uma segurança extra;
- *Views* podem limitar o nível de exposição de uma tabela para o “mundo externo” à aplicação.

O MySQL suporta *views* desde a sua versão 5.x e quase todas as funcionalidades das *views* de acordo com o padrão SQL 2003. As consultas de *views* no MySQL são processadas de duas formas:

- O MySQL cria uma tabela temporária baseada na consulta que define a *view* e então executa a consulta de entrada nesta tabela.
- O MySQL primeiro combina as consultas e parâmetros de entrada que definem uma *view*, então o MySQL executa esta consulta.

O MySQL permite ainda criar *views* de *views*. Na declaração `SELECT` da definição de uma *view*, podemos referenciar outras *views*.

A instrução `CREATE VIEW` cria uma nova *view*. Sua sintaxe está apresentada na **Listagem 2**. Para alterar a definição de uma *view* ou excluir uma *view*, use, respectivamente, `ALTER VIEW` ou `DROP VIEW`.

Listagem 2. Sintaxe para criação de uma View no MySQL

```
CREATE
[OU REPLACE]
[ALGORITMO = { UNDEFINED | MERGE | TEMPTABLE }]
[DEFINIDOR = { user | CURRENT_USER }]
[SEGURANÇA SQL { DEFINER | INVOKER }]
VIEW nome_view [(lista_colunas)]
AS declaração_select
[WITH [CASCADED | LOCAL] CHECK OPTION]
```

Uma *view* pode ser criada a partir de vários tipos de declarações de SELECT. Ela pode se referenciar a outras tabelas ou *views*, ela pode usar JOINS, UNION e subconsultas. O SELECT não precisa se referir a qualquer tabela. O exemplo apresentado na **Listagem 3** define uma *view* que seleciona duas colunas (quantidade e preço) a partir de uma tabela de *Vendas*, assim como o resultado da multiplicação entre essas colunas.

Listagem 3. Exemplo de View no MySQL

```
mysql> CREATE TABLE Vendas (qtde INT, preco INT);
mysql> INSERT INTO Vendas VALUES(3, 50), (5, 60);
mysql> CREATE VIEW v AS SELECT qtde, preco, qtde*preco AS valor
FROM Vendas;
mysql> SELECT * FROM v;
+----+-----+-----+
| qtde | preco | valor |
+----+-----+-----+
| 3    | 50    | 150   |
| 5    | 60    | 300   |
+----+-----+-----+
mysql> SELECT * FROM v WHERE qtde = 5;
+----+-----+-----+
| qtde | preco | valor |
+----+-----+-----+
| 5    | 60    | 300   |
+----+-----+-----+
```

Triggers

Uma *trigger* consiste em um código procedural que é executado automaticamente em resposta a certos eventos em uma tabela ou *view* particular em um banco de dados. *Trigger* é comumente usada para manter a integridade das informações em um banco de dados. Por exemplo, quando um novo registro (representando um novo funcionário) é adicionado à tabela de empregados, novos registros devem ser criados também nas tabelas de taxas, férias e salários.

Triggers são comumente usadas para:

- Prevenir mudanças a ocorrerem em um banco de dados;
- Registrar as mudanças em um arquivo de log (ex: manter uma cópia dos dados antigos);
- Auditar mudanças (ex: manter um log dos usuários e papéis envolvidos em mudanças);
- Garantir regras de negócio (ex: requerer que toda compra possua ao menos 1 item);
- Executar regras de negócio (ex: notificar um gerente sempre que o número da conta corrente de um cliente do banco for alterado);
- Replicar dados (ex: armazenar um registro de todas as mudanças para que ele seja persistido em outro banco de dados posteriormente);
- Garantir desempenho (ex: atualizar o saldo de uma conta corrente após uma transação, para consultas rápidas ao saldo).

O apoio a *triggers* foi incluído no MySQL a partir de sua versão 5.0.2. Uma *trigger* é um objeto de banco de dados associado a uma tabela e que é ativado quando um evento particular ocorre na tabela em questão. Alguns usos para *triggers* são feitos para verificar valores inseridos em uma tabela ou para realizar cálculos sobre valores envolvidos em uma atualização.

Uma *trigger* é definida para ser ativada quando uma instrução de INSERT, DELETE, ou UPDATE executa na tabela associada. Uma *trigger* pode ser configurada para ser ativada antes ou após a instrução. Por exemplo, podemos ter uma *trigger* que será ativada antes que cada linha de uma tabela seja inserida ou após a atualização de uma linha da tabela.

Importante: Triggers no MySQL são ativadas apenas por instruções SQL. Elas não são ativadas por mudanças em tabelas realizadas por APIs que não transmitem instruções SQL ao servidor MySQL.

Para criar ou excluir uma *trigger*, use, respectivamente, a instrução CREATE TRIGGER ou DROP TRIGGER. A sintaxe da instrução de criação está apresentada na **Listagem 4**.

Listagem 4. Sintaxe para criação de uma Trigger no MySQL

```
CREATE
[DEFINER = { user | CURRENT_USER }]
TRIGGER nome_trigger tempo_trigger evento_trigger
ON nome_tabela FOR EACH ROW declaração_trigger
```

Esta declaração cria uma nova *trigger*. Esta *trigger* estaria associada à tabela *nome_tabela*, que deve se referir a uma tabela permanente.

Vejam na **Listagem 5** um exemplo simples que associa uma *trigger* a uma tabela para instruções INSERT. A *trigger* atua como um acumulador, somando os valores inseridos em uma coluna da tabela.

Listagem 5. Exemplo de uma Trigger no MySQL

```
mysql> CREATE TABLE conta (numero_conta INT, quantidade
DECIMAL(10,2));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TRIGGER inserir_soma BEFORE INSERT ON conta
-> FOR EACH ROW SET @soma = @soma + NEW.quantidade;
Query OK, 0 rows affected (0.06 sec)
```

A instrução CREATE TRIGGER cria uma *trigger* chamada *inserir_soma* que está associada à tabela *conta*. Ela também inclui uma cláusula que especifica o tempo de ativação da *trigger*, o evento de disparo e o que ela deve fazer ao ser ativada (somar o valor inserido).

A partir de agora iremos aplicar estes três conceitos em nosso estudo de caso da ACDN Rental Car.

Recapitulando o Estudo de Caso: ACDN Rental Car

Vamos relembrar um pouco do nosso estudo de caso. O sistema para a ACDN Rental Car.

A ACDN Rental Car permite aos seus clientes reservar carros a partir da Internet ou indo diretamente a uma das sedes da

empresa. Ela ainda oferece várias classes de carros e vários preços de locação.

Entre as principais regras de negócio da empresa, podemos citar:

- A locação de um veículo pode ser feita em uma sede e a devolução na outra sede, sem restrição. Caso o ponto de devolução seja diferente do ponto de locação, ocorrerá uma multa.
- Os carros são agrupados em classes. O preço de locação varia por classe do carro e tempo de locação.
- Um cliente não pode realizar uma segunda reserva caso ainda possua uma locação aberta (cujo carro ainda não foi retornado).

O modelo físico (ver **Nota DevMan 1**) do nosso banco de dados *AcadnRentalCar* está apresentado na **Figura 1**, onde usamos a ferramenta DBDesigner. Observe nesta figura que são apresentadas as tabelas, seus atributos e relacionamentos, explicitando as chaves estrangeiras de cada tabela que instanciam os relacionamentos.

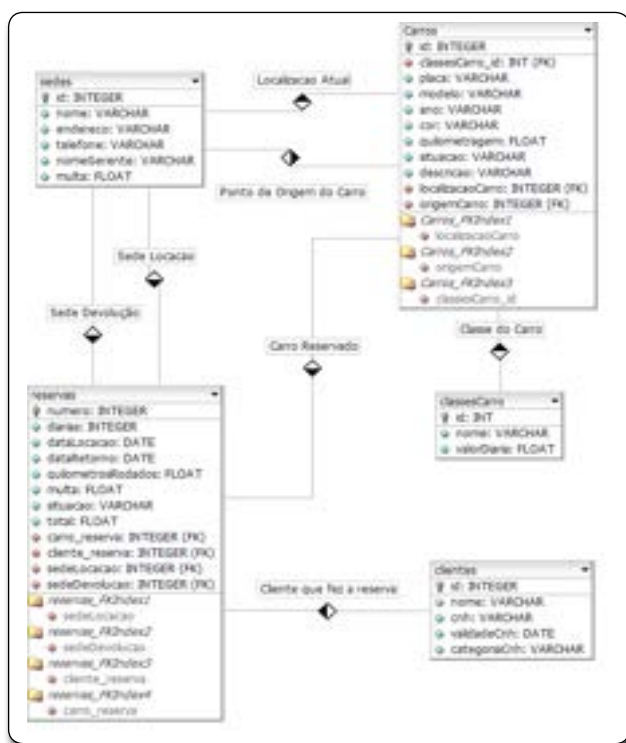


Figura 1. Modelo Físico do Sistema da ACDN Rental Car

Para atender à demanda de operações do sistema, o proprietário da empresa fez algumas solicitações aos desenvolvedores. São elas:

- Ao alugar um carro, este deve ficar inacessível aos demais clientes que desejam realizar uma reserva. Este só passa a ficar acessível novamente após a devolução do veículo ao seu ponto de origem.
- O proprietário da empresa deseja obter constantemente um relatório que apresenta qual o veículo que foi alugado pela



Nota do DevMan 1

Modelo Físico

Um modelo de dados físico é uma representação de um projeto de dados que leva em consideração as facilidades e restrições de um certo sistema gerenciador de banco de dados (SGBD). No ciclo de vida de um projeto, ele é tipicamente derivado a partir do modelo lógico. Um modelo de dados físico completo incluirá todos os artefatos de banco de dados necessários para criar relacionamentos entre tabelas ou obter os objetivos de desempenho, tais como índices, restrições, views, particionamentos, etc.

Por diversas vezes as pessoas tendem a confundir modelo de dados lógico e físico. A **Tabela 1** abaixo tenta esclarecer um pouco a diferença entre eles.

Modelo de Dados Lógico	Modelo de Dados Físico
Inclui entidades/tabelas, atributos/colunas/campos e relacionamentos	Inclui tabelas, colunas, chaves, tipos de dados, regras de validação, triggers, stored procedures, restrições de domínio e de acesso
Usa nome de elementos do domínio para seus atributos	Usa nomes de colunas abreviados limitados pelo SGBD
É independente de tecnologia (ex: plataforma ou SGBD)	Inclui chaves primárias e índices para acesso rápido aos dados
É normalizado até a quarta forma normal (ver artigo "Aplicando Normalização a um Modelo de Dados" publicado na edição 58 da SQL Magazine)	Pode ser desnormalizado para atingir requisitos de desempenho baseado na natureza do banco de dados

Tabela 1. Comparação entre Modelo Lógico x Modelo Físico

empresa, exibindo os dados do veículo, total da quilometragem rodada nas locações do período em questão, a quantidade de locações realizadas e, por fim, uma média de quilômetros rodados a cada locação.

- É necessário fazer o controle da quilometragem do veículo. Ou seja, ao final de uma locação deve ser atualizado no sistema a quantidade de quilômetros que o veículo rodou.

A seguir iniciaremos o processo para atender às demandas da empresa, realizando os ajustes necessários em nosso banco de dados.

Construindo Stored Procedures

Iniciaremos atendo à primeira demanda, onde foi solicitado que ao realizar uma venda, o carro fica impossibilitado de fazer uma nova reserva até que este seja devolvido ao seu ponto de origem.

Observem que nesta situação a operação de LOCAÇÃO possui duas tarefas: (1) registrar a locação do veículo na tabela de reserva e (2) atualizar a situação do carro para *alugado* em sua tabela. Da mesma forma, a operação de DEVOLUÇÃO possui duas tarefas: (1) registrar a devolução do veículo na tabela de reserva e (2) atualizar a situação do carro para *disponível*, caso esteja sendo devolvido em seu ponto de origem, ou *fora do ponto de origem*, caso esteja sendo devolvido em um ponto diferente. Essas operações podem ser implementadas com mais facilidade

se usarmos os recursos disponíveis pelas *stored procedures*. As **Listagens 6 e 7** apresentam os códigos referentes a essas duas operações.

Listagem 6. Stored procedure para a operação de LOCAÇÃO

```
CREATE PROCEDURE sp_registrarLocacao (v_diarias int(10), v_dataLocacao DATE, v_carro int(10), v_sedeOrigem int(10))
BEGIN
    // Inserindo registro de nova reserva
    INSERT INTO reservas (diarias, dataLocacao, carro_reserva, sedeLocacao) VALUES (v_diarias, v_dataLocacao, v_carro, v_sede);

    // Atualizando a situação do carro
    UPDATE carros SET situacao = 'alugado' WHERE id = v_carro;
END;
```

Observe na **Listagem 7** que esta SP parte do princípio que já foi feito o cálculo da multa a ser paga pelo cliente e que já foram obtidas as informações sobre a sede de locação.

Listagem 7. Stored procedure para a operação de DEVOLUÇÃO

```
CREATE PROCEDURE sp_registrarDevolucao (v_reserva int(10), v_dataRetorno DATE, v_quilometrosRodados float(8,2), v_multa float(8,2), v_total float(8,2), v_carro int(10), v_sedeLocacao int(10), v_sedeDevolucao int(10))
BEGIN
    // Atualizando registro de nova reserva
    UPDATE reservas SET dataRetorno = v_dataRetorno, quilometrosRodados = v_quilometrosRodados, multa = v_multa, total = v_total, sedeDevolucao = v_sedeDevolucao WHERE id = v_reserva;

    // Atualizando a situação do carro
    IF (v_sedeLocalcao != v_sedeDevolucao) THEN
        UPDATE carros WHERE id = v_carro SET situacao = "fora do ponto de origem";
    ELSE
        UPDATE carros WHERE id = v_carro SET situacao = "disponível";
    END IF
END;
```

Construindo Views

Vamos agora à segunda solicitação: um relatório com o carro mais alugado em um certo período de tempo.

Observe que para esta operação precisamos de diversas informações disponíveis em diferentes tabelas (como a tabela de *carros* e a de *reservas*). Para implementar esta necessidade, optamos pela criação de uma visão que possibilite exibir os dados desejados pelo cliente. A **Listagem 8** apresenta o código referente à criação desta *view*.

Se executarmos essa *view* para algum carro específico informado como parâmetro (ex: ID = 1), observamos que esta retorna exatamente o que o usuário deseja. A **Listagem 9** apresenta um exemplo de utilização da *view* criada e seu resultado.

Construindo Triggers

Vamos agora à terceira solicitação: manter atualizada a quilometragem rodada por um carro após a sua devolução.

Observe nessa situação que o ideal é que assim que um carro for devolvido seja feita a atualização da quilometragem do veículo. No entanto, fazer isso de forma manual seria muito custoso, e caso algum operador do sistema esquecesse ou colocasse um valor diferente daquele registrado na operação de devolução poderia deixar o sistema com dados incorretos. Então, optamos

Listagem 8. View para consulta aos dados de um carro

```
CREATE VIEW consultarReservasCarro AS
SELECT c.id, c.PLACA, c.MODELO, c.ANO, c.COR, c.SITUACAO, c.DESCRICAO, SUM( r.quilometrosRodados ) AS KMs, COUNT( r.numero ) AS QTDE
FROM carros AS c
JOIN reservas AS r ON c.id = r.carro_reserva
GROUP BY r.carro_reserva
```

Listagem 9. Utilizando a view consultarReservasCarro

```
SELECT * FROM consultarReservasCarro where id =1;
+-----+-----+-----+-----+-----+-----+
| id | PLACA | MODELO | ANO | COR | SITUACAO |
|-----+-----+-----+-----+-----+
| 1 | JWL-2981 | Sandero Stepway | 2009/2010 | Vermelho | disponível |
| 1 | JWL-2981 | Sandero Stepway | 2009/2010 | Vermelho | disponível |
| 1 | JWL-2981 | Sandero Stepway | 2009/2010 | Vermelho | disponível |
+-----+-----+-----+-----+-----+-----+
| id | PLACA | MODELO | ANO | COR | SITUACAO |
|-----+-----+-----+-----+-----+
| 1 | JWL-2981 | Sandero Stepway | 2009/2010 | Vermelho | disponível |
| 1 | JWL-2981 | Sandero Stepway | 2009/2010 | Vermelho | disponível |
+-----+-----+-----+-----+-----+-----+
```

pela criação de uma *trigger* que no momento em que for feita uma atualização na tabela *reservas*, a atualização caracteriza como uma operação de devolução, pois o registro da reserva é criado no momento da locação e atualizado na devolução, fará a atualização na coluna *quilometragem* da tabela *carros*, somando o valor rodado na locação que está sendo finalizada.

A **Listagem 10** apresenta o código da *trigger* responsável por esta operação em nosso banco de dados.

Listagem 10. Triggers para manter consistente a quilometragem rodada por um carro

```
CREATE TRIGGER tr_kmRodados after update on reservas
FOR EACH ROW
UPDATE carros SET quilometragem = quilometragem + NEW. quilometrosRodados WHERE NEW.carro_reserva = carros.id
```

Na **Listagem 11** podemos ver a *trigger* em funcionamento. Observe que antes de atualizarmos o registro da tabela *reservas* a quilometragem do carro com ID = 1 era 600 kms, e após a atualização deste registro ela passou a 750 kms, pois foi somado a ela o total de quilômetros rodados na locação finalizada, que foi de 150 quilômetros.

Listagem 11. Exibindo o funcionamento da trigger tr_kmRodados

```
SELECT id, placa, modelo, quilometragem FROM carros WHERE id =1;
+-----+-----+-----+-----+
| id | placa | modelo | quilometragem |
+-----+-----+-----+-----+
| 1 | JWL-2981 | Sandero Stepway | 600.00 |
+-----+-----+-----+-----+

UPDATE reservas SET quilometrosRodados = 150 WHERE numero = 1;

SELECT id, placa, modelo, quilometragem FROM carros WHERE id =1;
+-----+-----+-----+-----+
| id | placa | modelo | quilometragem |
+-----+-----+-----+-----+
| 1 | JWL-2981 | Sandero Stepway | 750.00 |
+-----+-----+-----+-----+
| id | placa | modelo | quilometragem |
+-----+-----+-----+-----+
| 1 | JWL-2981 | Sandero Stepway | 600.00 |
+-----+-----+-----+-----+
```

Com isso, atendemos às três demandas de nosso cliente de forma simples e eficiente, utilizando os recursos providos pelo nosso SGBD (MySQL). Os passos seguintes a serem realizados

seriam a implementação de novas *SPs*, *Views* e *Triggers*, assim como passar a olhar a questão de desempenho do banco de dados através da aplicação de técnicas de *tuning*, evitando um decaimento no desempenho do nosso banco de dados à medida que um grande volume de registros seja inserido.

Conclusões

Neste artigo vimos de uma forma prática como utilizar alguns dos principais recursos de banco de dados: *SPs*, *Views* e *Triggers*. Estes recursos fazem parte do dia-a-dia de qualquer DBA, pois constantemente nos deparamos com situações onde eles nos auxiliam de forma satisfatória.

Vimos que a aplicação destes recursos é de certa forma simples, quando utilizamos o MySQL. Para demonstrar tal

simplicidade, evoluímos o projeto do banco de dados de uma locadora de carros, já publicado em edições anteriores da SQL Magazine. Dessa forma, podemos ver uma possível forma de evolução do modelo e projeto de um banco de dados.

Abraços e até a próxima! ●

Dê seu feedback sobre esta edição!

A SQL Magazine tem que ser feita ao seu gosto. Para isso, precisamos saber o que você, leitor, acha da revista!

Dê seu voto sobre este artigo, através do link:

www.devmedia.com.br/sqlmagazine/feedback



Cursos Online



A Revista **WebMobile** oferece para seus assinantes uma série de Cursos Online de alto padrão de qualidade.

Conheça abaixo os cursos já disponíveis:

www.devmedia.com.br/curso/webmobile

- **Curso em .NET:** Aplicação completa de orçamento Doméstico no Visual Studio 2005
- **Curso em Java:** Introdução ao desenvolvimento para celulares com J2ME

A sua melhor opção de aprendizagem!

Assine a **WebMobile** e
Comece já seu treinamento!
www.devmedia.com.br/assine

Nesta seção você encontra artigos sobre banco de dados, SQL ou persistência

Statistics no Oracle

De que se trata o artigo?

Entender como as statistics e histograms auxiliam o CBO (Cost Based Optimizer – Otimizador Baseado em Custo).

Para que serve?

Para entendermos o impacto que estatísticas obsoletas ou ausentes causam nos planos de execução.

Em que situação o tema é útil?

Na análise de comandos SQL com problemas de desempenho.

Otimizador é o programa que realiza a transformação de comandos SQL em planos de execução. A partir do Oracle na versão 7, introduziu-se o Cost Based Optimizer (CBO) que, como o nome sugere, baseia-se no custo das operações. A partir do Oracle na versão 10g, esse otimizador se tornou default, sendo que seu antecessor, baseado em regras é bem mais simples de ser compreendido já que funciona a partir de 15 regras pré-definidas não levando em consideração a distribuição dos dados (ele é influenciado pela sintaxe do comando e ainda existe apenas por questões de compatibilidade). De acordo com [1], [2] e [3] (ver Referências Bibliográficas), o CBO gera suas

estimativas para acesso aos dados baseado nas métricas de custo, cardinalidade e seletividade, sendo influenciado por parâmetros, statistics e hints (diretivas de compilação). Daí a importância de entendermos como as estatísticas funcionam e auxiliam o CBO na geração do plano de execução.

Antes de entrarmos no tópico de estatísticas vamos entender um pouco mais sobre o otimizador e como são gerados os planos de execução. O CBO é dividido em 3 componentes principais, conforme mostrado na **Figura 1**.



José Eduardo Fiamengui Júnior

eduardoj.fiamengui@inmetrics.com.br

jeduardounicamp@gmail.com

Graduado em Tecnologia em Informática pela Unicamp e Pós-Graduado em Administração de Banco de Dados pelo IBTA. Analista de Desempenho da Inmetrics atuando em ambientes de alta criticidade.

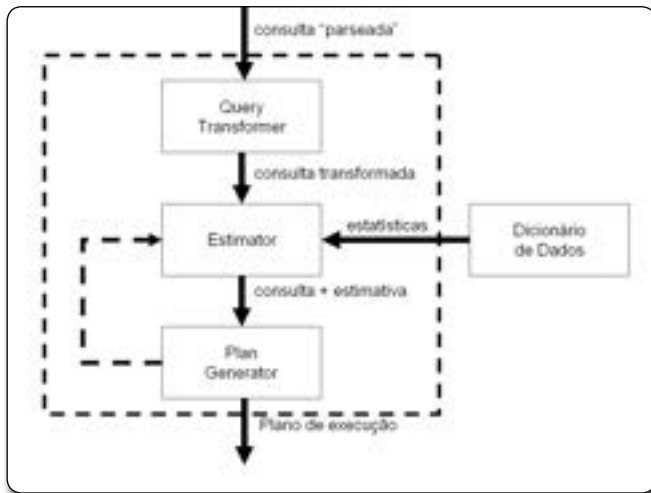


Figura 1. Componentes do CBO

O Query Transformer recebe um comando proveniente do analisador gramatical e o divide em diferentes blocos relacionados entre si. Seu objetivo é verificar se existe alguma forma mais otimizada para o comando em que se obtenha o mesmo resultado, gerando assim o melhor plano de execução. Para isto ele utiliza técnicas como:

- **view merging:** quando o analisador gramatical se depara com uma visão em um comando, ele a coloca em um bloco separado. O otimizador verifica qual é a melhor maneira de resolver essa visão com a criação de “sub-planos”. Após ter otimizado o bloco contendo a visão, o query transformer tenta agrupá-lo a outros blocos da consulta;
- **subquery unnesting:** assim como as visões, as sub-consultas são colocadas em blocos separados pelo analisador gramatical. O query transformer busca a melhor forma de resolver este comando, encontrando a maneira mais otimizada de resolver a sub-consulta e agrupando em um bloco da consulta principal, transformando esta consulta em uma junção da consulta principal;
- **query rewrite with materialized views:** uma visão materializada é similar a uma consulta armazenada em uma tabela. Quando o query transformer encontra uma consulta compatível com uma materialized view, a consulta é reescrita para utilizar a estrutura. Isto traz ganhos de desempenho, uma vez que parte do resultado já está previamente pronto, dispensando a necessidade de executá-lo novamente;
- **predicate pushing:** agrega um predicado que fica fora da visão para dentro da visão se isso melhorar na geração dos planos de execução.

O Estimator é responsável por indicar o esforço estimado para cada plano apresentado pelo QueryTransformer. Para gerar o custo, o estimator baseia-se em duas métricas: seletividade e cardinalidade, que serão explicados no decorrer deste artigo.

O Plan Generator testa diferentes planos de execução para uma consulta e escolhe aquele que apresenta o menor custo. Estes planos são gerados com variações de formas de acesso,

formas de junção e ordens de junção. Para gerar o plano de execução de uma consulta, são gerados os sub-planos para cada bloco de visões não agrupadas e sub-consultas. Estes blocos são otimizados separadamente de baixo para cima, ou seja, a sub-consulta mais interna é otimizada em primeiro lugar, e um sub-plano é gerado para ela. Já a consulta mais externa é a última a ser otimizada. O número de planos possíveis é proporcional à quantidade de junções na cláusula FROM. Para iniciar uma pesquisa pelo melhor plano, o gerador de planos organiza os itens da junção de acordo com sua cardinalidade, o item de menor cardinalidade é colocado em primeiro lugar e assim sucessivamente até o item de maior cardinalidade.

Agora vamos definir rapidamente os conceitos que influenciam o comportamento do otimizador:

- **Custo:** número gerado pelo otimizador para determinar o melhor método de acesso (estimativa);
- **Seletividade:** porcentagem de linhas retornadas conforme a aplicação de uma condição (filtro). A fórmula utilizada para cálculo depende da distribuição dos dados e da quantidade de buckets. Para as situações que analisaremos neste artigo utilizaremos as três **Equações 1, 2 e 3**.

$$\text{Sel Esperada} = \left(\frac{1}{\text{NP Values}} \right) * \left(\frac{\text{\#NP buckets}}{\text{\#buckets}} \right)$$

Equação 1. Seletividade para valores não-populares

$$\text{Sel Esperada} = \left(\frac{\text{\#PV buckets}}{\text{\#buckets}} \right)$$

Equação 2. Para valores populares

$$\text{Sel Esperada} = \frac{1}{\text{NDV}}$$

Equação 3. Quando não há histograma coletado para a coluna

Onde:

- NP values: valores não-populares;
- NP buckets: buckets não-populares;
- PV buckets: buckets de valores populares;
- buckets: buckets (onde a distribuição dos valores é armazenada);
- NDV: número de valores distintos.

As variáveis acima serão melhor explicadas no decorrer do artigo.

- **Cardinalidade:** representa o número de registros retornados em uma determinada operação do plano de execução (**Equação 4**).

$$\text{Card Esperada} = \frac{\text{Linhas Processadas}(\text{Prod Cartesiano}) * \text{Sel Esperada}}{\text{Sel Esperada}}$$

Equação 4. Cardinalidade

Agora que entendemos estes importantes conceitos, vamos a um exemplo prático do otimizador realizando estes cálculos conforme a **Listagem 1**.

Listagem 1. Exemplo prático

```

1 create table t2
2 as select trunc(dbms_random.value(0,5000)) join1,
3 trunc(dbms_random.value(0, 40 )) v1, rpad('x',100) padding
4 from all_objects where rownum <= 10000;

5 create table t1
6 as select trunc(dbms_random.value(0,4)) join1,
7 trunc(dbms_random.value(0, 40 )) v1,
8 rpad('x',100) padding from
9 all_objects where rownum <= 10000;

10 select count(*) from t1 ;

11 COUNT(*)
12 -----
13      10000

14 select count(*) from t2 ;

15 COUNT(*)
16 -----
17      10000

18 execute dbms_stats.gather_table_stats(ownname=>'SCOTT',
19 tabname=>'T2',method_opt=>'for all columns size 1');

20 execute dbms_stats.gather_table_stats(ownname=>'SCOTT'
21 tabname=>'T1',method_opt=>'for all columns size 1');

21 col TABLE_NAME format a10
22 col COLUMN_NAME format a10
23 select num_distinct,
24 density,
25 column_name,
26 table_name
27 from user_tab_col_statistics
28 where table_name in ('T1','T2')
29 and column_name in ('JOIN1','V1');

30 NUM_DISTINCT DENSITY COLUMN_NAME TABLE_NAME
31 -----
32          4          ,25      JOIN1      T1
33          40          ,025      V1      T1
34         4333      ,000230787      JOIN1      T2
35          40          ,025      V1      T2

36 explain plan for
37 select count(*) from t1, t2
38 where t1.join1 = t2.join1 and t1.v1 = 1;

39 select * from table(dbms_xplan.display);

40 -----
41 | Id | Operation | Name | Rows | Cost (%CPU) |
42 |----|-----|-----|-----|-----|
43 | 0 | SELECT STATEMENT | | 1 | 80 |
44 | 1 | SORT AGGREGATE | | 1 | |
45 | * 2 | HASH JOIN | | 577 | 80 |
46 | * 3 | TABLE ACCESS FULL | T1 | 250 | 40 |
47 | 4 | TABLE ACCESS FULL | T2 | 10000 | 39 |
48 -----

49 Predicate Information (identified by operation id):
50 -----
51 2 - access("T1"."JOIN1"="T2"."JOIN1")
52 3 - filter("T1"."V1"=1)

53 Operações:
54 Número de linhas processadas : 10000*250
55 Seletividade(sem histogramas): 1/NDV = 1/4334
56 Cardinalidade: 10000*250/4334 = 576,83 ~ 577

```

A coluna DENSITY no dicionário de dados da tabela USER_TAB_COL_STATISTICS representa a seletividade. Agora que já vimos como o CBO calcula a seletividade e cardinalidade das colunas, vamos falar do tópico cerne deste artigo que são as estatísticas.

As estatísticas quantificam a distribuição dos dados e as características de armazenamento de tabelas, colunas, índices,

blocos e partições. O CBO utiliza as estatísticas para calcular a cardinalidade e seletividade das colunas e, assim, gerar o método de acesso com menor custo. Na coleta de estatísticas, são armazenadas as seguintes informações:

- Table statistics (estatísticas da tabela):
 - Número de linhas;
 - Número de blocos;
 - Número de blocos vazios;
 - Número de *chained* ou *migrated rows* (**Nota DevMan 1**);
 - Data da coleta;
 - Tamanho da amostra; e
 - Tamanho médio da linha;
- Column statistics (estatísticas da coluna):
 - Número de valores distintos na coluna (NDV);
 - Quantidade de valores nulos;
 - Menor e maior valor;
 - Data da coleta; e
 - Tamanho da amostra;
- Index statistics (estatísticas do índice):
 - Altura, número de blocos folhas;
 - Número de chaves distintas;
 - Número médio de blocos folhas por chave;
 - Número de entradas, *cluster in factor*;
 - Data da coleta; e
 - Tamanho da amostra;
- System statistics (estatísticas do sistema):
 - I/O performance e utilização;
 - CPU performance e utilização.



Nota do DevMan 1

Chained ou Migrated Rows

O banco de dados Oracle precisa gerenciar as linhas que são grandes demais para caber em um único bloco. As seguintes situações são possíveis:

- A linha é grande demais para caber em um bloco de dados quando ele é inserido pela primeira vez:

No encadeamento de linha (row chaining), o Oracle armazena os dados da linha em uma cadeia de um ou mais blocos de dados reservado para o segmento. Row chaining ocorre mais frequentemente com linhas (registros) grandes, em campos do tipo LONG ou LONG RAW, um VARCHAR2 (4000), ou uma linha com um número muito grande de colunas. Chained rows, nestes casos, é inevitável.

- A linha que originalmente "cabe" em um bloco de dados é atualizada (update) de modo que seu tamanho aumente, mas não existe espaço livre suficiente no bloco para manter a linha atualizada.

Na migração de linha (row migration), o Oracle move toda a linha para um novo bloco de dados, assumindo que a linha pode caber em um novo bloco. O bloco original de uma linha migrada contém um ponteiro ou "endereço de encaminhamento" para o novo bloco que contém as linhas migradas. O ROWID de uma linha migrada não muda.

- A linha tem mais de 255 colunas.

O banco de dados Oracle só pode armazenar 255 colunas em um pedaço de linha. Assim, se você inserir uma linha em uma tabela que tem 1000 colunas, então o banco de dados cria 4 pedaços de linha, normalmente ligadas por vários blocos.

Visualizando as informações, torna-se fácil entender porque a coleta deve ser realizada periodicamente. Podemos verificar se as estatísticas para os objetos estão coletadas e atualizadas através das views: DBA_TABLES, DBA_TAB_COL_STATISTICS, DBA_INDEXES, DBA_CLUSTERS, DBA_TAB_PARTITIONS, DBA_TAB_SUBPARTITIONS, DBA_IND_PARTITIONS, DBA_IND_SUBPARTITIONS, DBA_PART_COL_STATISTICS, DBA_SUBPART_COL_STATISTICS.

No Oracle9i, é necessário criar um método para a coleta periódica de estatísticas e agendá-la para execução. A partir da versão 10g, isso é controlado pelo Oracle que, na sua instalação, cria o job GATHER_STATS_JOB e sua execução é realizada nos dias úteis entre as 22h e 06h e em todos os finais de semana. O parâmetro `statistics_level` controla o nível de coleta que é executado na base de dados, podendo apresentar os seguintes valores:

- BASIC: estatísticas não são coletadas;
- TYPICAL (default e recomendada): coleta estatísticas de objetos e das views advisors buffer cachê, shared pool, PGA target, MTTR;
- ALL: typical + estatísticas de SO (Sistema Operacional).

A package DBMS_STATS é recomendada a partir do Oracle8i para coleta de estatísticas. Com esta package é possível paralelizar o processo de coleta e a partir do Oracle9i ela deve ser utilizada para coletar as estatísticas do sistema. As estatísticas do sistema têm por objetivo fazer com que o otimizador colete métricas como tempo de leitura single-block e multiblock, entre outras.

As principais funcionalidades dessa package podem ser vistas na **Tabela 1**.

Procedure	Coletas
GATHER_INDEX_STATS	Estatísticas dos índices
GATHER_TABLE_STATS	Tabelas, colunas e estatísticas dos índices
GATHER_SCHEMA_STATS	Estatísticas de todos os objetos do <i>schema</i>
GATHER_DATABASE_STATS	Estatísticas para todos os objetos do banco de dados
GATHER_SYSTEM_STATS	Estatísticas de CPU e I/O do sistema
EXPORT_SCHEMA_STATS	Realiza <i>export</i> das estatísticas de um <i>schema</i>
IMPORT_SCHEMA_STATS	Realiza <i>import</i> das estatísticas de um <i>schema</i>
LOCK_TABLE_STATS	Bloqueia a coleta de estatísticas em uma tabela
UNLOCK_TABLE_STATS	Desbloqueia a coleta de estatísticas em uma tabela
RESTORE_TABLE_STATS	Restaura estatísticas de uma tabela

Tabela 1. package dbms_stats

Note que algumas destas funcionalidades podem ser utilizadas para simular ambientes, por exemplo: em empresas onde o ambiente de homologação não contém disco (espaço) suficiente para todos os dados que há em produção, pode-se exportar as estatísticas de produção na base de homologação. Desta forma é possível visualizar os planos de execução analisando cada método de acesso e testando a eficiência do comando SQL.

A partir do Oracle9i foi disponibilizado também o parâmetro `optimizer_dynamic_sampling` que controla o nível de coleta de estatísticas em tempo real. Ou seja, coleta de informações de tabelas sem estatísticas em tempo de execução. Os valores default deste parâmetro variam de acordo com a versão, conforme mostrado na **Tabela 2**.

Versão	Default
Oracle >= 9.0.0	0
Oracle >= 9.2.0	1
Oracle >= 10.0.0	2

Tabela 2. Default optimizer_dynamic_sampling

A grande vantagem da utilização deste parâmetro é quando temos na base de dados *volatile tables* que são definidas como tabelas cujo conteúdo pode variar de vazio a muito grande em tempo de execução. Esta volatilidade extrema pode resultar em estatísticas imprecisas e conseqüentemente em planos incorretos e com mau desempenho. Por exemplo, se as estatísticas são coletadas quando a tabela está vazia o otimizador tende a favorecer este acesso como forma de restringir os registros. Para estes casos é recomendável que as estatísticas da tabela fiquem bloqueadas, pois assim o `optimizer_dynamic_sampling` será utilizado.

Agora vejamos na **Tabela 3** os níveis de coleta realizados possíveis a se definir no `optimizer_dynamic_sampling`.

Nível	Descrição
0	Feature desabilitada
1	Coleta de tabelas sem estatísticas e índices
2	Coleta de todas as tabelas sem estatísticas (default sample size = 32KB)
3	Idem nível 2 + tabelas que otimizador utiliza estimativas em predicados
4	Idem nível 3 + tabelas com predicados em mais de 1 coluna
5	Idem nível 4, mas com 2 * default sample size
6	Idem nível 4, mas com 4 * default sample size
7	Idem nível 4, mas com 8 * default sample size
8	Idem nível 4, mas com 32 * default sample size
9	Idem nível 4, mas com 128 * default sample size
10	Idem nível 4, mas com todos os blocos

Tabela 3. Níveis de coleta do Optimizer_dynamic_sampling

Na **Listagem 2** foram criadas as tabelas `prob_tab` e `build_tab` cópias da `all_objects` e realizamos uma consulta relacionando essas tabelas, mas sem a coleta de estatísticas. Vamos analisar qual será o plano de execução gerado pelo otimizador.

Nota-se a grande diferença nos PEs gerados, já que no primeiro deles o otimizador une as tabelas através do NESTED LOOPS. Neste método o CBO elege uma fonte principal (outer table), que servirá de base para buscar os registros na fonte dependente (inner table). Para cada registro existente na fonte principal, o Oracle buscará os registros na fonte dependente que atendam às condições da fonte principal, o retorno dos dados é realizado a cada colisão.

Listagem 2. Falta de estatística

```

1  create table probe_tab as
2  select 10000 + rownum id, trunc(dbms_random.value(0,5000)) n1,
3  rpad(rownum,20) probe_vc, rpad('x',500) probe_padding
4  from all_objects
5  where rownum <= 5000 ;

6  alter table probe_tab add constraint pb_pk primary key(id);

7  create table build_tab as Select rownum id,
8  10001 + trunc(dbms_random.value(0,5000)) id_probe,
9  rpad(rownum,20) build_vc,
10 rpad('x',500) build_padding
11 from all_objects
12 where rownum <= 5000;

13 alter table build_tab add constraint bu_pk primary key(id);
14 alter table build_tab add constraint bu_fk_pb foreign key (id_probe)
15 references probe_tab;

16 -- Após a criação das tabelas, vamos verificar o PE para a
17 -- query abaixo
18 -- (utilizando a ferramenta dbms_xplan):

18 EXPLAIN PLAN FOR select bu.build_vc,
19 pb.probe_vc,
20 pb.probe_padding
21 from build_tab bu,
22 probe_tab pb
23 where bu.id between 1 and 500
24 and pb.id = bu.id_probe ;

25 select * from table(dbms_xplan.display) ;

26 | Id | Operation | Name | Rows
27 |---|---|---|---|
28 | 0 | SELECT STATEMENT | | 82
29 | 1 | NESTED LOOPS | |
30 | 2 | NESTED LOOPS | | 82
31 | 3 | TABLE ACCESS BY INDEX ROWID | BUILD_TAB | 82
32 | * 4 | INDEX RANGE SCAN | BU_PK | 147
33 | * 5 | INDEX UNIQUE SCAN | PB_PK | 1
34 | 6 | TABLE ACCESS BY INDEX ROWID | PROBE_TAB | 1
35 |---|---|---|---|

36 Predicate Information (identified by operation id):
37
38 4 - access("BU"."ID">=1 AND "BU"."ID"<=500)
39 5 - access("PB"."ID"="BU"."ID_PROBE")

40 -- Vamos analisar o desempenho desta consulta:

41 set timing on
42 select bu.build_vc, pb.probe_vc, pb.probe_padding
43 from build_tab bu, probe_tab pb
44 where bu.id between 1 and 500
45 and pb.id = bu.id_probe ;

46 500 linhas selecionadas.
47 Decorrido: 00:00:00.08

48 -- Coletando statistics para as tabelas envolvidas:
49 EXEC DBMS_STATS.gather_table_stats('SCOTT', 'PROBE_TAB');
50 Procedimento PL/SQL concluído com sucesso.
51 EXEC DBMS_STATS.gather_table_stats('SCOTT', 'BUILD_TAB');
52 Procedimento PL/SQL concluído com sucesso.

53 EXPLAIN PLAN FOR select bu.build_vc,
54 pb.probe_vc,
55 pb.probe_padding
56 from build_tab bu,
57 probe_tab pb
58 where bu.id between 1 and 500
59 and pb.id = bu.id_probe ;

60 select * from table(dbms_xplan.display) ;

61 | Id | Operation | Name | Rows
62 |---|---|---|---|
63 | 0 | SELECT STATEMENT | | 500
64 | * 1 | HASH JOIN | | 500
65 | 2 | TABLE ACCESS BY INDEX ROWID | BUILD_TAB | 500
66 | * 3 | INDEX RANGE SCAN | BU_PK | 500
67 | 4 | TABLE ACCESS FULL | PROBE_TAB | 5000
68 |---|---|---|---|

70 Predicate Information (identified by operation id):
71
72 1 - access("PB"."ID"="BU"."ID_PROBE")
73 3 - access("BU"."ID">=1 AND "BU"."ID"<=500)

74 set timing on
75 select bu.build_vc, pb.probe_vc, pb.probe_padding
76 from build_tab bu, probe_tab pb
77 where bu.id between 1 and 500
78 and pb.id = bu.id_probe ;

79 Decorrido: 00:00:00.04

```

Após a coleta de estatísticas o CBO passa a utilizar o método HASH JOIN. Neste método uma hash table é construída para a menor das duas tabelas. A menor tabela é então colocada em memória e é usada para encontrar as linhas que combinam com a tabela maior. O retorno dos dados neste método não ocorre a cada colisão.

Os histogramas ajudam o CBO a visualizar a distribuição dos dados nas colunas. Quando falamos em histogramas devemos pensar em:

- Valores Não-Populares (PN): aparecem no máximo uma vez como ENDPOINT_VALUE;
- Valores Populares (PV): aparecem mais de uma vez como ENDPOINT_VALUE;
- Buckets: buckets onde a distribuição dos valores é armazenado.

O ENDPOINT_VALUE é o valor do ponto final de um bucket. Vamos a um exemplo didático para entender como os histogramas auxiliam o CBO. Imagine uma coluna que possua 100 valores uniformemente distribuídos de 1 a 100, conforme mostrado na **Figura 2**.

Segundo a **Equação 1**, temos (supondo que a coluna não tem histogramas):

$$\text{Sel Esperada} = 1/100 * 1/1 = 0,01$$

$$\text{Card Esperada} = 100 * 0,01 = 1$$

Com essa distribuição o otimizador estimaria a cardinalidade corretamente e isso faria com que o acesso aos dados fosse o menos custoso.

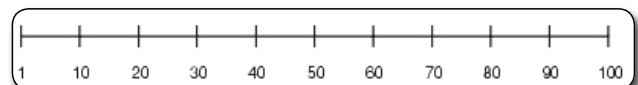


Figura 2. Distribuição uniforme

Agora vejamos como seria a distribuição não-uniforme desses dados, na **Figura 3**.

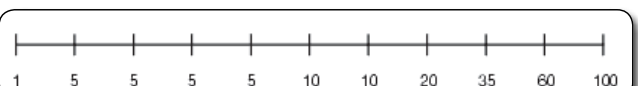


Figura 3. Distribuição não-uniforme

Neste caso, a maioria das linhas tem o valor 5 e apenas 10% das linhas apresentam valores entre 60 e 100. A cardinalidade seria calculada de maneira incorreta e seria erroneamente estimada pelo otimizador.

Vamos imaginar a distribuição destes valores em 10 buckets e realizar os cálculos do otimizador.

Segundo a **Equação 2**, temos:

Sel Esperada: $4/10 = 0,4$

Card Esperada = $100 * 0,4 = 40$

Nota-se que desta forma a cardinalidade estimada pelo otimizador seria muito mais próxima da realidade permitindo-lhe o acesso aos dados da forma mais eficiente.

Na **Listagem 3** temos um caso real de como os histogramas auxiliam o otimizador.

Listagem 3. Histogramas em ação

```

1 select coluna, count(*) from t1
2   group by coluna
3   order by count(*) ;
4 COLUNA      COUNT(*)
5 -----
6 1              3
7 2             39
8 3             58
9 4            400
10 5          1883969 (30%)
11 6          4314398 (69%)

12 -- verificando a presença de estatísticas(sem histogramas)
13 TABLE_NAME NUM_ROWS LAST_ANALYZED
14 -----
15 T1          6198867  17-jul-2009

16 -- comando SQL

17 explain plan for select * from t1
18   where coluna in ('1','2')
19   order by 1 ;
20
21 | Id | Operation | Name | Rows |
22 |----|-----|-----|-----|
23 | 0 | SELECT STATEMENT |      | 2704K |
24 | 1 | SORT ORDER BY |      | 2704K |
25 | * 2 | TABLE ACCESS FULL | T1 | 2704K |

26 Predicate Information (identified by operation id):
27
28 2 - filter("T1"."COLUNA"='1' OR "T1"."COLUNA"='2')

31 -- criando histogramas
32 EXECUTE DBMS_STATS.GATHER_TABLE_STATS (ownname => 'SCOTT',
33   tabname => 'T1',method_opt => 'FOR COLUMNS SIZE 100 COLUNA',
34   degree => 2, cascade => TRUE) ;

35 -- explain do commando
36 explain plan for select * from t1
37   where coluna in ('1','2')
38   order by 1 ;
39
40 | Id | Operation | Name | Rows |
41 |----|-----|-----|-----|
42 | 0 | SELECT STATEMENT |      | 1 |
43 | 1 | SORT ORDER BY |      | 1 |
44 | 2 | INLIST ITERATOR |      | 1 |
45 | 3 | TABLE ACCESS BY INDEX ROWID | T1 | 1 |
46 | * 4 | INDEX RANGE SCAN | IDX001 | 1 |

47 Predicate Information (identified by operation id):
48
49 3 - filter("T1"."COLUNA"='1' OR "T1"."COLUNA"='2')

```

Os principais parâmetros que deverão ser utilizados para a coleta de estatísticas são:

- **estimate_percent**: estima a estatística por amostragem. Este valor é a porcentagem de linhas que o Oracle utilizará como amostra. Caso você coloque o valor NULL, são computadas as estatísticas de todo o objeto.
- **method_opt**: controla para quais colunas deverão ser construídos os histogramas e quantos buckets eles devem ter. A opção AUTO permite que o otimizador decida com base na

carga de trabalho e distribuição de dados da coluna. A opção REPEAT recria o histograma com o mesmo número de buckets que havia anteriormente, e a opção SKEWONLY cria um novo histograma conforme a distribuição dos dados. O número default de buckets é 75 e o máximo de buckets que pode ser criado é 255.

- **cascade**: controla se objetos dependentes como índices, serão analisados.

A presença de histogramas pode ser verificada através das views DBA_TAB_HISTOGRAMS, DBA_HISTOGRAMS e DBA_TAB_COL_STATISTICS.

Ao mostrar estes exemplos pode ser que seja vista com bom olhos a criação de histogramas para quaisquer colunas da base de dados, no entanto há um problema relacionado a histogramas e a utilização de binds conhecido como bind variable peeking. Para entendermos esta feature é necessário lembrarmos as etapas de processamento do comando SQL, conforme mostrado na **Figura 4**.



Figura 4. Fases de processamento do SQL

Os comandos SQL são processados na Shared Pool que fica dentro da SGA (System Global Área) que é por sua vez subdividida em:

- Library Cache: armazena informações sobre comandos SQL e PL/SQL e são gerenciados pelo algoritmo LRU (Least Recently Used – algo como Menos Usado Recentemente) que consiste em descartar o comando que está há mais tempo sem uso na memória e colocar o comando requisitado nessa posição;
- Data Dictionary Cache: armazena metadados do dicionário, como informações sobre datafiles, tabelas, índices, colunas, usuários, privilégios e outros objetos.

Explicando rapidamente cada etapa de processamento:

- OPEN: é aberto um cursor para processamento do comando;
- PARSE: análise sintática do comando (se o comando está escrito corretamente), análise semântica do comando (se os objetos e privilégios necessários existem), gera o identificador do comando (hash_value), verifica se o ambiente em que o comando foi executado é o mesmo (alterações de parâmetros), transforma os comandos (para que haja mais possibilidades para geração do plano de execução) e gera os possíveis planos de execução;
- BIND: atribui valores aos binds da instrução;
- EXECUTE: nesta fase o comando é executado;
- FETCH: retorno dos dados buscado pelos comandos
- CLOSE: fechamento do cursor.

Após entendermos as etapas do processamento de um SQL é necessário entendermos também os conceitos de Hard Parse e Soft Parse:

- Hard Parse: é o processamento do comando SQL passando por todas as etapas demonstradas na **Figura 4**.
- Soft Parse: é o processamento do comando SQL até o cálculo do hash value e verificação do ambiente. Após isso o plano de execução é reaproveitado.

Explicados estes conceitos essenciais, podemos voltar a falar da feature bind variable peeking. Conforme demonstrado na **Figura 4**, normalmente os binds são atribuídos ao comando SQL após a geração do plano de execução (após a fase de PARSE). Com esta feature, para que não seja ignorada a presença de histogramas nas colunas apenas no Hard Parse do comando SQL o otimizador realiza a fase de bind anteriormente à fase de PARSE.

Pensando no exemplo da **Listagem 3**, quando isso nos traria problemas?

Imaginemos que no Hard Parse do nosso comando SQL os valores buscados sejam os menos populares, como 1 e 2. Se na segunda execução (Soft Parse) o valor buscado seja 6 (super popular, com 69% das linhas), o PE (Plano de Execução) gerado não será o mais eficiente já que para a busca deste valor requisições multiblocks (Full Table Scan e/ou Index Fast Full Scan) retornariam os dados mais rapidamente.

No Oracle 11g há uma nova feature denominada Adaptive Cursor que analisa colunas com a presença de histogramas e caso haja alteração no valor selecionado, por exemplo, de Popular Value para Non-Popular Value o plano de execução é gerado novamente.

Agora que conhecemos todas as possibilidades, quando devemos criar histogramas?

A resposta a essa pergunta é especialmente em duas situações: quando a coluna for utilizada na cláusula WHERE dos comandos e apresentarem distribuição não-uniforme.

Conclusão

Concluímos que a coleta periódica de estatísticas e o estudo de colunas não-uniformes que são utilizadas pelos filtros das operações são de grande importância para que o CBO consiga estimar corretamente a cardinalidade e seletividade para cada método de acesso e, dessa forma, chegar ao plano de execução com menor custo real. ●

Referências Bibliográficas

- [1] LEWIS, Jonathan. Cost Based Oracle 2005
- [2] Oracle Database 10g Release 1 (10.1)
- [3] Oracle 9i Database Performance Guide and Reference Release 2 (9.2)
- [4] <http://www.oracle-base.com/articles/10g/PerformanceTuningEnhancements10g.php>, acessado em 15 de julho de 2009.
- [5] http://download.oracle.com/docs/cd/B19306_01/server.102/b14211/stats.htm#i42219, acessado em 15 de julho de 2009.

Dê seu feedback sobre esta edição!

A SQL Magazine tem que ser feita ao seu gosto. Para isso, precisamos saber o que você, leitor, acha da revista!

Dê seu voto sobre este artigo, através do link:
www.devmedia.com.br/sqlmagazine/feedback



Nesta seção você encontra artigos sobre banco de dados, SQL ou persistência

Introdução à Gerência de Configuração

De que se trata o artigo?

Este artigo apresenta conceitos gerais da gerência de configuração de software. Além disso, resalta alguns dos benefícios que esta área provê, e sua relação com o processo de desenvolvimento de software.

Para que serve?

A Gerência de Configuração de Software surgiu da necessidade de controlar modificações ocorridas ao longo do desenvolvimento do software, por meio de métodos e ferramentas, com o intuito de maximizar a

produtividade e minimizar os erros cometidos durante a evolução.

Em que situação o tema é útil?

A gerência de configuração é uma disciplina que controla e notifica as inúmeras correções, extensões e adaptações aplicadas durante o ciclo de vida do software de forma a assegurar um processo de desenvolvimento e evolução sistemático e rastreável, sendo indispensável quando equipes manipulam, muitas vezes em conjunto, artefatos comuns.

Os sistemas de software estão em constante evolução. A manutenção do software chega a consumir 75% do custo total do seu ciclo de vida. Aproximadamente, 20% de todo o esforço de manutenção é usado para consertar erros de implementação e os outros 80% são utilizados na adaptação do software em função de modificações em requisitos funcionais, regras de negócios e na reengenharia da aplicação. A Gerência de Configuração de Software surgiu da necessidade de controlar estas modificações, por meio de métodos e

ferramentas, com o intuito de maximizar a produtividade e minimizar os erros cometidos durante a evolução.

É uma disciplina que controla e notifica as inúmeras correções, extensões e adaptações aplicadas durante o ciclo de vida do software de forma a assegurar um processo de desenvolvimento e evolução sistemático e rastreável, sendo indispensável quando equipes manipulam, muitas vezes em conjunto, artefatos comuns.

Apesar de existir um forte apelo para o uso da Gerência de Configuração de Software durante a etapa de manutenção, a



Rodrigo Spínola

rodrigo@sqlmagazine.com.br

Editor Chefe – SQL Magazine | WebMobile |

Engenharia de Software Magazine

Diretor de Operações - Kali Software

(www.kalisoftware.com)

Doutorando e Mestre em Engenharia de Software pela COPPE/UFRJ.

sua aplicação não se restringe somente a essa etapa do ciclo de vida do software. O uso dos sistemas de Gerência de Configuração é fundamental para prover controle sobre os artefatos produzidos e modificados por diferentes recursos desde o planejamento e levantamento de requisitos até a construção e entrega do produto. O motivo da sua importância está geralmente associado aos problemas identificados quando a Gerência de Configuração não é utilizada no desenvolvimento de software. Abaixo, vamos analisar alguns destes problemas.

Imagine que uma organização desconhece o que seja Gerência de Configuração e que em um determinado projeto um desenvolvedor esteja modificando os artefatos C1, C2 e C3 em um diretório compartilhado na rede. Simultaneamente, um segundo desenvolvedor modifica os artefatos C4, C5 e também o artefato C3, como exemplifica a **Figura 1**.

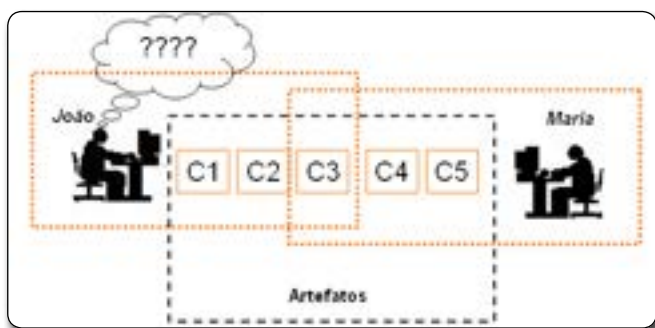


Figura 1. Espaço de Trabalho compartilhado por vários desenvolvedores

Neste cenário, o segundo desenvolvedor não notifica o primeiro desenvolvedor sobre o impacto que a modificação do artefato C3 pode causar no código. Conseqüentemente, o primeiro desenvolvedor, que está usando o mesmo espaço de trabalho, não conseguirá identificar, de forma rápida, o motivo que levou sua implementação a falhar. Este problema acontece pela falta de notificação e pelo compartilhamento de artefatos de software por diversos desenvolvedores.

Sob a perspectiva de desenvolvimento, a Gerência de Configuração de Software abrange três sistemas principais:

- O **sistema de controle de versões** permite que os artefatos sob Gerência de Configuração evoluam de forma distribuída, concorrente e disciplinada, evitando perdas ou sobreposições durante o desenvolvimento e a manutenção do artefato. Podemos citar como exemplos de ferramentas de mercado: CVS, Subversion, IBM Rational ClearCase e Microsoft Visual Source Safe.
- O **sistema de controle de modificações** armazena todas as informações geradas durante o andamento das solicitações de modificação e relata essas informações aos participantes interessados e autorizados. Podemos citar como exemplos de ferramentas de mercado: Bugzilla, Jira, Trac e IBM Rational ClearQuest.
- O **sistema de gerenciamento de construção** automatiza o processo de transformação dos diversos artefatos do software que compõem um projeto em um sistema executável

propriamente dito. Este processo é nomeado construção do software que, por exemplo, testa e empacota a aplicação java como um arquivo jar. Este processo ocorre de forma aderente às normas, procedimentos, políticas e padrões definidos para o projeto. Podemos citar como exemplos de ferramentas de mercado: Maven e Apache Ant.

As vantagens da utilização da Gerência de Configuração de Software são inúmeras. Dentre elas, podemos listar:

- 1) ganho de produtividade e eficiência;
- 2) diminuição do retrabalho e dos erros;
- 3) aumento da disciplina no processo de desenvolvimento;
- 4) aumento da memória organizacional;
- 5) acesso às informações qualitativas e quantitativas referentes ao processo de desenvolvimento, como por exemplo, medida de esforço para efetuar uma alteração e frequência de modificações por componente;
- 6) possibilidade de estabelecer uma trilha de auditoria indicando por que, quando e por quem um artefato foi alterado;
- 7) auxílio à gerência de projetos e
- 8) garantia de ambiente estável no qual o produto deve ser desenvolvido.

Gerência de Configuração e o Desenvolvimento de Software

O desenvolvimento de um software pode ser organizado de várias formas. A cada uma dessas formas de organização dá-se o nome de um *paradigma de ciclo de vida*. Os paradigmas mais estudados são o clássico, a prototipação, o modelo espiral, as técnicas de quarta geração e os modelos evolutivos tais como RUP e XP (Pressman, 2005).

Um processo de desenvolvimento de software, independente do paradigma de ciclo de vida adotado, inclui as fases de engenharia de sistemas, análise de requisitos, projeto de software, codificação, testes e manutenção (Pressman, 2005). Na **Tabela 1** é apresentada uma breve descrição de cada uma dessas fases.

Durante o processo de desenvolvimento de um software, uma grande quantidade de itens de informação

Fases	Descrição
Engenharia de Sistemas	Coleta dos requisitos em nível do sistema, com uma pequena quantidade de projeto e análise de alto nível.
Análise de Requisitos	Compreensão do domínio da informação através dos requisitos coletados na fase anterior.
Projeto de Software	Desenvolvimento de quatro atributos distintos do software: estrutura de dados, arquitetura de software, detalhes procedimentais e caracterização de interfaces.
Codificação	Tradução do projeto de software numa forma legível para a máquina.
Testes	Realização de testes dos programas. Esses testes concentram-se nos aspectos lógicos internos do software e nos aspectos funcionais externos.
Manutenção	Modificações após o software ser liberado. Essas mudanças ocorrem por erros, adaptações, novos ambientes e novas funcionalidades.

Tabela 1. Fases do Desenvolvimento de Software

é produzida. Alguns desses itens são selecionados de acordo com sua relevância e necessidade de controle tanto de versão quanto de mudança. Esses itens selecionados são chamados *itens de configuração de software* e o conjunto dos mesmos compõe a *configuração de software* (Mahler, 1994).

A criação e as alterações de cada item de configuração de software devem ser acompanhadas e controladas pelo gerente do projeto. Para isso, é necessário o estabelecimento de pontos bem definidos dentro do processo de desenvolvimento do software: as *Linhas de Referência (baselines)*. Esses pontos podem ocorrer ao final de cada uma das fases do processo de desenvolvimento de software, ou de algum outro modo definido pela gerência.

Nos pontos estabelecidos pelas linhas de referência, os itens de configuração devem ser identificados, analisados, corrigidos, aprovados e armazenados como uma configuração de software. Os itens aprovados são armazenados em um local sob controle de acesso, denominado *repositório dos itens de configuração*. Esses itens só poderão ser alterados após uma solicitação de mudança formalmente aprovada pelo gerente de configuração. Essa é uma forma de prover controle sobre a situação de cada um dos itens de configuração, evitando inconsistências.

O método utilizado para trabalhar com itens de configuração que já estão no repositório é chamado de *Check In/Check Out* (Bersoff, 1979), ou seja, conferência na entrada e conferência na saída. Quando for desejada uma alteração em algum item de configuração do repositório, uma cópia do item é colocada numa área de trabalho do desenvolvedor (*"check out"*). A partir desse momento, nenhum outro desenvolvedor poderá alterar o mesmo item: a isso dá-se o nome de *controle de concorrência*. Dentro de sua área, o desenvolvedor tem total liberdade de trabalho. Após o final das alterações no item de configuração, ele será revisado e recolocado no repositório (*"check in"*). Nesse momento, uma nova *baseline* poderá ser estabelecida, de modo que uma nova configuração, contendo o item alterado, seja formada e armazenada no repositório (Figura 2).

Depois do armazenamento e da definição da baseline, o acesso é liberado, permitindo que outros desenvolvedores possam acessar e também executar alterações sobre esse item de configuração.

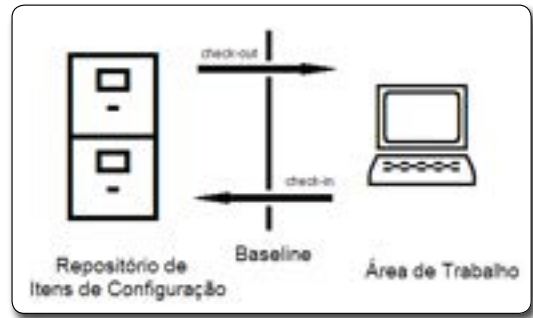


Figura 2. Controle de concorrência (check-in / check-out)

Conclusão

Neste artigo foram apresentados conceitos gerais da gerência de configuração de software. Além disso, foram ressaltados alguns dos benefícios que esta área provê, além de sua relação com o processo de desenvolvimento de software. ●

Referências

- (Bersoff, 1979) BERSOFF, E. H.; Henderson, V. D. e Siegel, S.G. Software Configuration Management: A tutorial. Los Alamos, Califórnia. IEEE Computer. v.12, n.1, 1979.
- (IEEE Std 828, 1998) IEEE for Software Configuration Management Plans. 1998.
- (Mahler, 1994) MAHLER, A. Variants: Keeping things together and telling them apart. In Configuration Management, Vol. 2 of Trends in Software, Wiley, New York, 1994.
- (Pressman, 2005) PRESSMAN, R. S. Software Engineering: a practitioner's approach. Mc Graw Hill Higher Educational, 6ª. Edição. 2005.
- (Versão 1.1), Associação para Promoção da Excelência do Software Brasileiro.
- "Software Configuration Management Handbook", Alexis Leon (2004)
- "Software Configuration Management Patterns: Effective Teamwork, Practical Integration", Stephen Berczuk e Brad Appleton (2002)
- ISO, 1995b, ISO/IEC 12207 - Information technology - Software life cycle processes, International Organization for Standardization.
- SOFTEX, 2006b, MPS.BR - Melhoria de Processo do Software Brasileiro - Guia de Implementação

Dê seu feedback sobre esta edição!

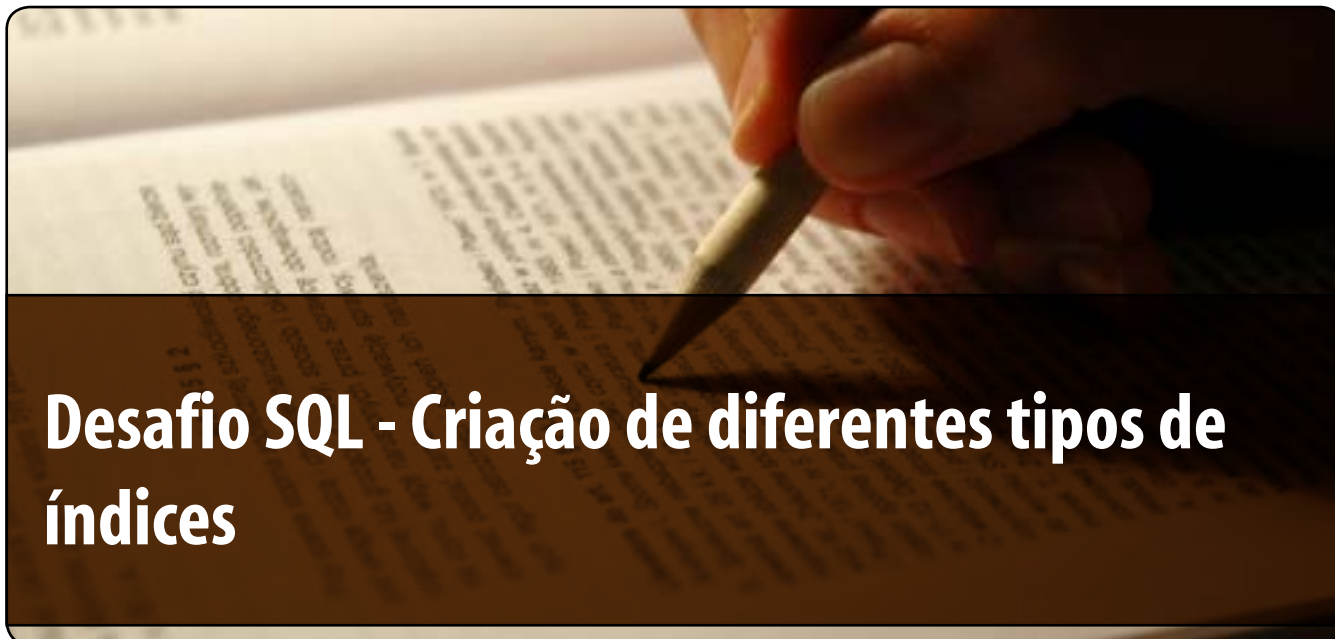
A SQL Magazine tem que ser feita ao seu gosto. Para isso, precisamos saber o que você, leitor, acha da revista!

Dê seu voto sobre este artigo, através do link:

www.devmedia.com.br/sqlmagazine/feedback



Nesta seção você encontra artigos sobre banco de dados, SQL ou persistência



Desafio SQL - Criação de diferentes tipos de índices

Novos problemas vão surgindo no sistema da empresa ItsMyBusiness.

Desta vez os analistas, nos testes de inserção de dados, encontraram um problema potencial de duplicidade de dados.

Os relatórios, consultas e sumarizações poderão ser gravemente impactados caso o problema não seja solucionado.

Este é o desafio deste mês, que Wagner Crivelini lhe traz.

Aproveitem.

Ricardo Rezende - Editor técnico



Wagner Crivelini

wcrivelini@gmail.com

Atua a mais de 15 anos na área TI, particularmente com Business Intelligence. Engenheiro formado pela UNICAMP, trabalha na IBM na unidade de IBM Global Account. Profissional com certificação em DB2 (IBM Certified Solution Designer DB2 Business Intelligence). Colunista convidado do portal internacional www.SQLServerCentral.com. Atualmente ocupa a posição de DBA DB2.

Neste mês, o trabalho na empresa fictícia ItsMyBusiness está bem mais calmo. Mas, como sempre acontece, a empresa não está parada. O projeto de melhoria do controle de pedidos está terminando e logo tudo estará em produção (**Nota DevMan 1**).

Atualmente, o modelo do banco de dados da empresa está da maneira como mostra a **Figura 1**.

O script para criação desta base de dados está disponível no portal da SQL Magazine. Este script adota um SQL padrão, de tal modo que ele roda, com pequenas alterações, em SQL

De que se trata o artigo?

Desenvolvimento de soluções para problemas cotidianos enfrentados por DBAs e desenvolvedores de aplicações para banco dados.

Para que serve?

Fornecer conceitos de utilização de funcionalidades do padrão SQL ANSI na resolução de problemas enfrentados no dia-a-dia na recuperação de informações do banco de dados.

Em que situação o tema é útil?

Criação de diferentes tipos de índices.

SERVER, DB2, ORACLE, FIREBIRD e POSTGRES. As alterações necessárias para cada SGBD são apresentadas como comentários.

Mas o que nos interessa no Desafio SQL deste mês é que uma das razões para o projeto ainda não ter terminado é que, durante os testes de inserção de dados, se descobriu que o sistema permite o cadastramento de dois ou mais produtos com o mesmo nome. Cada registro terá um ID diferente, mas ainda assim a descrição dos produtos pode ser idêntica.

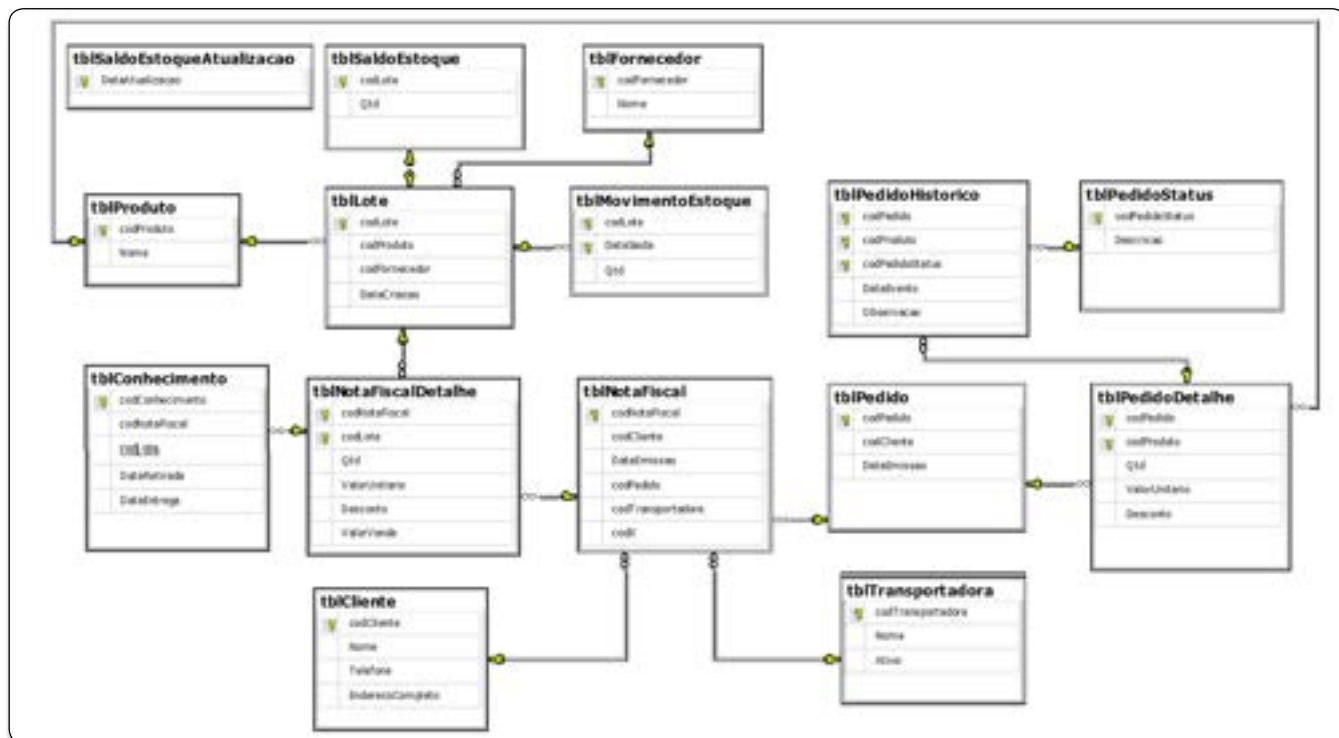


Figura 1. Modelo de dados simplificado da empresa ItsMyBusiness

Para fins de ilustração, a **Listagem 1** mostra o script de definição da tabela de produtos.

Listagem 1. Criação da tabela de produtos

```
1 CREATE TABLE dbo.tblProduto(
2   codProduto int NOT NULL,
3   Nome varchar(100) ,
4   PRIMARY KEY (codProduto));
```

No caso da ItsMyBusiness, assim como seria na maioria das empresas, este comportamento é completamente indesejável.

Ter no banco de dados vários códigos para um mesmo produto cria uma duplicidade indesejável nos dados e certamente será fonte de erros. Para ilustrar a questão, cito alguns exemplos:

1. quando um cliente fizer um pedido pelo site, qual dos “códigos válidos” daquele produto será considerado?
2. quando analisarmos o histórico das vendas do produto, quais dos “códigos válidos” vamos considerar para geração do relatório?
3. quando consultarmos o saldo disponível em estoque, qual dos “códigos válidos” vamos considerar?

Pois bem. Cada resposta que escolhermos para as perguntas acima vai gerar ações diferentes... E novos problemas.

Por exemplo: imagine o caso da geração de um relatório com histórico de vendas dos produtos, conforme o exemplo no item 2. Vamos supor que escolhemos gerar um relatório mostrando separadamente o histórico de vendas por código de produto.

Isso vai causar um problema sério para os analistas de negócio que usarem este relatório porque, para eles, não interessa como o banco de dados trata o produto X. Eles precisam de informações sumarizadas de todas as transações que aconteceram envolvendo este produto.

Por outro lado, imagine agora que decidimos atender a solicitação dos analistas de negócio e geramos relatórios sumarizados por descrição de produto ao invés dos seus códigos. Aí o problema muda de lado.

Vamos pensar nos detalhes técnicos da geração deste relatório. Nós faremos uma consulta no banco de dados agrupando as transações com base na descrição do nome do produto. Veja que agora não basta que o mecanismo do banco de dados trate uma massa de dados que inclua o campo *codProduto*, que é um campo numérico que ocupa apenas 4 bytes por registro.

Toda operação vai se basear no campo *tblProduto.NOME*, que é um campo texto de tamanho variável (varchar) com até 100 bytes por registro.

Não bastasse este tremendo “peso extra”, devemos lembrar que os campos de códigos são, geralmente, indexados, enquanto os campos texto nem sempre recebem indexação, forçando o banco de dados a executar um Full Table Scan (Nota **DevMan 1**).

E um terceiro ponto: para conseguir cruzar as transações com os nomes de produtos, precisamos ainda fazer uma junção adicional de tabelas, incluindo a tabela *tblProduto*.

A esta altura, espero que você já tenha se convencido de que esta consulta não terá nem de longe a mesma performance que teria uma consulta semelhante baseada nos códigos.



Nota do DevMan 1

Full Table Scan

Full Table Scan é a operação executada no banco de dados em que o mesmo efetua uma varredura em todos os dados da tabela, “lendo” sequencialmente todas as linhas.

Nesta varredura, o banco de dados exclui da seleção as linhas que não atendem ao critério de seleção (cláusula WHERE da consulta).

Nem sempre um Full Table Scan é um “mau negócio”, pois quando a consulta busca a maior parte das linhas da tabela, a performance será melhor em um Full Table Scan se comparado a efetuar a leitura do índice e depois dos dados.

Nos casos em que não exista um índice apropriado para a consulta, um Full Table Scan será “forçado” e, na grande maioria das vezes, a performance não será aceitável.

Resumindo: no mundo real, é muito provável que você tenha vários tipos de problema se o seu banco de dados aceitar entradas duplicadas de nomes de produto.

Tendo entendido a situação, vamos a uma segunda má notícia: os mesmos testadores da ItsMyBusiness descobriram que a base aceita também duplicidade de cadastramento de clientes, fornecedores e transportadoras.

Nós precisamos resolver estes problemas antes que o novo sistema vá para produção. Sua missão é criar uma estratégia para solucionar esta questão. E você deve tratar todas as entidades envolvidas, a saber:

- produtos;
- clientes;
- fornecedores;
- transportadoras.

Resposta do desafio

Ao contrário do que possa parecer, duplicidade de registros lamentavelmente é muito mais comum do que você possa imaginar. Eu mesmo já me deparei com isso em vários clientes onde trabalhei.

O grande problema é que muitos arquitetos de dados não dão a devida atenção a esta questão. Em resumo, o problema é de qualidade de dados e não de modelagem.

É muito frequente vemos arquitetos e DBAs preocupados com modelo de dados, performance e integridade referencial. Porém, mesmo que nós cuidemos destes tópicos, podemos esquecer de detalhes e aí sim, teremos o problema da duplicidade que vimos aqui.

Observe, por exemplo, a tabela de produtos na **Figura 1** (tblProduto). Esta tabela está devidamente normalizada e tem uma chave primária (codProduto). Podemos observar também que esta chave é usada para definição de uma chave estrangeira na tabela de detalhes do pedido (tblPedidoDetalhe). Você vai notar que acontece a mesma coisa com as tabelas de clientes, fornecedores e transportadoras (nota **DevMan 2**).

Basicamente, o modelo de dados inteiro não permite redundância de informações, porque está devidamente normalizado (3a Forma Normal, no mínimo). A performance das consultas também foi lembrada. Por conta disso, todas as



Nota do DevMan 2

Normalização, Chave Primária e Chave Estrangeira

Normalização:

O objetivo da normalização de um banco de dados é evitar os problemas que podem provocar falhas no projeto do banco de dados, bem como eliminar a mistura de assuntos e as correspondentes redundâncias dos dados desnecessárias.

O processo de normalização aplica uma série de regras sobre as tabelas (também chamadas de relações) de um banco de dados, para verificar se estão corretamente projetadas.

Existem diversas formas normais. Entre as mais conhecidas, temos: a primeira forma normal, a segunda forma normal, a terceira forma normal e a forma normal de Boyce-Codd.

Chave Primária:

Chaves primárias (em inglês Primary Keys ou PK) sob o ponto de vista de um banco de dados relacional, referem-se às tuplas (conjuntos) de um ou mais campos, cujos valores, considerando a combinação de valores de todos os campos da tupla, nunca se repetem e que podem ser usadas como um índice para os demais campos da tabela do banco de dados. Em chaves primárias, não pode haver valores nulos nem repetição de tuplas.

Simplificando, quando a chave primária é simples, ou seja, é formada por um único campo da tabela, esse campo não pode ter dois ou mais registros de mesmo valor, e também não pode conter nenhum registro nulo. Se a chave primária é composta, ou seja, formada por mais de um campo, os valores de cada campo podem se repetir, mas não a combinação desses valores.

Chave Estrangeira:

O conceito de Chave estrangeira em uso de banco de dados se refere ao tipo de relacionamento entre as tabelas de dados do banco de dados.

Uma chave estrangeira é chamada quando há o relacionamento entre duas tabelas.

Uma chave estrangeira é um campo, que aponta para a chave primária de outra tabela. Ou seja, passa a existir uma relação entre essas duas tabelas. A finalidade da chave estrangeira é garantir a integridade dos dados referenciais, pois apenas serão permitidos valores que supostamente vão aparecer na Base de Dados.

Esse tipo de atributo não permite exclusão, modificação e/ou inserção de dados em tabelas que estejam dependentes umas das outras (foreign key), o que requer modificadores especiais, como cascade, por exemplo. Isso também exige uma maior atenção do administrador da base de dados, quanto à própria manipulação dos dados.

tabelas possuem uma chave primária definida sobre campos numéricos (que é um cuidado muito importante quando se trata de chaves primárias). E também se tratou da integridade referencial dos dados, havendo chaves estrangeiras entre as tabelas relacionadas. Isso garante que não se possa cadastrar nas tabelas fatos códigos que não existam nas respectivas tabelas cadastrais.

Naturalmente, todos estes cuidados ajudam muito para garantir a qualidade dos dados, mas ainda assim, não evitaram o problema que foi identificado.

Então como resolver isso?

Nós precisamos criar uma restrição que impeça o cadastramento de valores duplicados. E isso existe como parte da linguagem SQL.

A linguagem SQL define vários tipos de índice. Índices são estruturas de dados usadas para acelerar as operações de recuperação de registros numa base. Eles funcionam de modo muito parecido com o índice de um livro: temos um dicionário com os dados que estão sendo indexados e as referências das páginas onde os dados são encontrados.

Por exemplo, num livro em que se fala de animais, podemos criar uma lista de todos os animais mencionados no texto, de A a Z. E vamos também apontar no índice em quais páginas do livro cada uma destas palavras parecem.

No caso dos índices de bancos de dados, os valores listados nas páginas de índice são os valores dos dados que existem no(s) campo(s) indexado(s). E as “páginas” referenciadas são, na verdade, os endereços para as páginas de dados gerenciadas pelo SGBD.

Agora vamos falar sobre os tipos de índices. Para falarmos apenas dos mais comuns, podemos classificá-los como clusterizados ou não, únicos ou não.

Os não-clusterizados apenas montam este “dicionário” que eu exemplifiquei acima tomando por base as páginas de dados disponíveis.

Já no caso dos índices clusterizados, acontece algo bem diferente. O índice não “consulta” as páginas de dados, mas as reescreve! Ou seja, a própria ordenação dos dados nas páginas de dados é alterada para acompanhar a ordenação do índice. Mal comparando, é como se o seu livro fosse reescrito para que as páginas do livro que contém as palavras indexadas apareçam na mesma sequência das próprias palavras que fazem parte do índice.

No que diz respeito aos índices únicos ou não-únicos, estamos informando se o SGBD aceita ou não a inserção de novos registros que tenham no campo indexado um valor já usado em outro registro.

Um detalhe que não pode passar despercebido: por definição, cada tabela aceita apenas um índice clusterizado.

Importante destacar que ao implementarmos a chave primária de uma tabela, estamos criando automaticamente um índice clusterizado também (mas o inverso não é verdadeiro: a tabela pode ter o índice clusterizado e não ter uma chave primária implementada, o que é um problema para integridade referencial do banco).

Tudo entendido, terminado nosso resuminho sobre índices, voltemos ao que realmente nos interessa aqui: como vamos resolver o problema de duplicidade de produtos da ItsMyBusiness?

Bom, vamos aplicar ao nosso caso os conceitos que vimos acima.

Chaves primárias não permitem duplicidade de valores e nem valores nulos, exatamente como acontece com o campo `tblProduto.NOME`. Mas já existe esta chave nesta tabela. Por isso, nosso campo terá que se contentar com o papel de chave alternativa (nota **DevMan 3**).



Nota do DevMan 3

Escolhendo a chave primária e chave alternativa

Eventualmente, pode existir numa tabela mais de um campo que seja um possível candidato a chave primária.

Todos estes campos são chamados de chaves candidatas, mas apenas um deles será chave primária. E os demais, aqueles que não forem escolhidos, serão chamados de chaves alternativas.

No exemplo que estamos estudando, temos na tabela `tblProduto` duas chaves candidatas: os campos `codProduto` e `Nome`. O que nos impede de escolher um campo ao invés do outro?

Devemos lembrar que a chave primária é sempre um índice clusterizado e isso tem ao menos dois impactos importantes na análise que vamos fazer:

1. os dados da tabela são fisicamente escritos de maneira ordenada em relação a este índice.
2. todos os demais índices que forem criados na tabela serão dependentes do seu índice clusterizado se este último existir.

Vamos analisar o primeiro item. Imagine que a sua tabela terá naturalmente que receber novas inserções de dados. Como um índice clusterizado define que os registros são fisicamente escritos nas páginas de dados gerenciadas pelo SGBD respeitando a ordenação do campo indexado, se tivermos um campo alfanumérico como chave primária (`tblProduto.Nome`) e formos inserir um registro novo para o produto “repimboca de parafuseta”, o SGBD terá que localizar em qual página está escrito o registro que contenha o produto com o nome imediatamente anterior a este na ordem alfabética e achar um espaço nesta página para escrever o novo registro. Se não houver espaço suficiente, todas as páginas de dados seguintes deverão ser reescritas em tempo real para que o SGBD aceite este comando `INSERT`.

Agora vamos pensar no segundo item que eu listei. Se todos os novos índices vão se basear no índice clusterizado, no caso de escolhermos mal a chave primária da tabela, todos os índices que forem definidos para esta tabela também serão afetados.

Obviamente, indexar um campo texto de tamanho variável (`varchar`) com até 100 bytes por registro é muito mais caro do ponto de vista do banco de dados do que indexar um campo numérico de 4 bytes por registro.

O “tamanho do índice”, ou melhor, o espaço destinado às páginas que compõem este índice, será muito maior (mais páginas). Além disso, a velocidade do índice será mais baixa, já que o SGBD terá que pesquisar muito mais páginas quando for utilizar este índice.

Nestas alturas, a solução é óbvia. Temos que reforçar o papel de chave alternativa que atribuímos ao campo `tblProduto.Nome`. Precisamos definir um índice que impeça inserção de valores duplicados e que não seja um índice clusterizado, pois a tabela já tem a chave primária.

Tecnicamente, este terá que ser um índice único não-clusterizado. A solução é a mesma para todas as quatro tabelas que estamos tratando. A **Listagem 2** mostra o script para criação de todos estes índices.

Uma vez executado o script acima, veremos que o problema de duplicidade não ocorre mais. Ao executarmos uma declaração de `INSERT` ou `UPDATE` na tabela, o SGBD recusará a

operação e vai retornar uma mensagem de erro informando que não podem existir valores duplicados no campo que indexamos. Será uma mensagem parecida com esta: "Cannot insert duplicate key row in object 'dbo.tblProduto' with unique index 'ixProdutoNome'".

Listagem 2. Implementando índices únicos não-clusterizados

```
1 CREATE UNIQUE INDEX ixProdutoNome
2 ON dbo.tblProduto(Nome);
3 CREATE UNIQUE INDEX ixClienteNome
4 ON dbo.tblCliente(Nome);
5 CREATE UNIQUE INDEX ixFornecedorNome
6 ON dbo.tblFornecedor(Nome);
7 CREATE UNIQUE INDEX ixTransportadoraNome
8 ON dbo.tblTransportadora(Nome);
```

Mas tome cuidado.

Dependendo das propriedades de agrupamento de dados do seu banco (conhecido como Collation), o SGBD pode fazer distinção entre letras maiúsculas e minúsculas. E aí o banco

vai entender como produtos distintos e aceitar a inserção de dois registros com os valores "repimboca de parafuseta" e "Repimboca da Parafuseta".

Em geral, criamos os bancos de dados de modo a não fazer esta distinção (collation case insensitive) e daí não teremos este problema.

Chegamos ao fim de outro Desafio SQL. Espero que tenha gostado!

Abraço e até a próxima. ●

Dê seu feedback sobre esta edição!

A SQL Magazine tem que ser feita ao seu gosto. Para isso, precisamos saber o que você, leitor, acha da revista!

Dê seu voto sobre este artigo, através do link:

www.devmedia.com.br/sqlmagazine/feedback



Chegou o Sistema de Créditos DevMedia

Agora o **conteúdo completo** do nosso
site está **ao seu alcance!**

- A partir de agora todas as **2000 vídeo-aulas** do site DevMedia podem ser compradas individualmente.
- **Economia** - Você não precisa mais assinar oito produtos diferentes para ter acesso ao conteúdo completo do site!

Todas as vídeos que você sempre
quis ver a partir **R\$0,75!**

Saiba mais sobre o Sistema de Créditos!

www.devmedia.com.br/creditos



Nossos **servidores** dedicados possuem monitoramento 24x7x365, link dedicado e **full duplex**, ou seja, os servidores atingem a **velocidade** do link tanto para **upload** quanto para **download**.

HOST

Servidor
Sites
Gerenciamento



DATACENTER

Co-location
E-mail
Backup



TELECOM

Cross Connection
Banda IP
Link PAP



SOLUÇÕES INTELIGENTES

Solicite um representante comercial

www.localdatacenter.com.br

SOLICITE UM ORÇAMENTO
TEL.: 3956 - 4900
comercial@localdatacenter.com.br



O seu relacionamento está desbalanceado?



Experimente o diferencial FairCom: Tecnologia de banco de dados flexível, licenciamento flexível

- Comprovada em aplicativos de missão crítica por mais de 30 anos
- Um único produto, suporte a mais de 30 sistemas operacionais
- Utilize as APIs SQL e ISAM simultaneamente
- Baixo TCO (Custo total de propriedade) e licenciamento flexível para atender ao seu modelo de negócios
- SDK de desenvolvimento grátis
- Suporte técnico de engenheiros para engenheiros



Então é hora de conhecer o:

c-treeACE™
the right-size Database 

Baixe seu SDK grátis • www.faircom.com • 11-3872-9802